GRUPO JAP

ONBOARDING+

FORMAÇÃO 04 | ASP.NET CORE MVC AVANÇADO

Carlos Freire
MCSD | Microsoft Certified Solution Developer
Software Architecture

GRUPO JAP

Roadmap

4 – ASP.NET CORE MVC ADVANCED

OBJETIVO:

Esta formação tem o objetivo de apresentar conceitos mais avançados do ASP.NET Core MVC.

E irá capacitar você a trabalhar em nível mais profissional e conseguir entregar mais valor.

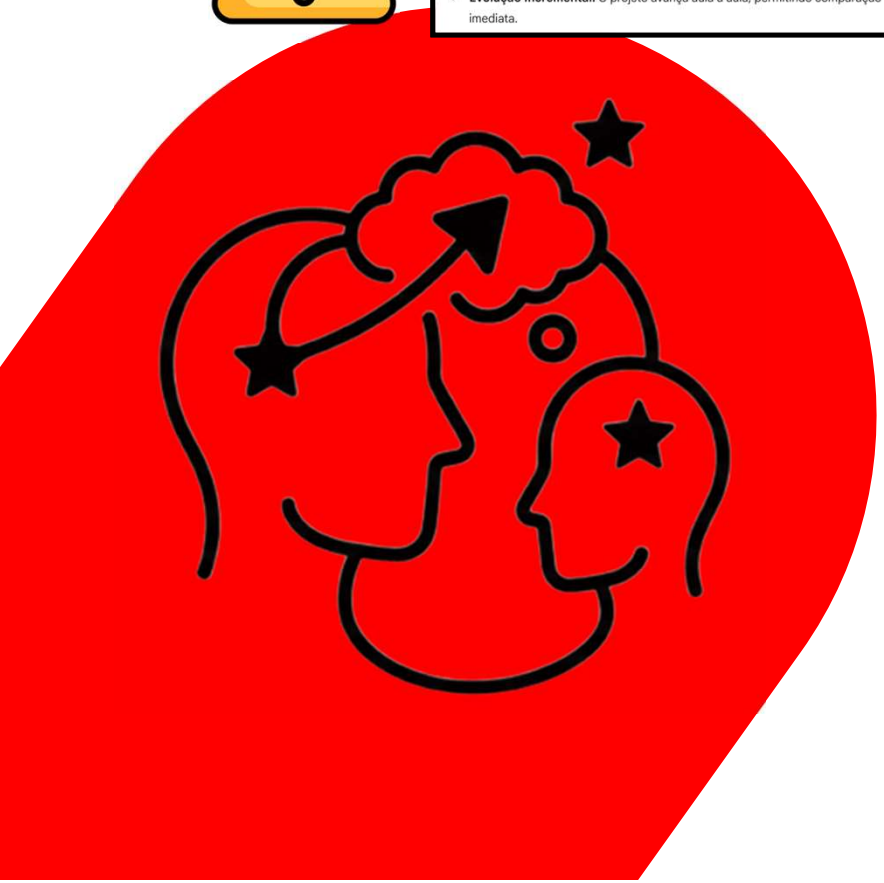
Nível de Complexidade

★★☆ Intermédio

GRUPO JAP

Metodologia de Aprendizado:

- **Ciclo de Entrega:** Cada tópico é composto por Teoria + Demo, encerrando uma videoaula.
- **Snapshot de Código:** Uma pasta exclusiva com o código-fonte final para cada aula gravada.
- **Evolução Incremental:** O projeto avança aula a aula, permitindo comparação imediata.

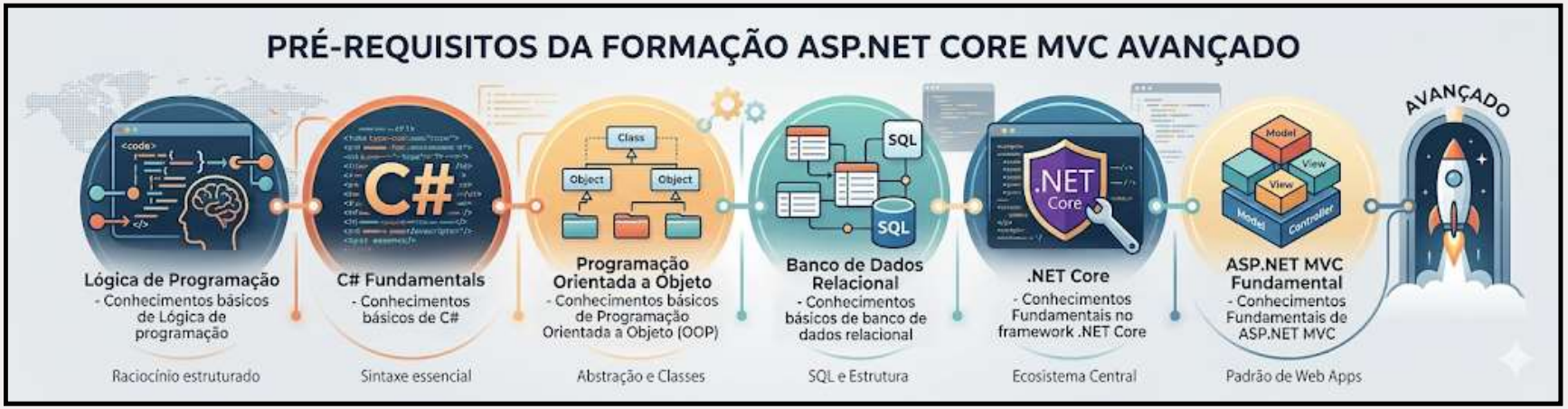


Tópicos da formação:

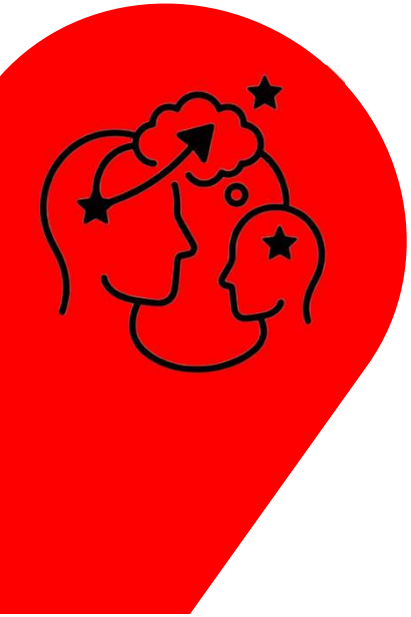
- **Pré-Requisitos**
- **Módulo: Aprofundar no Essencial**
  - Pipeline do ASP.NET
  - Criar projeto ASP.NET (Core Empty)
  - [DEMO-01] – Criar projeto do Zero (ou quase)
  - Ferramentas de Front-end
  - [DEMO-02] – Ferramentas de Front-end
  - O que são Bundling e Minication?
  - [DEMO-03] – Bundling e Minication
  - Trabalhar com mais Layouts
  - [DEMO-04] – Utilizar Layout Admin
  - Trabalhar com Tag Helper Customizado
  - [DEMO-05] – Criar um CRUD e Tag Helper
  - Trabalhar com View Component
  - [DEMO-06] – Implementar o View Component
  - Roteamento Inteligente
  - [DEMO-07] – Rotas Inteligentes
- **Módulo: Injeção de Dependência (DI)**
  - Como funciona o DI no ASP.NET Core
  - Como funciona Ciclos de Vida do DI
  - Configurar a DI no ASP.NET Core
  - Testar os 3 Tipos DI no ASP.NET Core
  - [DEMO] – O Maravilhoso mundo das DIs
- **Módulo: Segurança**
  - CSRF (Cross-Site Request Forgery)
  - XSS (Cross-Site Scripting)
  - HSTS (HTTPS Strict Transport Securiry)
  - [DEMO] – CSRF + XSS + HSTS = Segurança
  - Explorar ASP.NET Identity (Authentication)
  - [DEMO] – Identity | Authentication
  - Explorar o Identity | Authorization | Roles
  - Authorization | Policies Required Roles
  - Authorization | Policies Required Claims
  - Custom Authorization
  - [DEMO] – Identity | Authorization
- **Módulo: Configurações Avançadas**
  - Organizar a classe program.cs
  - Configurar Ambientes
  - [DEMO] – Organizar e Configurar
  - Ler ficheiro de configuração
  - Utilizar User Secrets (secrets.json)
  - [DEMO] – Ler ficheiro e User Secrets
  - Tratamento de Erros e Middlewares
  - [DEMO] – Tratamento de Erros
  - Logs para Observabilidade e Monitoria
  - [DEMO] – Observabilidade e Monitoria
  - Auditoria/Log através de Filtro
  - [DEMO] – Monitoria com Filtro
- **Módulo: Globalização da Aplicação**
  - Trabalhar com Culturas
  - Ajustar validações do front-end
  - Criar Data Annotation Customizado
  - Adicionar Suporte para mais Culturas
  - [DEMO] – WebApp Multi-Linguagem
- **Módulo: Conhecimentos Indispensáveis**
  - Trabalhar com Ajax e Modal Window
  - [DEMO] – Modal Window
  - Políticas de Cookies
  - [DEMO] – Políticas de Cookie
  - Upload de ficheiro (Create)
  - Upload de ficheiro (Edit)
  - [DEMO] – Upload de ficheiro
  - Tarefas em Background Services
  - [DEMO] – Adicionar Watermark
- **Encerramento**

GRUPO JAP

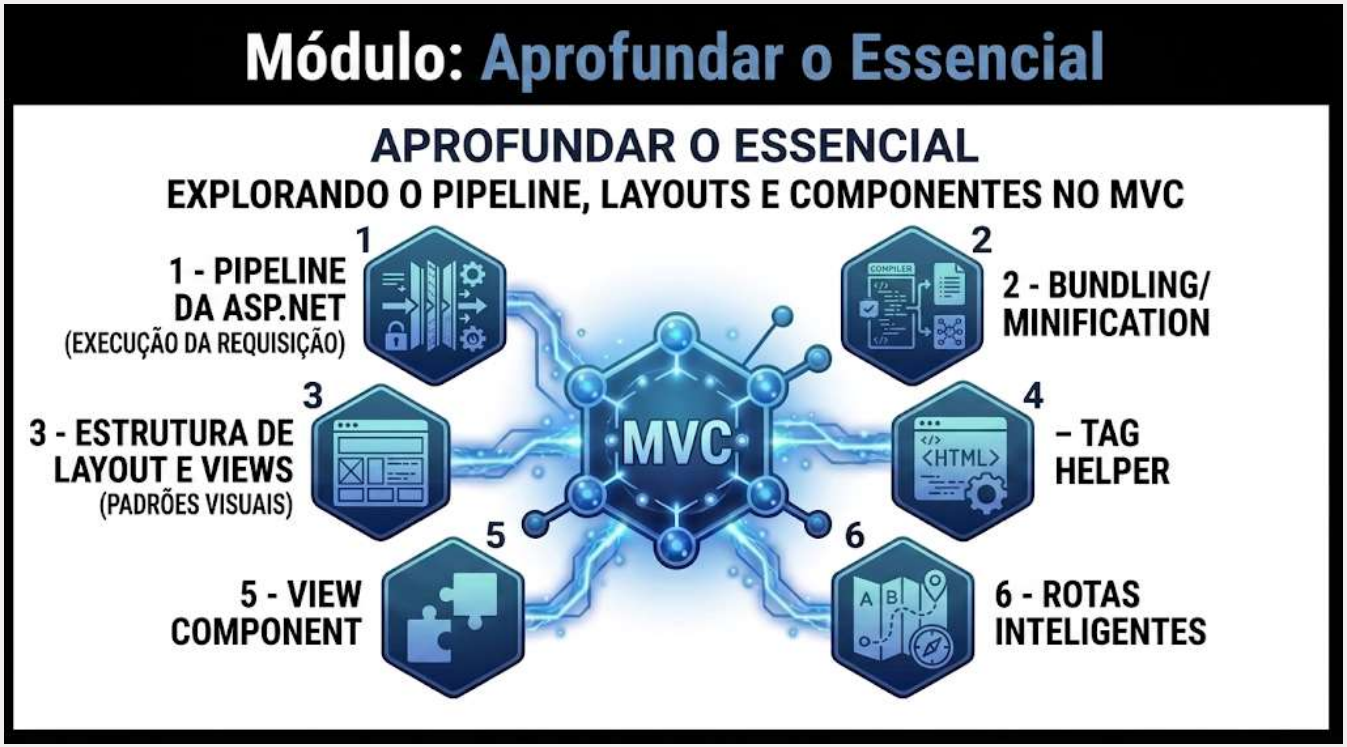
Pré-Requisitos



GRUPO JAP



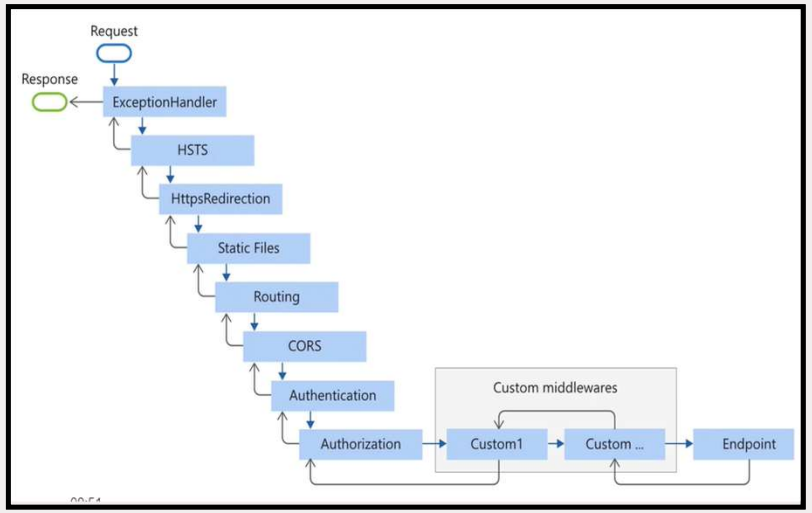
Módulo: Aprofundar o Essencial

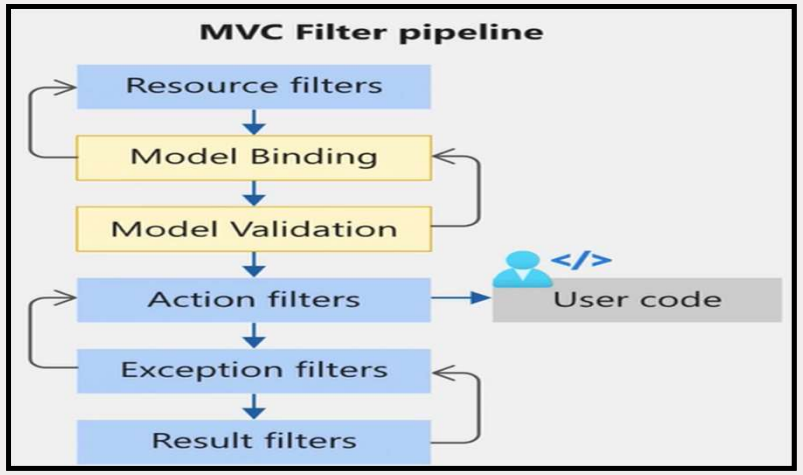


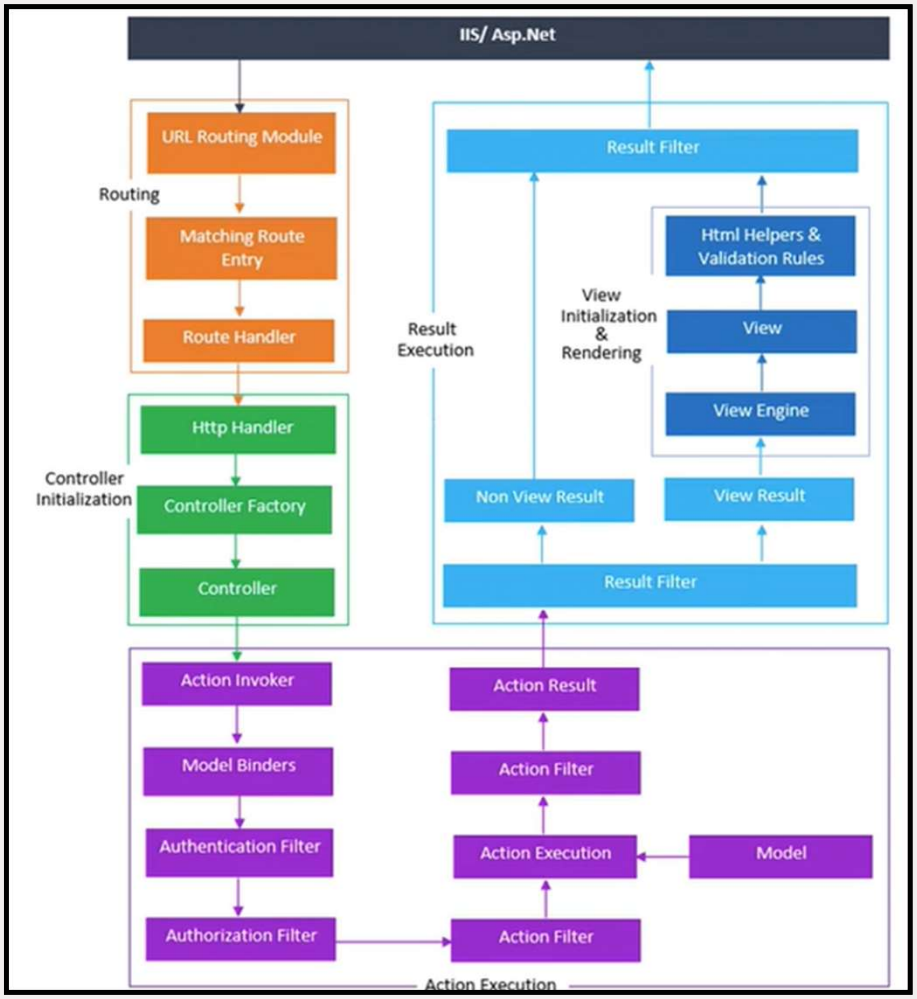
GRUPO JAP

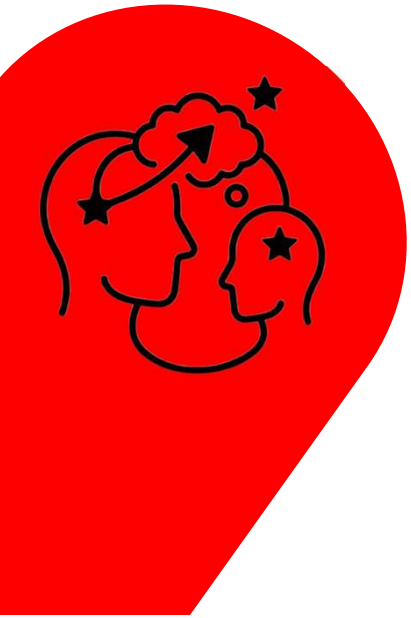
Pipeline do ASP.NET

https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/?view=aspnetcore-10.0



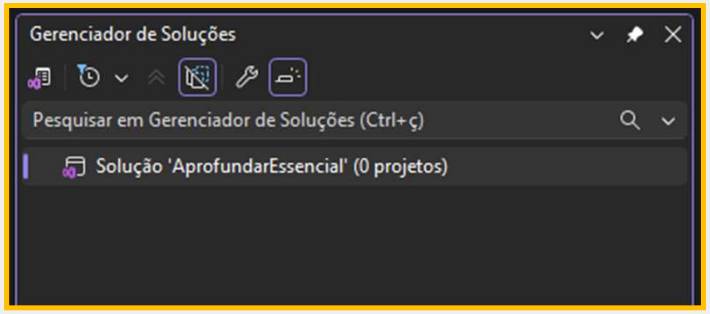


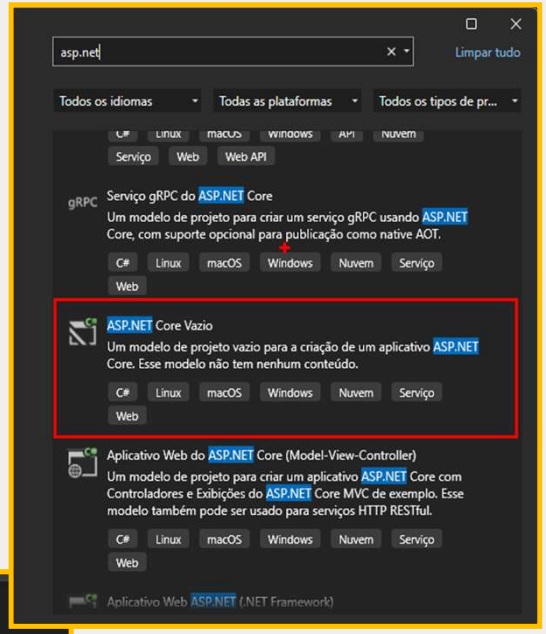


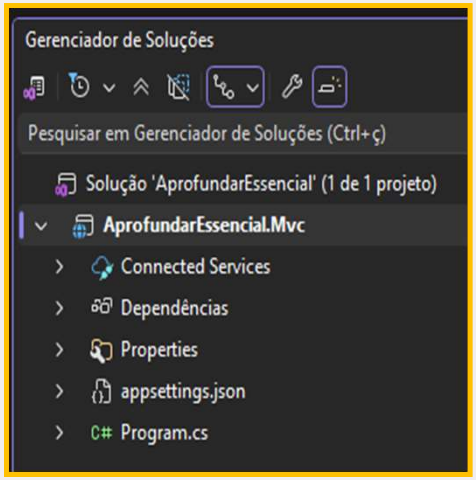


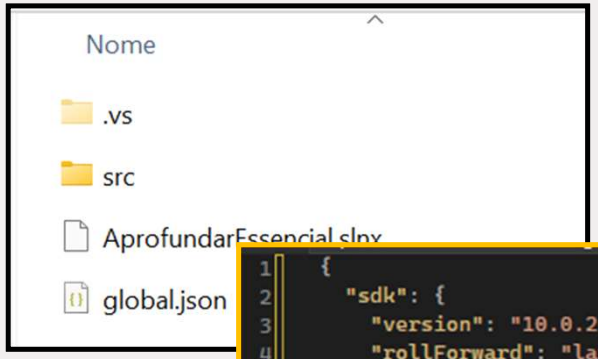
GRUPO JAP

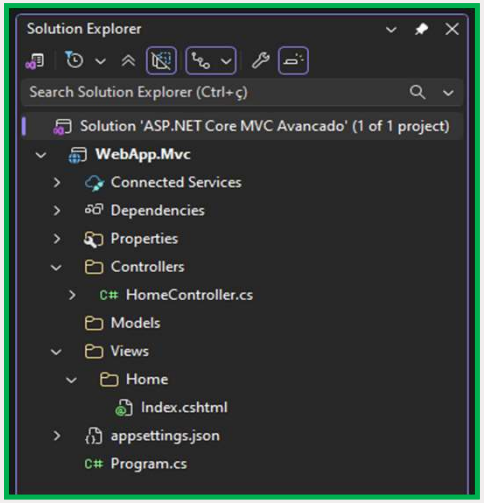
Criar projeto ASP.NET (Core Empty)

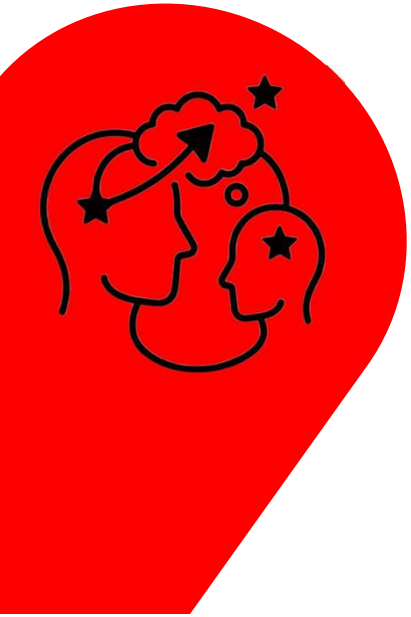






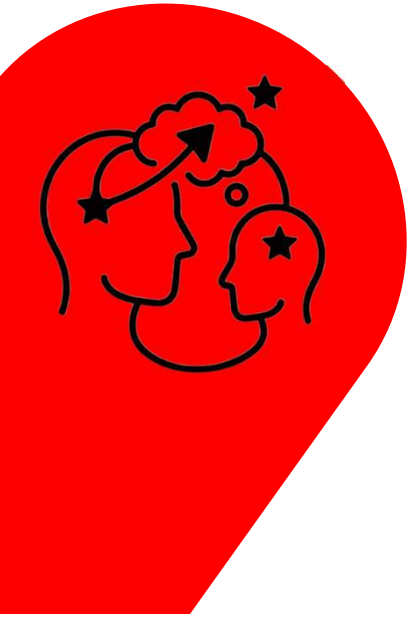






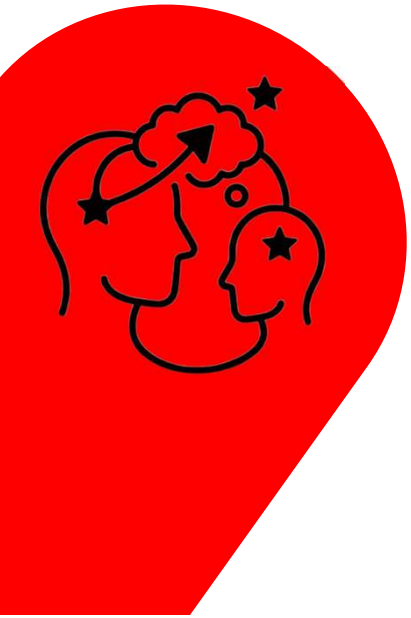
GRUPO JAP

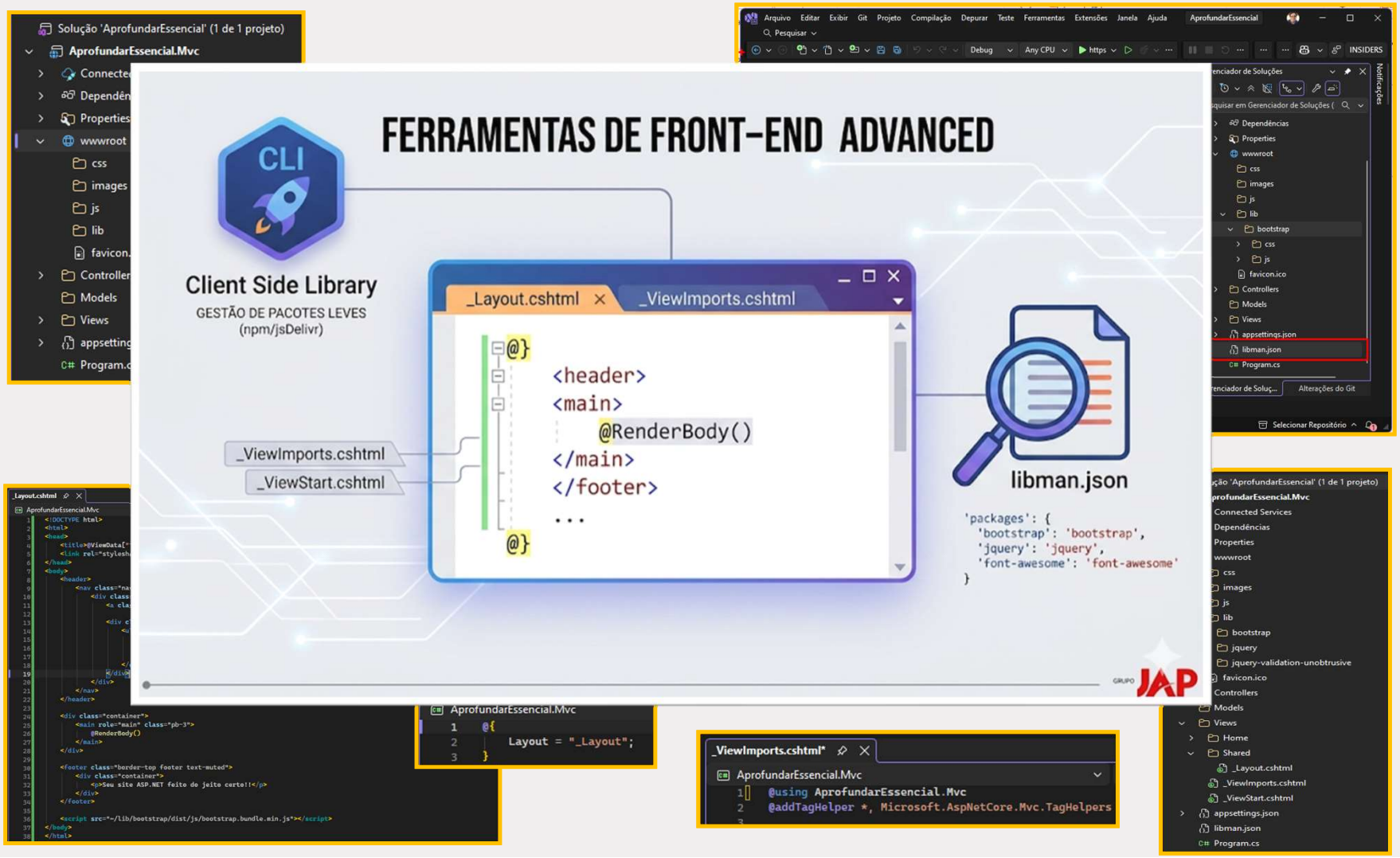
[DEMO] - Criar projeto do Zero (ou quase)



GRUPO JAP

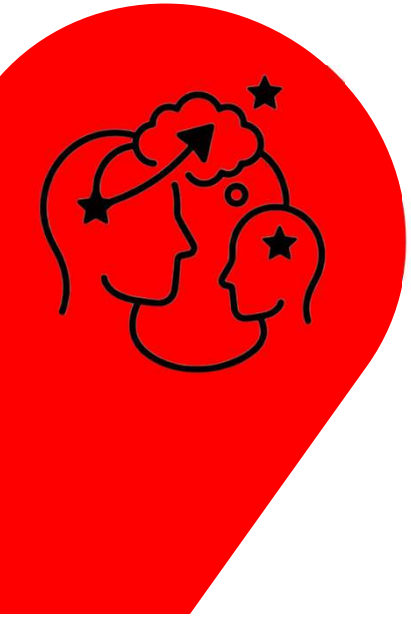
Ferramentas de Front-End

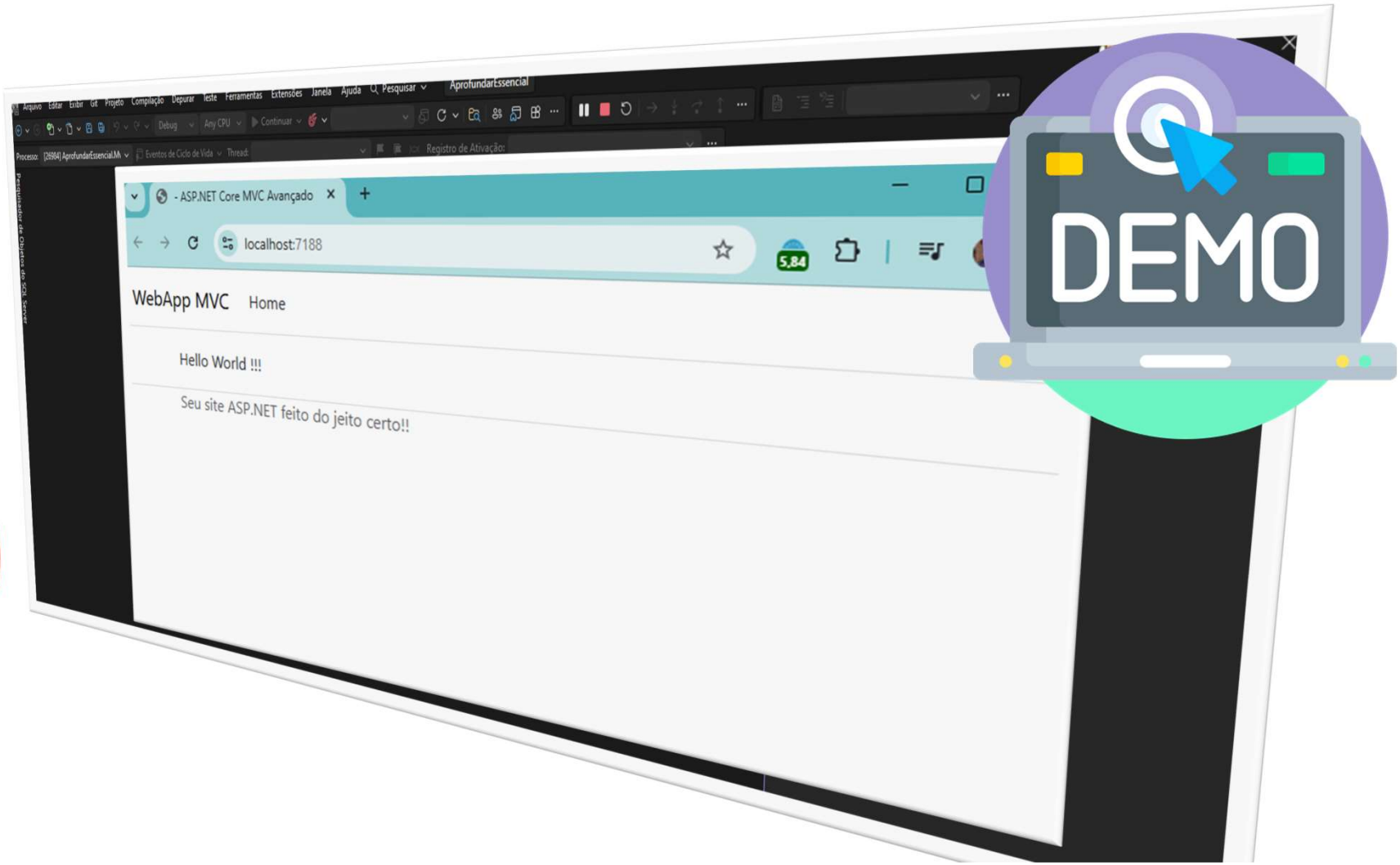




GRUPO JAP

[DEMO] - Ferramentas de Front-end





GRUPO JAP

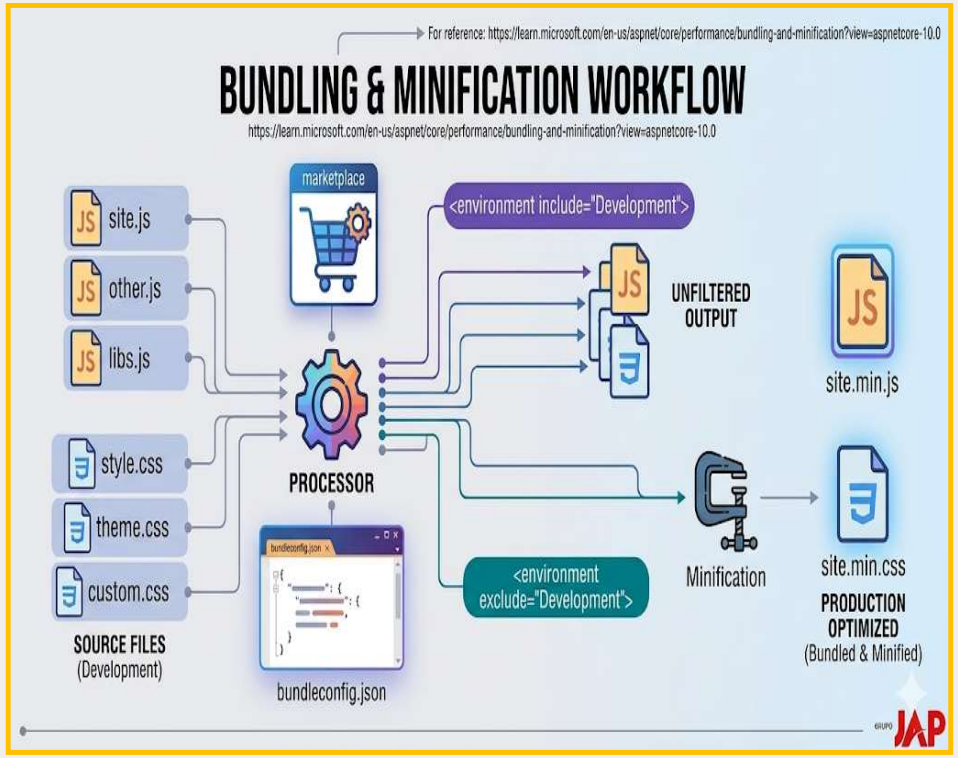
O que são Bundling e Minication?

Visão geral criada por IA

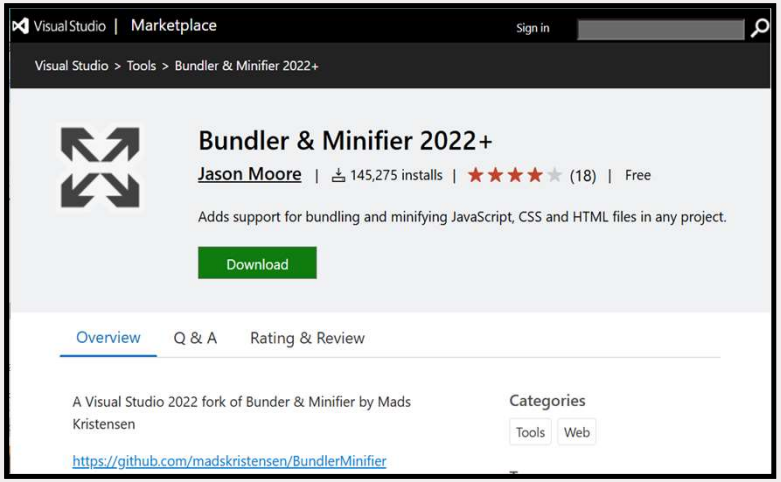
O **Bundling** (agrupamento) no ASP.NET é uma técnica de otimização de desempenho que combina vários arquivos CSS ou JavaScript em um único arquivo. Ele reduz o número de requisições HTTP necessárias para carregar uma página web, acelerando o tempo de carregamento no navegador do usuário. Microsoft Learn +3

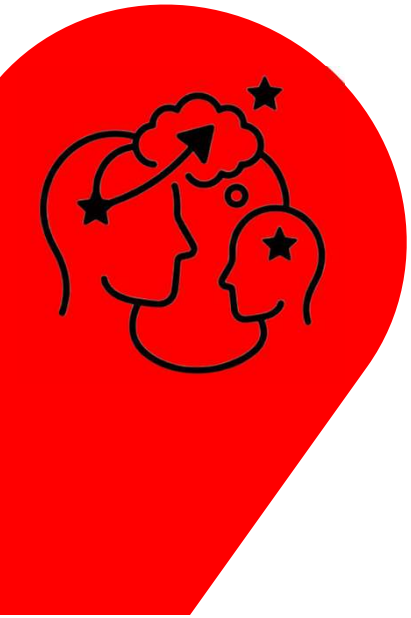
Visão geral criada por IA

Em ASP.NET, **minification** (minificação) é uma técnica de otimização que reduz o tamanho de arquivos CSS, JavaScript e HTML, removendo caracteres desnecessários como espaços em branco, comentários e quebras de linha, sem alterar a funcionalidade do código. DevMedia +2



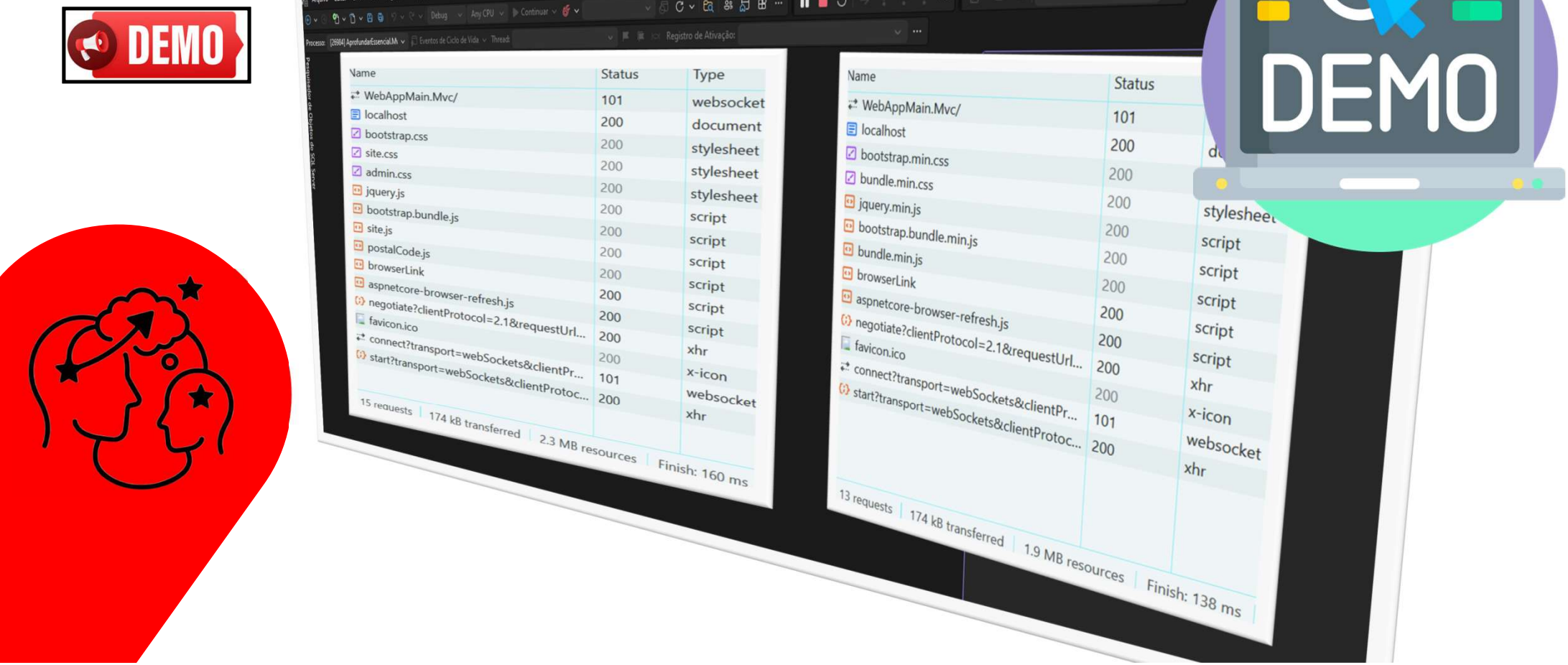
https://marketplace.visualstudio.com/items?itemName=Failwyn.BundlerMinifier64





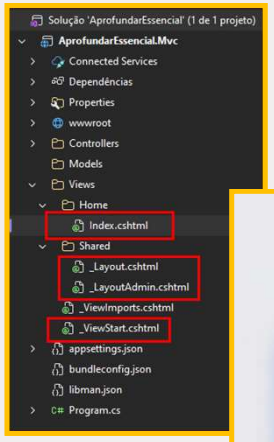
GRUPO JAP

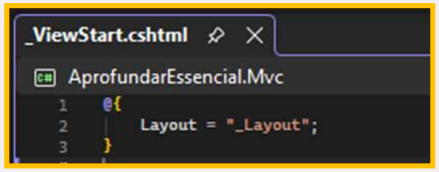
[DEMO] - Bundling & Minification

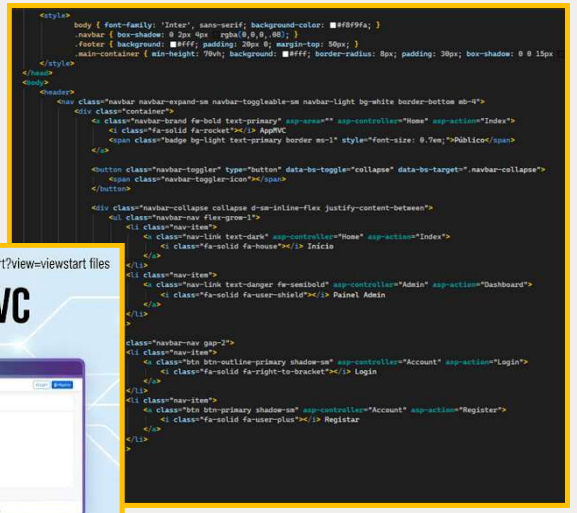


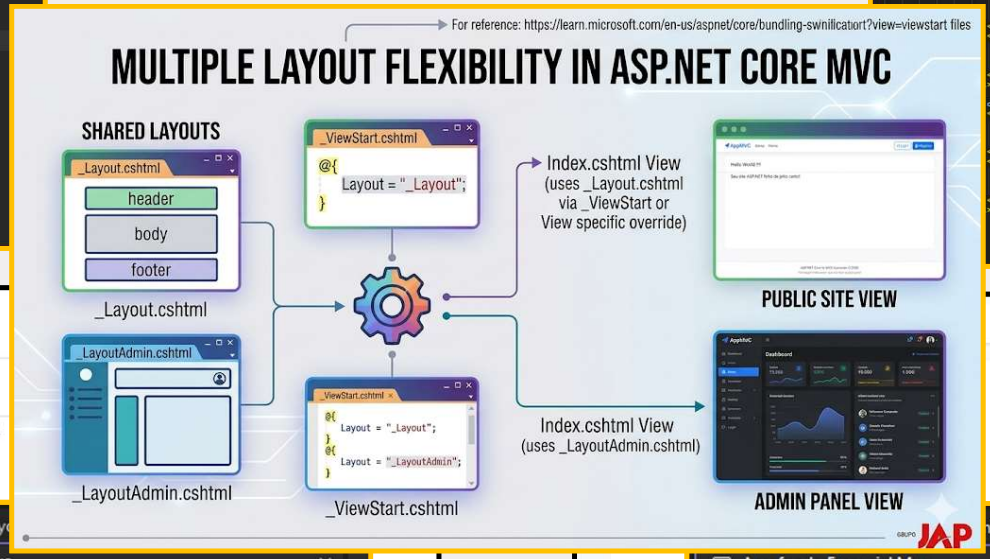
GRUPO JAP

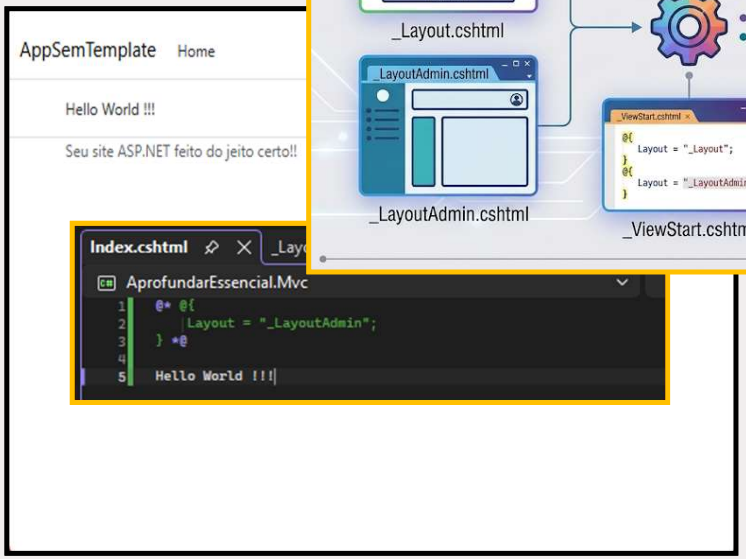
Trabalhar com mais Layouts

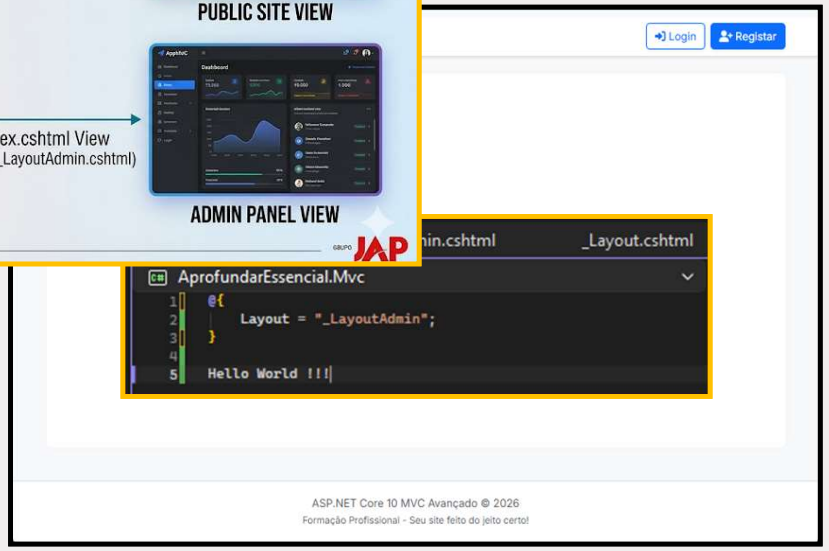


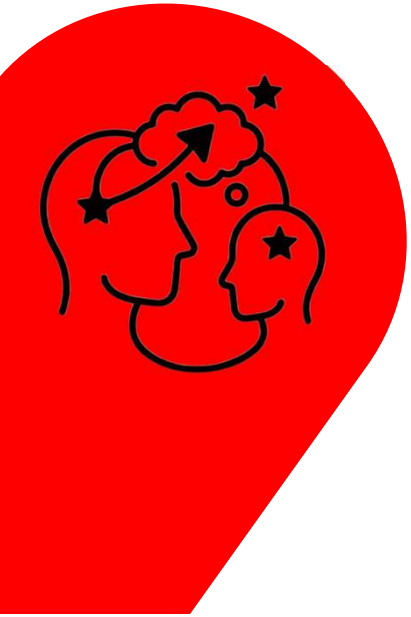






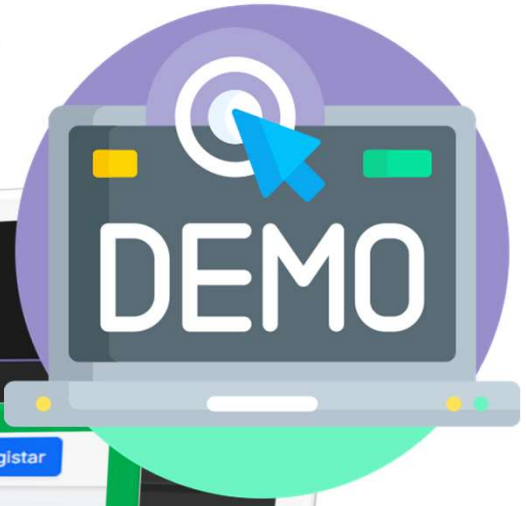


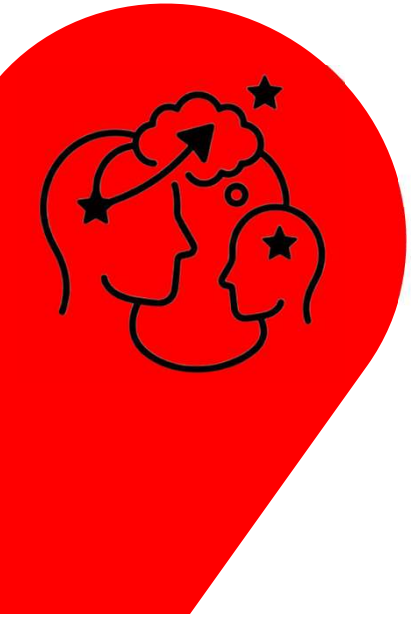


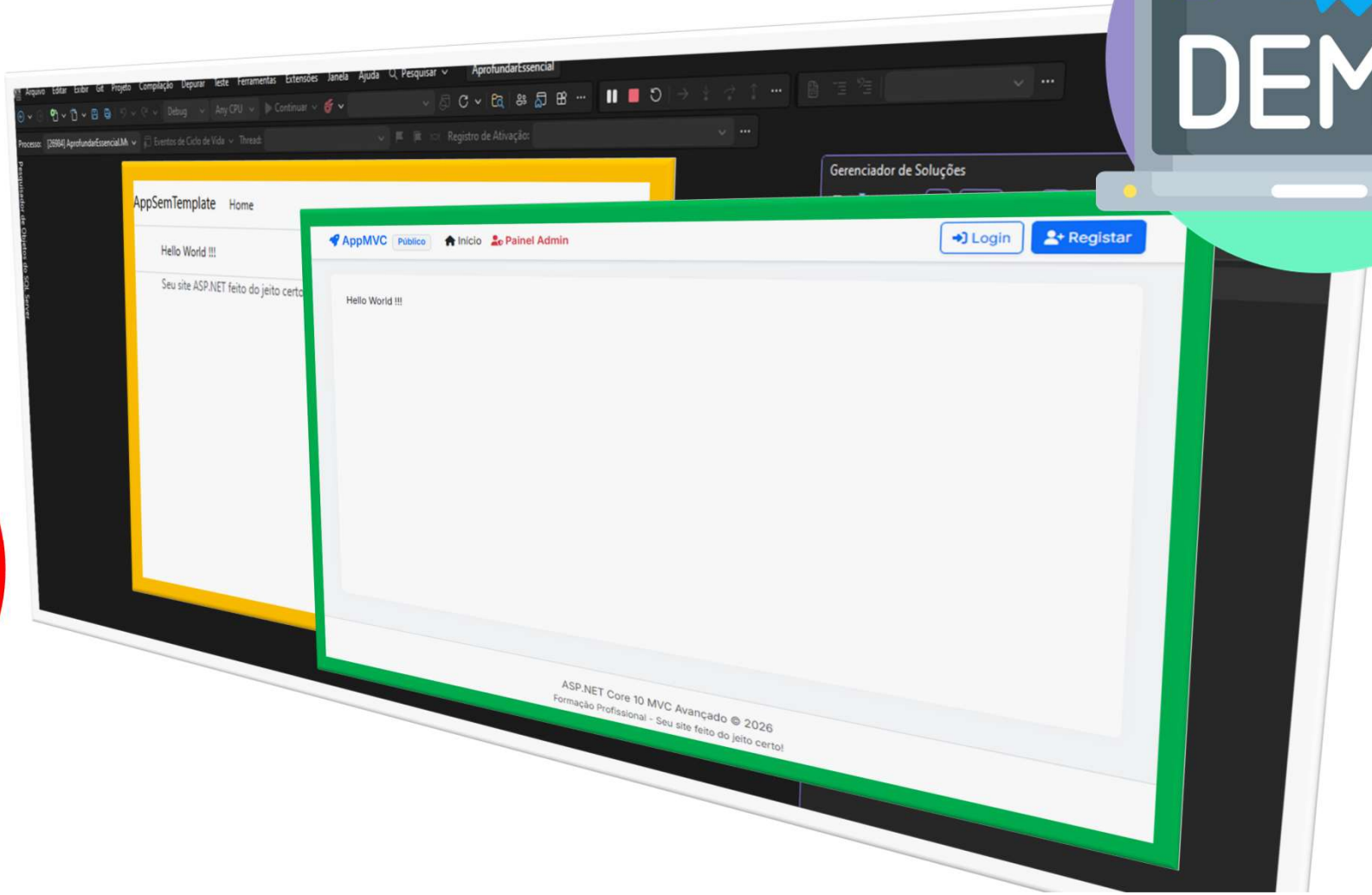


GRUPO JAP

[DEMO] - Utilizar Layout Admin

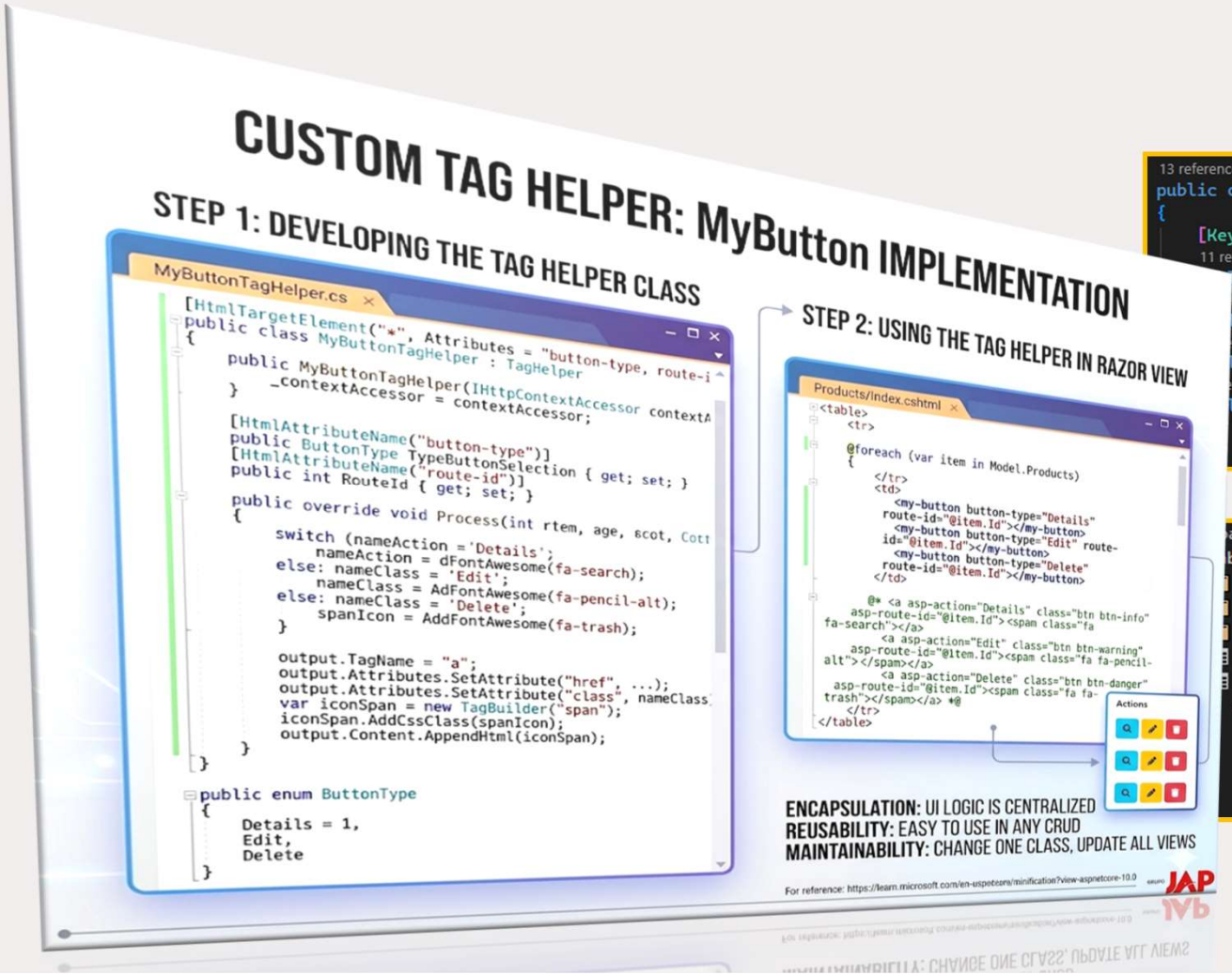






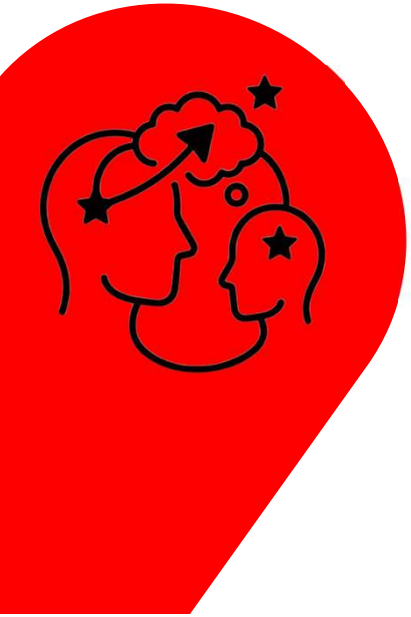
GRUPO JAP

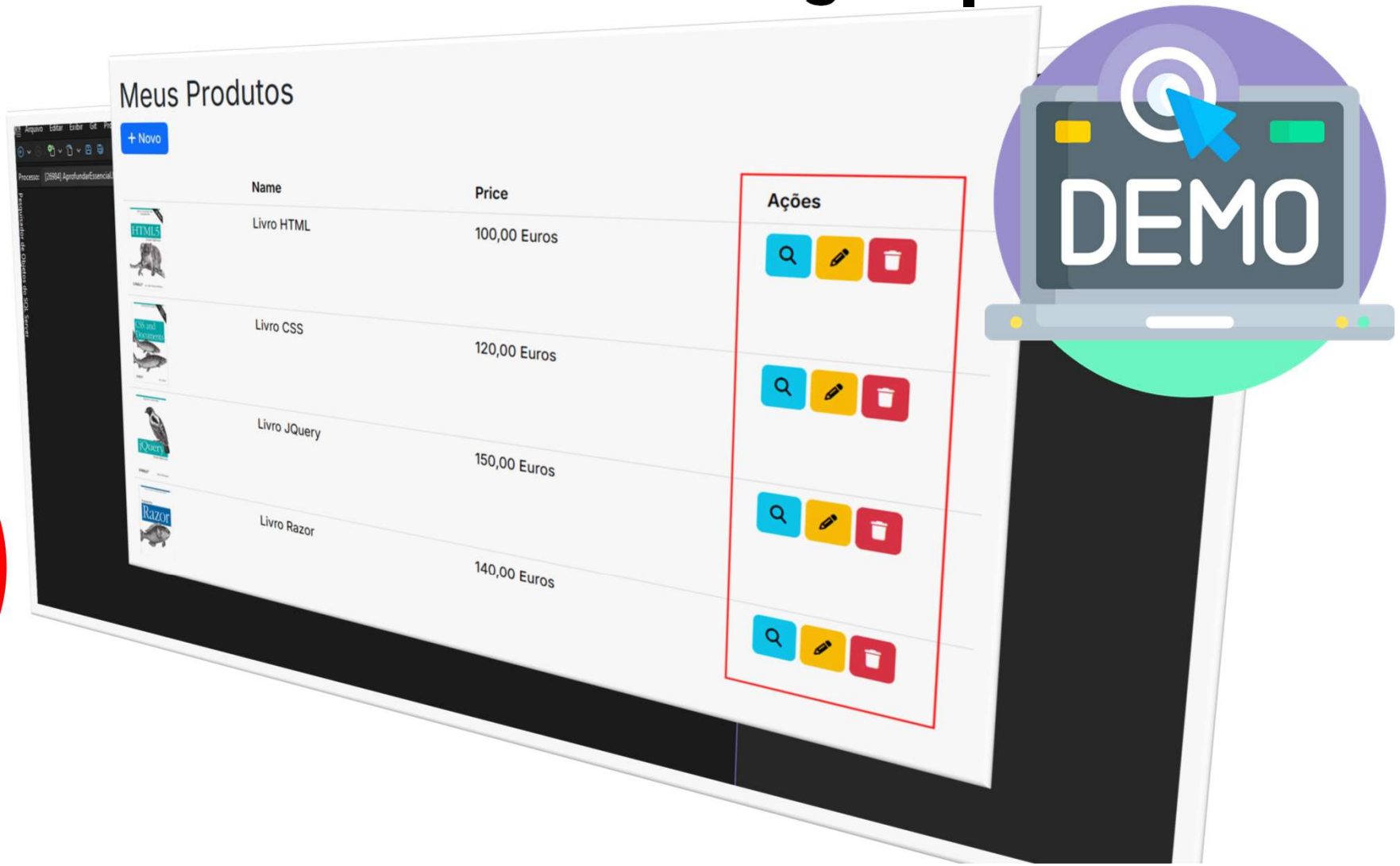
Trabalhar com Tag Helper Customizado



GRUPO JAP

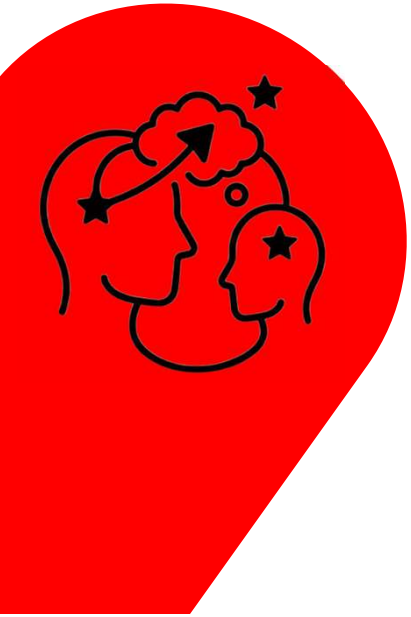
[DEMO] - Criar CRUD e Tag Helper

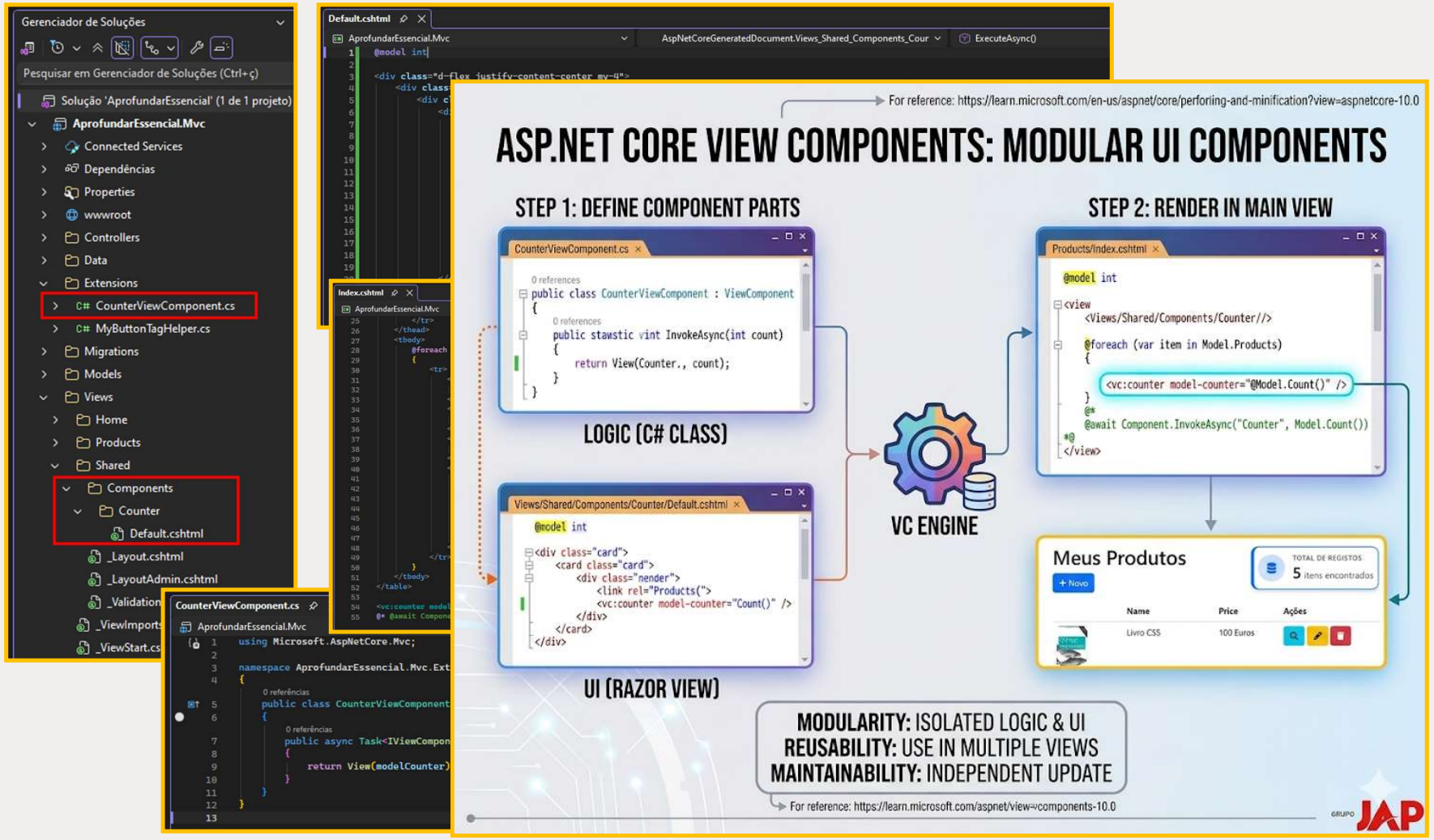




GRUPO JAP

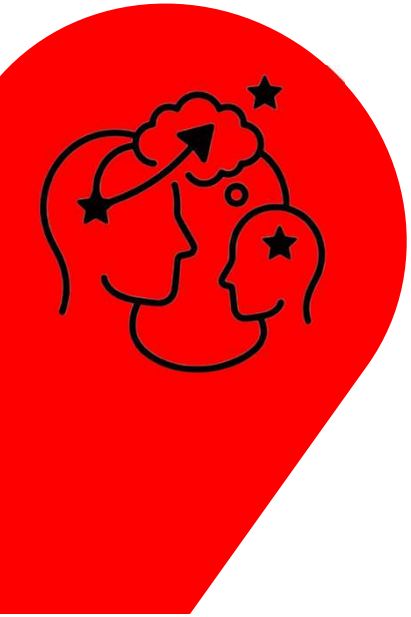
Trabalhar com View Component

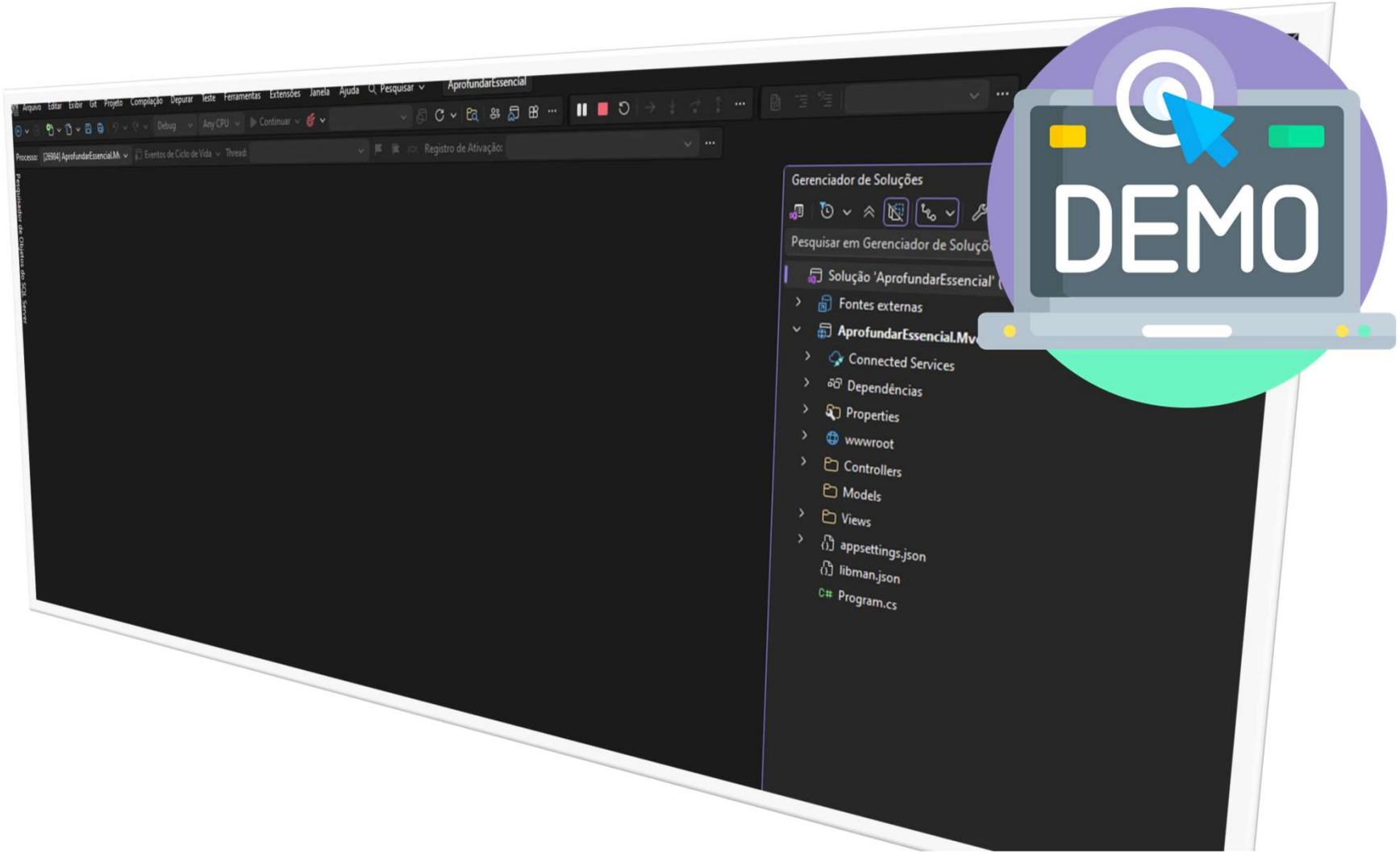




GRUPO JAP

[DEMO] - Usar Tag Help e View Component

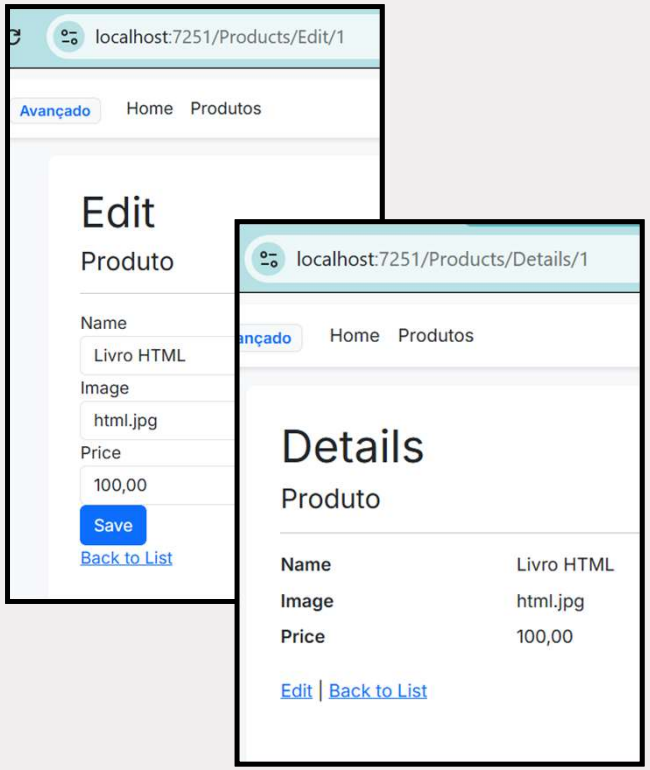


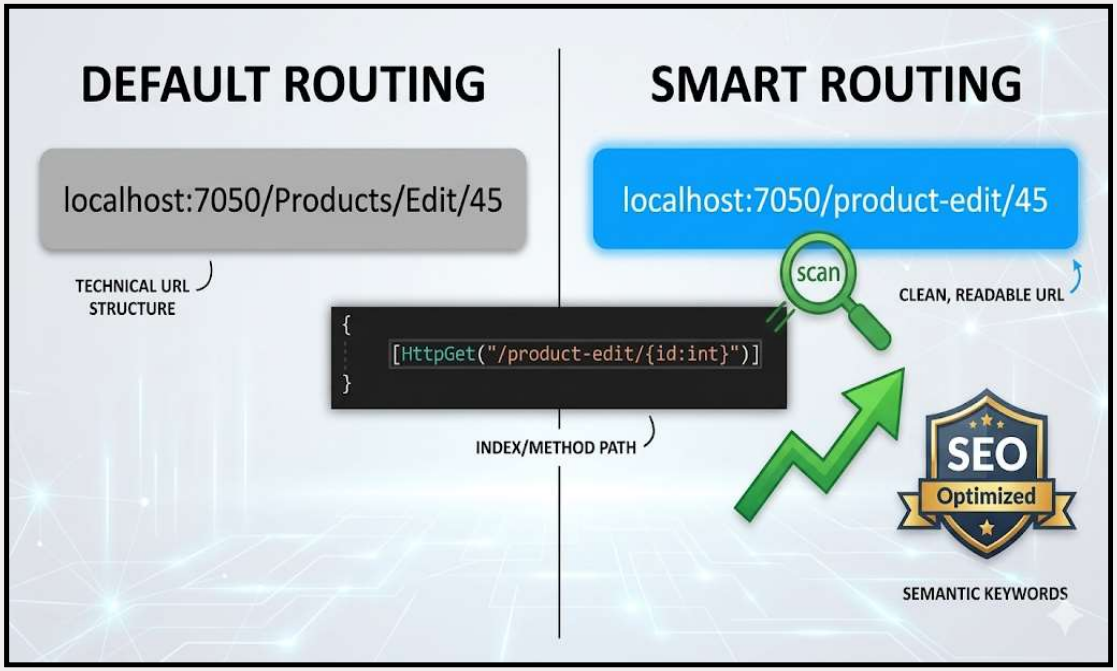


GRUPO JAP

Roteamento Inteligente

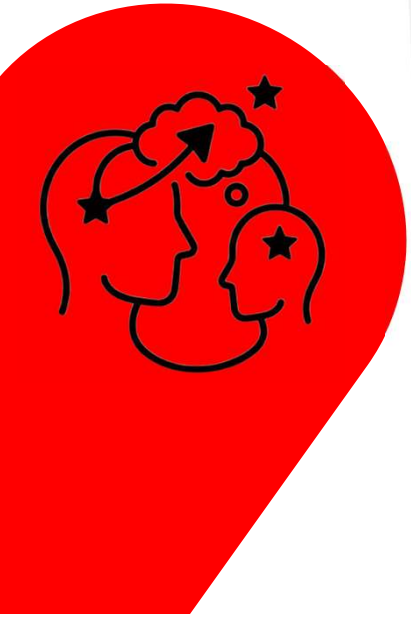
SEO, sigla para *Search Engine Optimization* (Otimização para Mecanismos de Busca), é um conjunto de estratégias de marketing digital que visa melhorar o posicionamento de um site nos resultados orgânicos (gratuitos) de buscadores como Google e Bing. O objetivo é aumentar o tráfego qualificado e a visibilidade online, tornando o site mais relevante para os usuários. Google for Developers +3

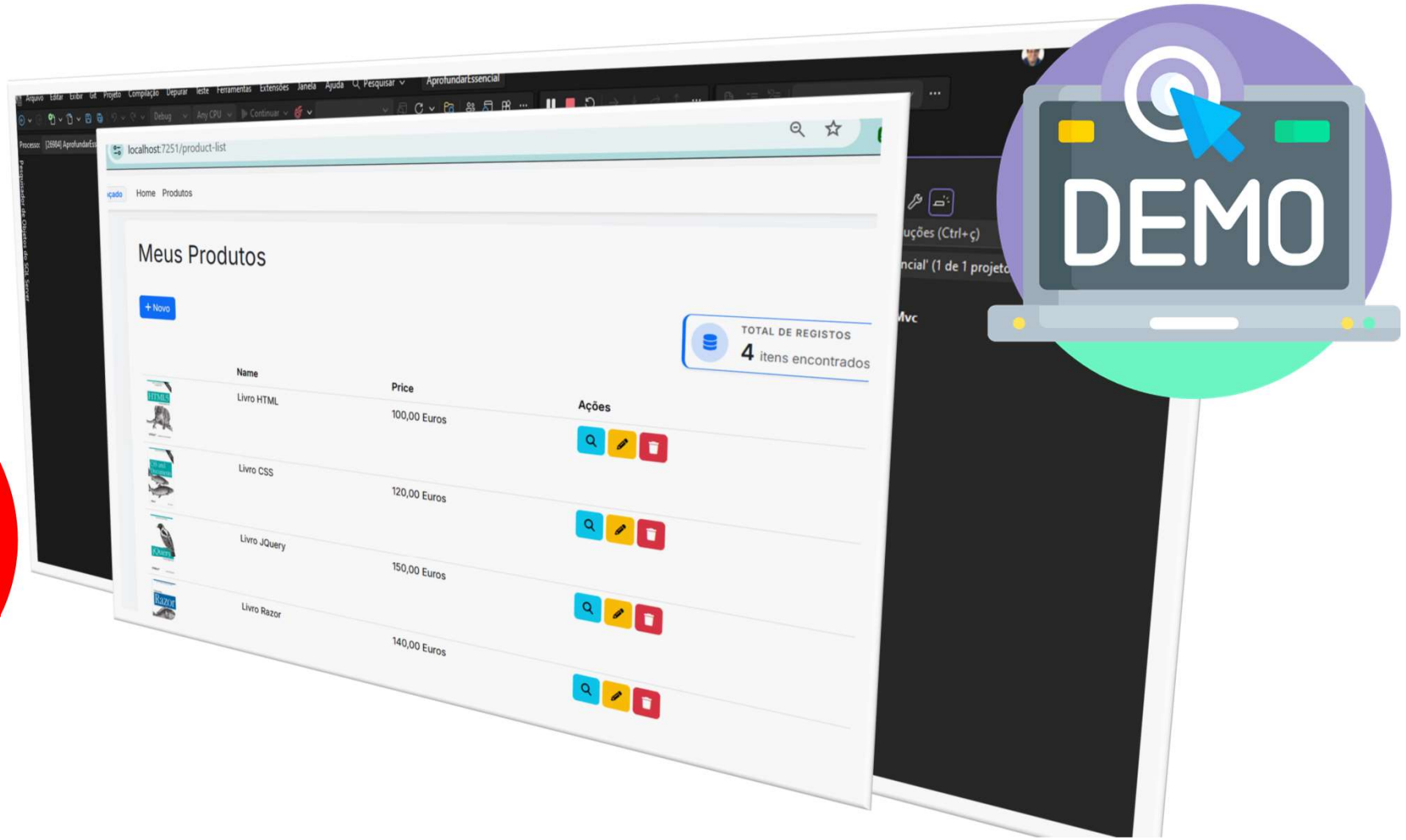




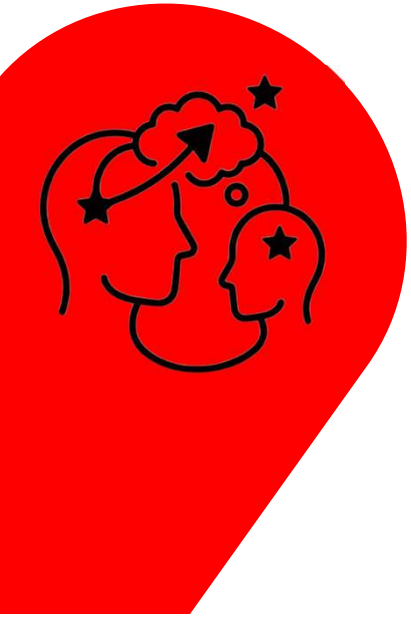
GRUPO JAP

[DEMO] - Rotas Inteligentes





GRUPO JAP




Módulo: Injeção de Dependência (DI)
O CORAÇÃO DO ASP.NET CORE
DESACOPLANDO CÓDIGO E GERENCIANDO CICLOS DE VIDA COM EFICIÊNCIA
1
FUNDAMENTOS DA DI & INVERSÃO DE CONTROLE (IoC)
IoC
2
CONFIGURANDO O CONTAINER: REGISTRO DE SERVIÇOS NATIVOS
services
3
TIPOS DE CICLO DE VIDA (LIFETIMES)
4
INJEÇÃO DE CONSTRUTOR VS. PROPERTY/METHOD INJECTION
Constructor
5
OUTRAS MANEIRAS DE FAZER DI: SERVICE PROVIDER E FACTORIES
6
ACESSANDO O CONTAINER DE DI & BOAS PRÁTICAS
DI

GRUPO JAP

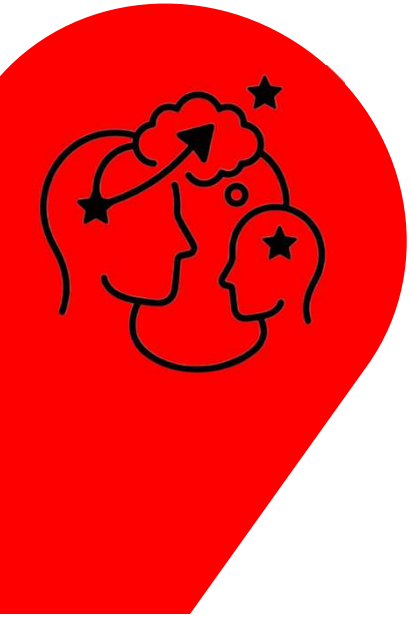
Como funciona o DI no ASP.NET Core

A Injeção de Dependência (DI) é um padrão de projeto que ajuda a reduzir o acoplamento de código em sua aplicação.

No contexto do ASP.NET Core a injeção de dependência é usada para adicionar serviços e gerenciar a vida útil dos objetos.

O ASP.NET Core possui um contêiner de injeção de dependência integrado.

Este contêiner é responsável por construir os objetos necessários e fornecê-los quando necessário.



Para que um serviço seja injetado no código é necessário registrá-lo ao contêiner de DI

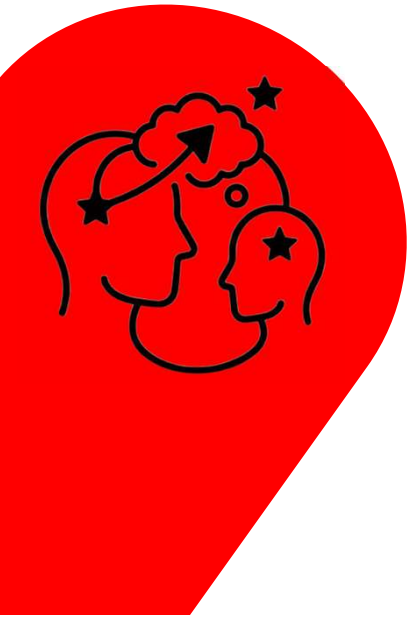
```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<ApplicationDbContext>();

builder.Services.AddControllersWithViews();

var app = builder.Build();
```

Para fazer uso do serviço registrado é de costume injetar este serviço via construtor.

```
public class AlunosController : Controller
{
    private readonly ApplicationDbContext _context;

    public AlunosController(ApplicationDbContext context)
    {
        _context = context;
    }
}
```

GRUPO JAP

Configurar a DI no ASP.NET Core



GenerateGuidService.cs*

AprofundarEssencial.Mvc

```
1  namespace AprofundarEssencial.Mvc.Services
2  {
3      2 referências
       public class GenerateGuidService : IGenerateGuidService
4      {
5          3 referências
           public Guid OperationId { get; private set; }
6
7          0 referências
           public GenerateGuidService()
8          {
9              OperationId = Guid.NewGuid();
10         }
11     }
12
13     4 referências
       public interface IGenerateGuidService
14     {
15         3 referências
           Guid OperationId { get; }
16     }
17 }
```

Gerenciador de Soluções

Pesquisar em Gerenciador de Soluções (Ctrl+ç)

- Solução 'AprofundarEssencial' (1 de 1 projeto)
  - AprofundarEssencial.Mvc
    - Connected Services
    - Dependências
    - Properties
    - wwwroot
    - Controllers
      - HomeController.cs
      - ProductsController.cs
      - TestDIController.cs
    - Data
    - Extensions
    - Migrations
    - Models
    - Services
      - GenerateGuidService.cs
    - Views
    - appsettings.json
    - bundleconfig.json
    - libman.json
    - Program.cs

Program.cs

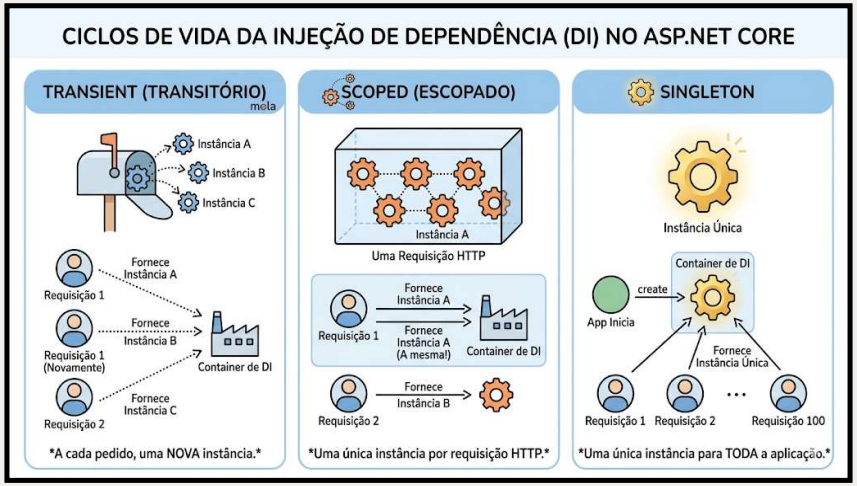
AprofundarEssencial.Mvc — Program

```
1  using AppSemTemplate.Data;
2  using AprofundarEssencial.Mvc.Extensions;
3  using AprofundarEssencial.Mvc.Services;
4  using Microsoft.EntityFrameworkCore;
5
6  var builder = WebApplication.CreateBuilder(args);
7
8  builder.Services.AddControllersWithViews();
9
10 //builder.Services.AddRouting(options =>
11 //    options.ConstraintMap["slugify"] = typeof(RouteSlugifyParameterTransformer));
12
13 builder.Services.AddSingleton<IHttpContextAccessor, HttpContextAccessor>();
14
15 builder.Services.AddScoped<IGenerateGuidService, GenerateGuidService>();
16
17 builder.Services.AddDbContext<AppDbContext>(o =>
18     o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
19
```

```
public class TestDIController : Controller
{
    private readonly IGenerateGuidService _generateGuidService;

    0 referências
    public TestDIController(IGenerateGuidService generateGuidService)
    {
        _generateGuidService = generateGuidService;
    }
    0 referências
    public IActionResult Index()
    {
        var result = $"Operation ID: {_generateGuidService.OperationId}";
                                  result | Exibir | "Operation ID: 6033f483-31d6-4e13-8bb2-5e1004d4c67b"
        return View();
    }
}
```

GRUPO JAP

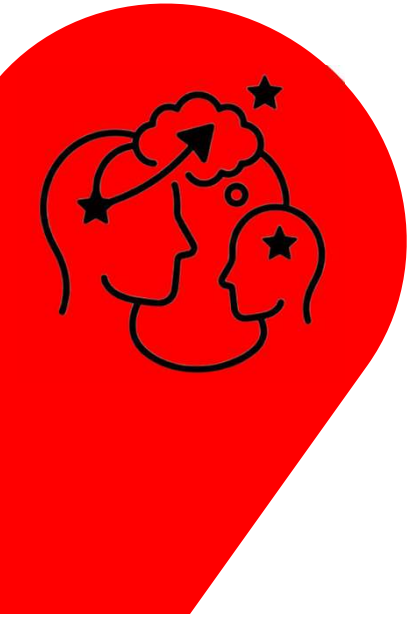
Tipos de Ciclos de Vida do DI #1



2. Scoped (Escopado)

*Use quando você precisa que a mesma instância seja compartilhada por todos os componentes envolvidos em **uma única requisição HTTP** (como o Controller e um ViewComponent).*

- **Contexto de Banco de Dados (Entity Framework):** O exemplo clássico do `AppDbContext`. Você quer que todas as alterações feitas no Controller e nos serviços durante aquele "clique" do usuário usem a mesma conexão e transação.
- **Informações do Utilizador Logado:** Um serviço que carrega os dados do perfil do usuário do banco de dados uma única vez no início da requisição e os mantém disponíveis para qualquer View ou Componente que precise deles.
- **Carrinho de Compras (durante o checkout):** Para garantir que o cálculo total e a validação de estoque usem exatamente os mesmos dados de memória enquanto o pedido está sendo processado.



1. Transient (Transitório)

Use quando o serviço é leve, não guarda estado e você quer evitar qualquer efeito colateral entre diferentes partes da aplicação.

- **Serviços de Cálculo ou Utilitários:** Um serviço que apenas formata strings, calcula impostos ou valida o **Código Postal de Portugal** que criamos. Não há razão para reaproveitar a instância.
- **Envio de E-mail/SMS:** Cada vez que um componente precisa disparar uma notificação, ele recebe uma instância nova e limpa para configurar o destinatário e a mensagem.
- **Mapeadores (AutoMapper):** Transformar um Model em ViewModel. É uma tarefa rápida e isolada que não depende de dados anteriores.

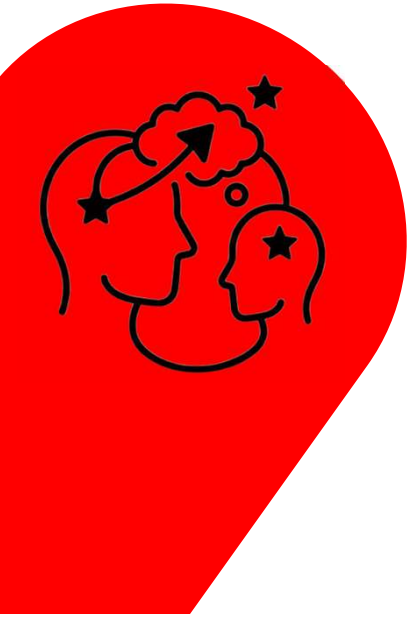
3. Singleton

*Use quando o serviço deve ser partilhado por **todos os usuários** e durar enquanto o site estiver no ar. Cuidado: o serviço deve ser "Thread-Safe".*

- **Cache de Configurações Globais:** Se o seu site lê configurações de um arquivo JSON ou de uma API externa (como taxas de câmbio que mudam pouco), você carrega uma vez no Singleton e todos os usuários lêem da memória.
- **Logs de Auditoria em Memória:** Um serviço que conta quantos usuários visitaram certas páginas ou armazena logs rápidos antes de enviá-los para um arquivo físico.
- **Clientes de API Externas (HttpClient):** Manter uma única instância do `HttpClient` (geralmente via `IHttpClientFactory`) evita o problema de esgotamento de sockets no servidor, sendo reaproveitado por toda a aplicação.

GRUPO JAP

Tipos de Ciclos de Vida do DI #2



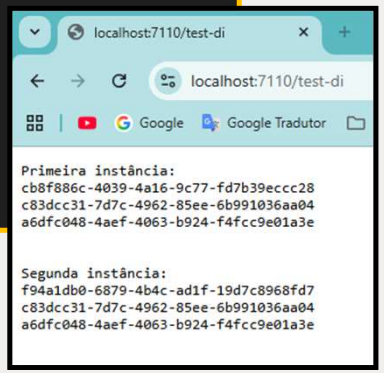
GenerateGuidService.cs

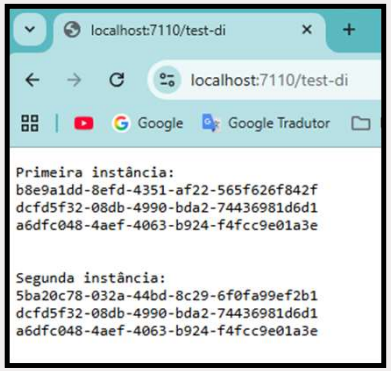
```
{
    7 referências
    public class OperacaoService
    {
        0 referências
        public OperacaoService(IGenerateGuidServiceTransient transient,
                               IGenerateGuidServiceScoped scoped,
                               IGenerateGuidServiceSingleton singleton)
        {
            Transient = transient;
            Scoped = scoped;
            Singleton = singleton;
        }

        4 referências
        public IGenerateGuidServiceTransient Transient { get; }
        4 referências
        public IGenerateGuidServiceScoped Scoped { get; }
        4 referências
        public IGenerateGuidServiceSingleton Singleton { get; }
    }

    4 referências
    public class GenerateGuidService : IGenerateGuidServiceTransient, IGenerateGuidServiceScoped, IGenerateGuidServiceSingleton
    {
        0 referências
        public GenerateGuidService()
        {
            OperationId = Guid.NewGuid();
        }

        11 referências
        public Guid OperationId { get; private set; }
    }

    3 referências
    public interface IGenerateGuidService
    {
        11 referências
        Guid OperationId { get; }
    }

    4 referências
    public interface IGenerateGuidServiceTransient : IGenerateGuidService
    {
    }

    4 referências
    public interface IGenerateGuidServiceScoped : IGenerateGuidService
    {
    }

    4 referências
    public interface IGenerateGuidServiceSingleton : IGenerateGuidService
    {
    }
```

```
// Registos específicos para as interfaces específicas
builder.Services.AddTransient<IGenerateGuidServiceTransient, GenerateGuidService>();
builder.Services.AddScoped<IGenerateGuidServiceScoped, GenerateGuidService>();
builder.Services.AddSingleton<IGenerateGuidServiceSingleton, GenerateGuidService>();

// Registo do serviço que consome as três dependências
builder.Services.AddTransient<OperacaoService>();
```

TestDIController.cs

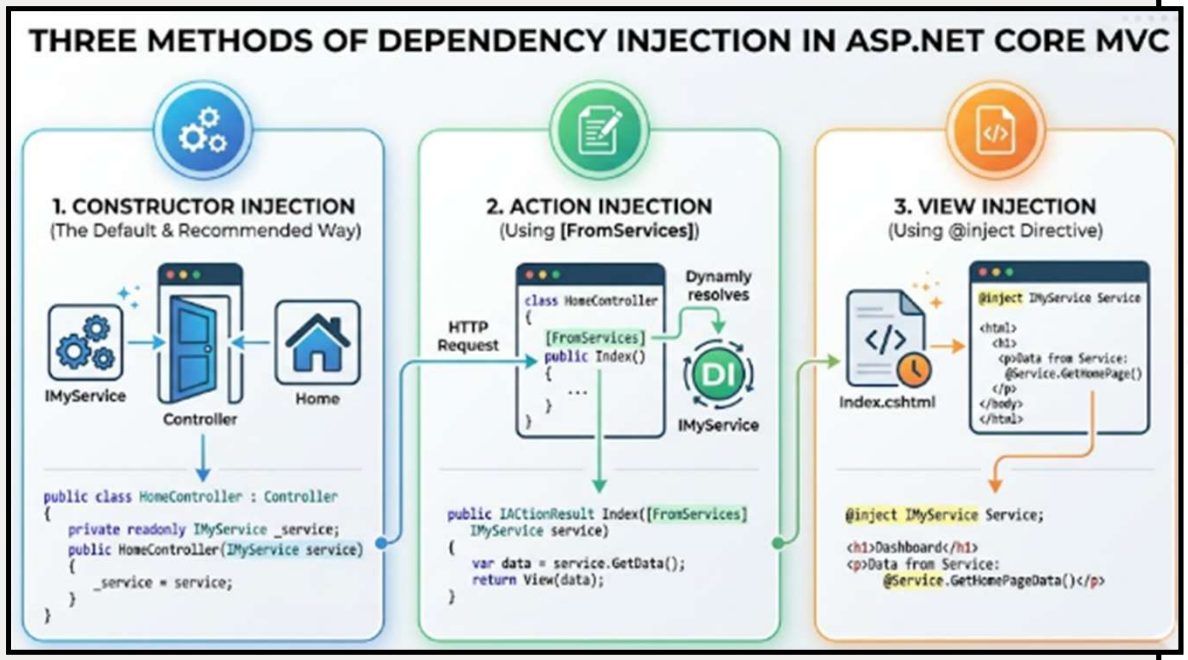
```
using AprofundarEssencial.Mvc.Services;
using Microsoft.AspNetCore.Mvc;

namespace AprofundarEssencial.Mvc.Controllers
{
    [Route("test-di")]
    1 referência
    public class TestDIController : Controller
    {
        4 referências
        public OperacaoService OperacaoService { get; }
        4 referências
        public OperacaoService OperacaoService2 { get; }


        0 referências
        public TestDIController(OperacaoService operacaoService, OperacaoService operacaoService2)
        {
            OperacaoService = operacaoService;
            OperacaoService2 = operacaoService2;
        }
        0 referências
        public string Index()
        {
            return
                "Primeira instância: " + Environment.NewLine +
                OperacaoService.Transient.OperationId + Environment.NewLine +
                OperacaoService.Scoped.OperationId + Environment.NewLine +
                OperacaoService.Singleton.OperationId + Environment.NewLine +

                Environment.NewLine +
                Environment.NewLine +

                "Segunda instância: " + Environment.NewLine +
                OperacaoService2.Transient.OperationId + Environment.NewLine +
                OperacaoService2.Scoped.OperationId + Environment.NewLine +
                OperacaoService2.Singleton.OperationId + Environment.NewLine;
        }
    }
}
```





GRUPO JAP

Outras formas de se fazer DI



1. Injeção via Construtor (Constructor Injection)

É a abordagem padrão e mais recomendada. As dependências são declaradas nos parâmetros do construtor da classe e o framework resolve todas elas automaticamente. Garante que a classe não seja criada sem suas dependências.

```csharp
public class HomeController : Controller
{
    private readonly IMeuServico _servico;

    public HomeController(IMeuServico servico)
    {
        _servico = servico;
    }
}
```

Use o código com cuidado.

2. Injeção via Action / Método (Method Injection)

Usada quando você precisa de um serviço específico apenas para um método pontual (como uma action de Controller ou um endpoint de Minimal API), evitando inflar o construtor da classe inteira. Utiliza o atributo `[FromServices]`.

```csharp
public IActionResult MinhaAction([FromServices] IMeuServico servico)
{
    var dados = servico.ObterDados();
    return View(dados);
}
```

Use o código com cuidado.

3. Injeção na View (View Injection)

Ideal para quando você precisa expor um serviço de domínio ou utilitário diretamente no arquivo `.cshtml`, sem precisar passar a informação via *Controller*. Utiliza a diretiva `@inject`.

```razor
@inject IMeuServico Servico

<div>
    <span>@Servico.ObterMensagemRodape()</span>
</div>
```

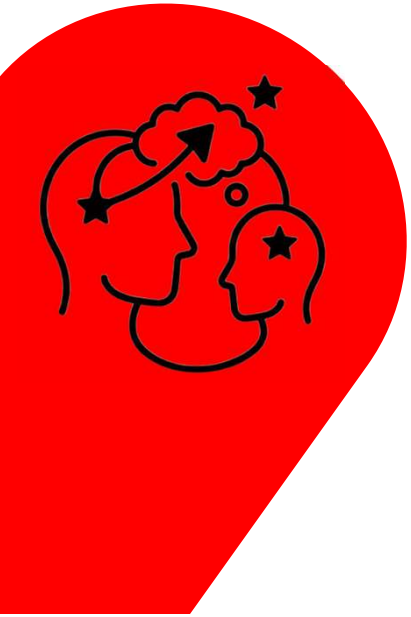
GRUPO JAP

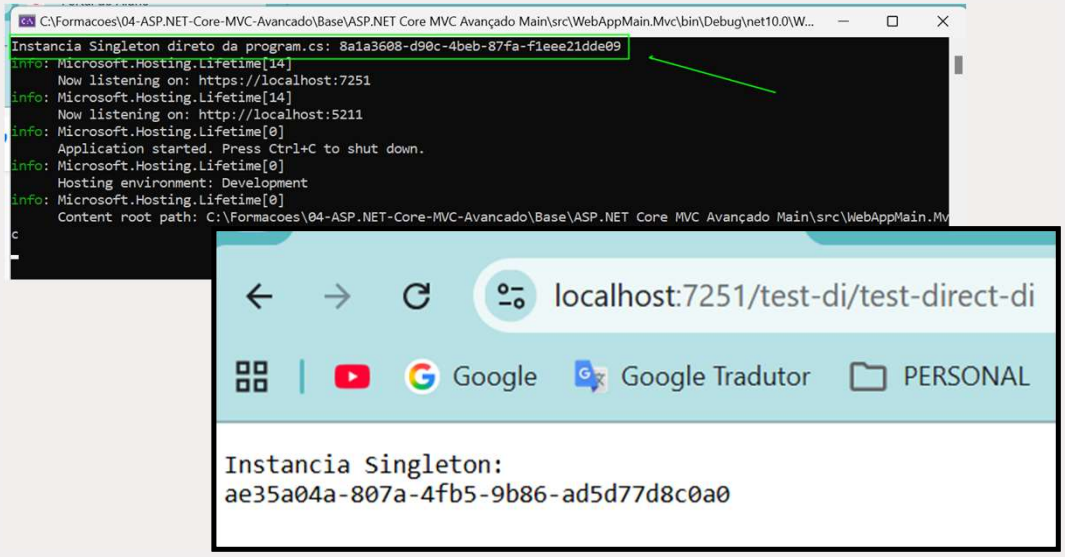
Aceder diretamente o Container DI

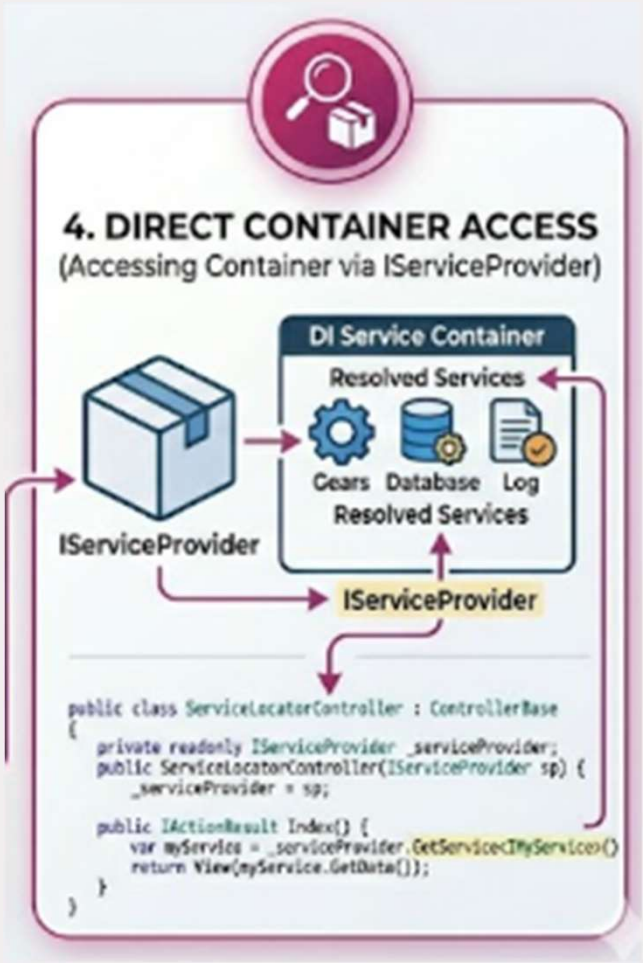
Visão geral criada por IA

Para aceder diretamente ao container de Injeção de Dependência (DI) do ASP.NET Core usando o `IServiceProvider`, você pode obter a instância do provedor e utilizar o método `GetService` ou `GetRequiredService`. Microsoft Learn +3

Embora o acesso direto seja uma alternativa viável, ele é geralmente evitado (conhecido como *Service Locator anti-pattern*) em favor da injeção no construtor. LinkedIn · Guilherme Zordan +1

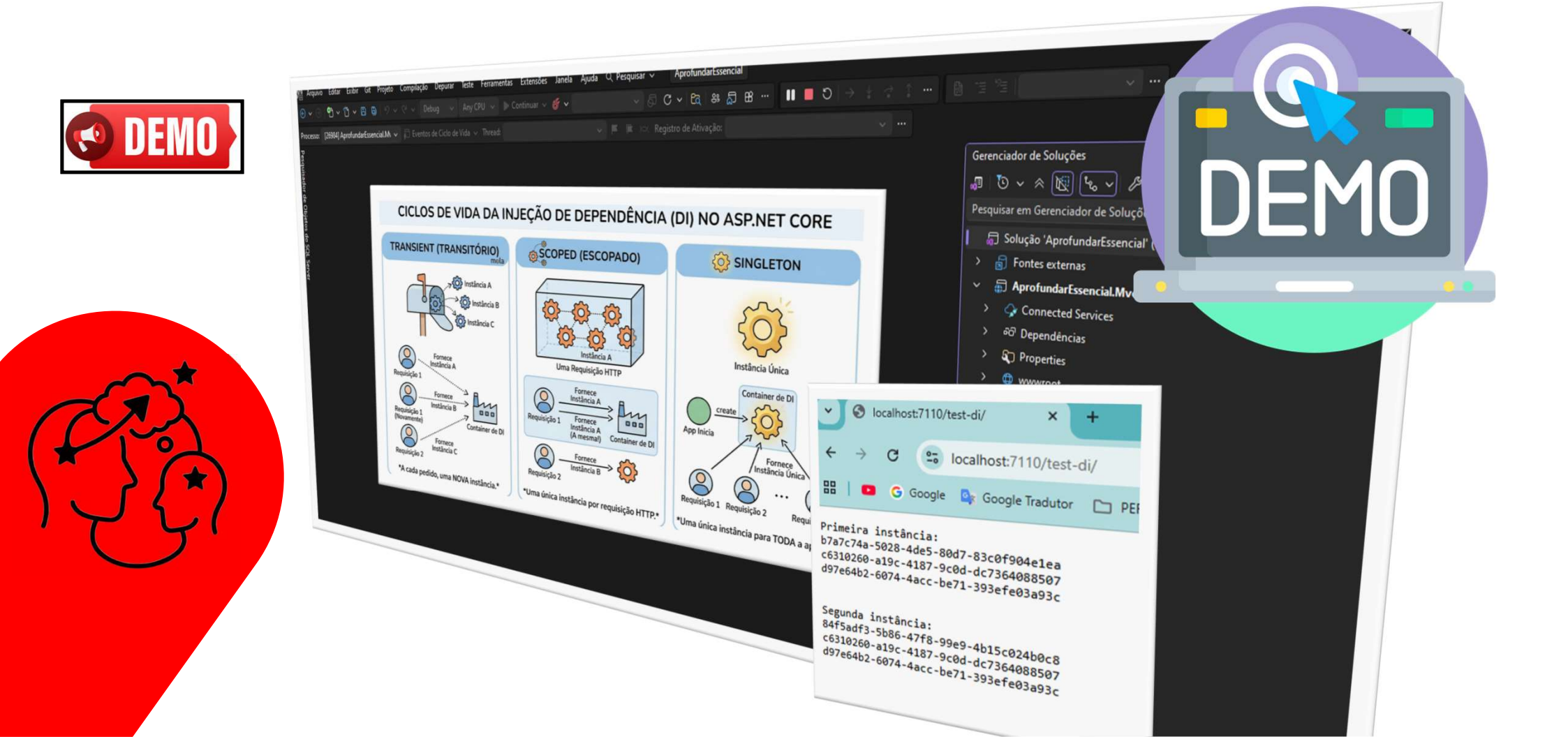




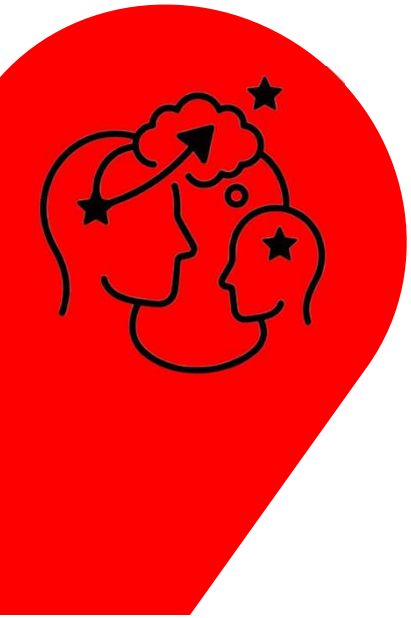


GRUPO JAP

[DEMO] - O Maravilhoso mundo das DIs



GRUPO JAP



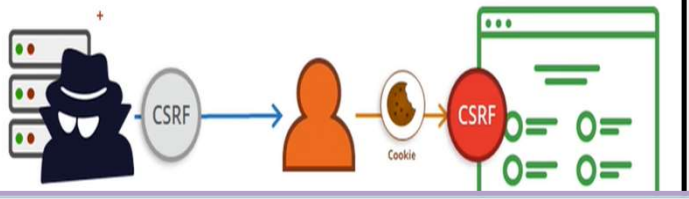

Módulo: Segurança
A FORTALEZA DO ASP.NET CORE
BLINDANDO A APLICAÇÃO CONTRA AMEAÇAS MODERNAS E GERENCIANDO ACESSO
1
PROTEÇÃO CONTRA CSRF (ANTI-FORGERY TOKEN)
2
PROTEÇÃO CONTRA XSS (CROSS-SITE SCRIPTING)
3
CONFIGURANDO O ASP.NET IDENTITY & AUTENTICAÇÃO NA PRÁTICA
SECURITY
4
HSTS & HTTPS (CONEXÕES SEGURAS GLOBALMENTE)
5
AUTORIZAÇÃO NA PRÁTICA: CONTROLE DE ACESSO POR FUNÇÕES
6
TRABALHANDO COM CLAIMS & CUSTOMIZANDO A AUTENTICAÇÃO

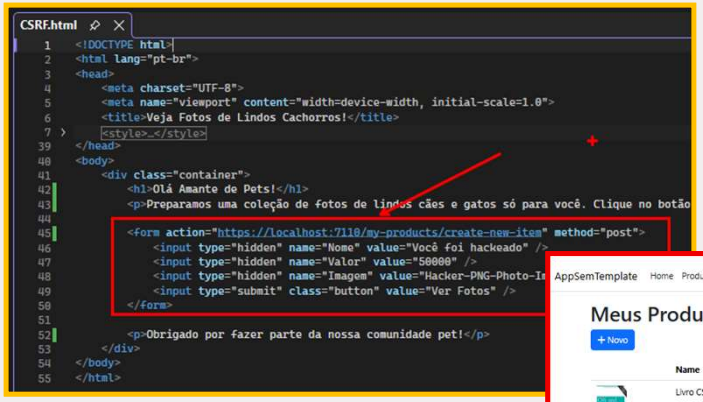
GRUPO JAP

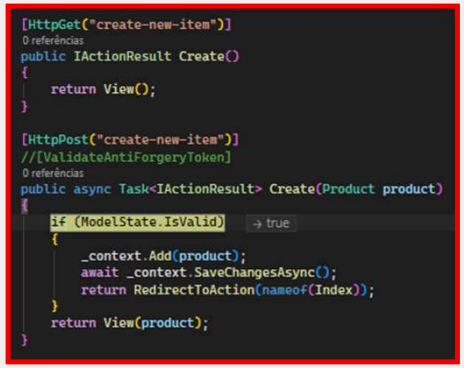
CSRF (Cross-Site Request Forgery)

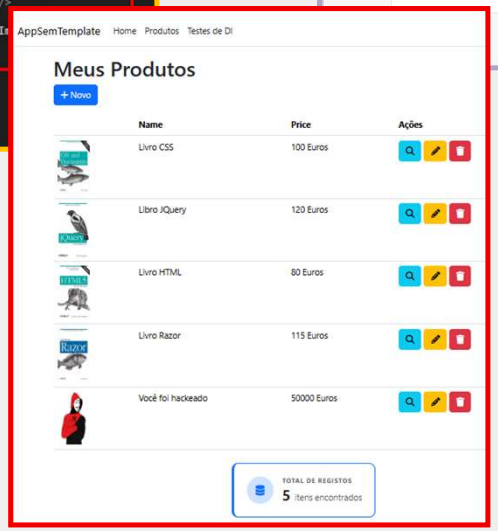
Visão geral criada por IA

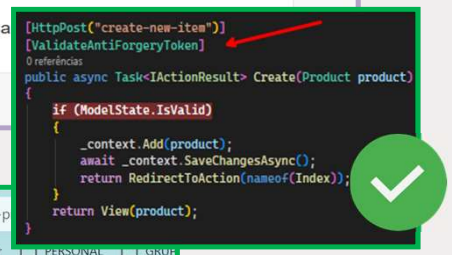
CSRF (**Cross-Site Request Forgery** ou Falsificação de Solicitação entre Sites) é um tipo de ataque cibernético que engana o navegador de um usuário autenticado para executar ações indesejadas em outro site (como alterar senhas, e-mails ou transferir fundos) sem consentimento. Explora a confiança que um site tem nas credenciais salvas do usuário. Fortinet +3

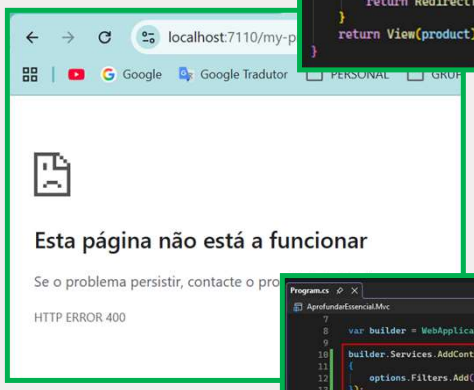


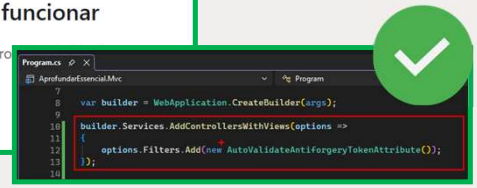












GRUPO JAP

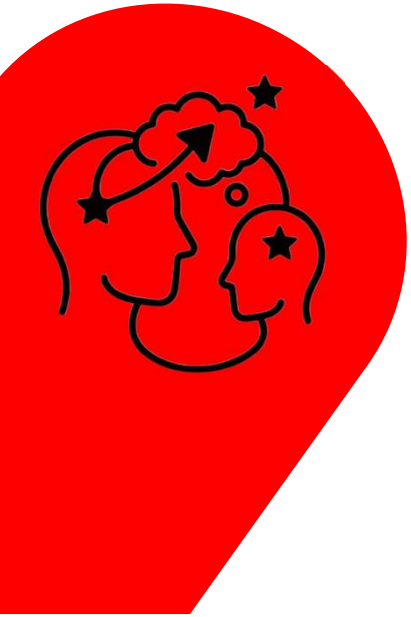
XSS (Cross-Site Scripting)

Visão geral criada por IA

Cross-site scripting (XSS) é uma vulnerabilidade de segurança web onde criminosos injetam scripts maliciosos (geralmente JavaScript) em sites legítimos. Quando usuários visitam essas páginas, o navegador executa o código, permitindo o roubo de cookies, tokens de sessão, credenciais ou o redirecionamento da vítima para sites maliciosos. Kaspersky +3

```
@{
    var inseguro = "<img src='https://t3.ftcdn.net/jpg/05/56/29/10/360_F_556291020_
}

@inseguro

Olá!
```


```
@{
    var inseguro = "<img src='https://t3.ftcdn.net/jpg/05/56/29/10/360_F_556
}

<br />
<br />
@inseguro
<br />
<br />
@Html.Raw(inseguro)

Olá!
```

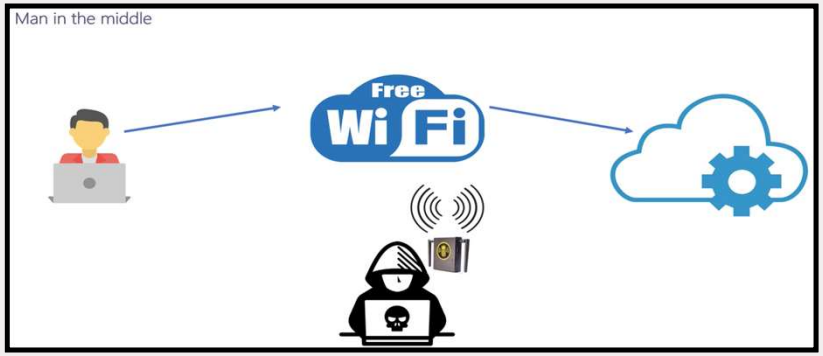


GRUPO JAP

HSTS (HTTPS Strict Transport Securiry)

Visão geral criada por IA

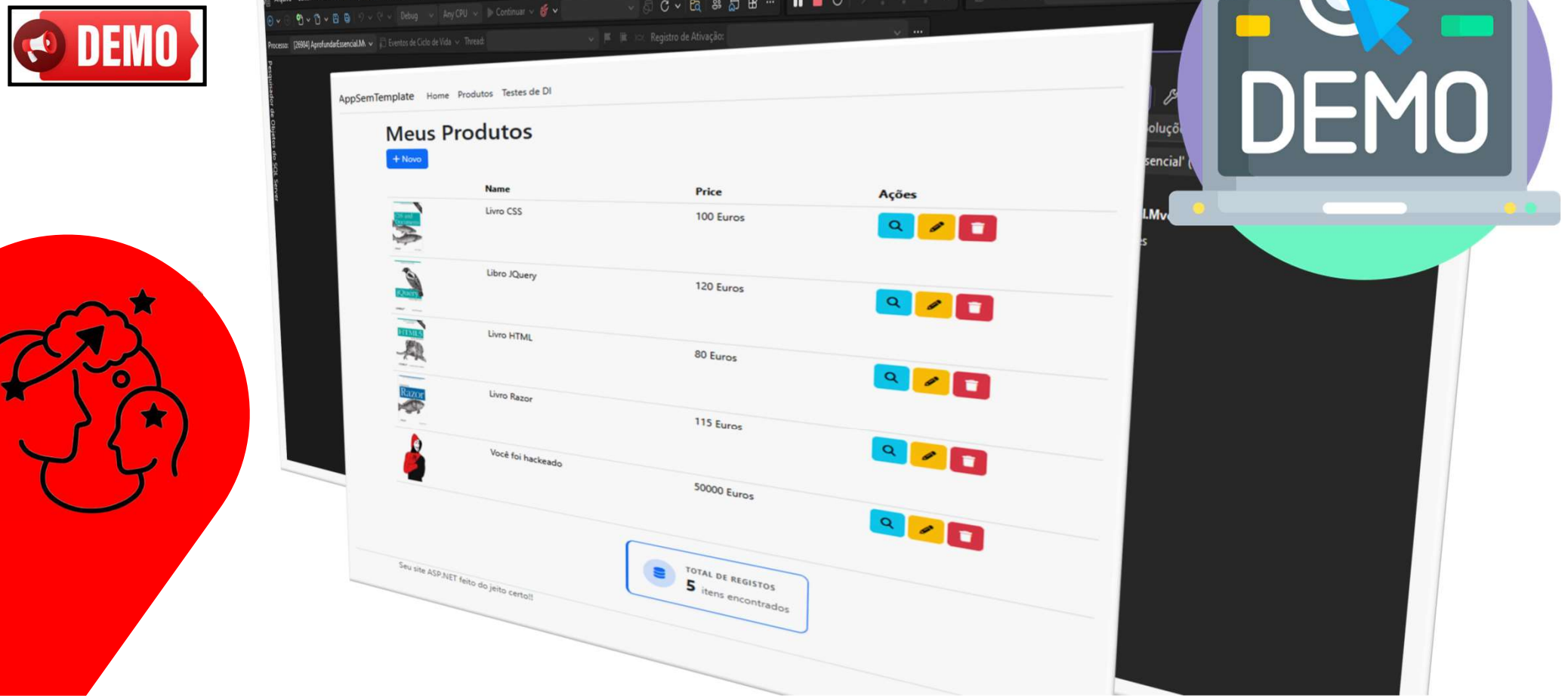
HSTS (HTTP Strict Transport Security) é uma política de segurança web que obriga os navegadores a se conectarem a um site apenas via HTTPS, eliminando conexões HTTP inseguras. Ele impede ataques de "homem no meio" (man-in-the-middle), como o *SSL stripping*, garantindo que o navegador converta automaticamente solicitações HTTP em HTTPS. MDN Web Docs +3




```
using AppSemTemplate.Data;
using AprofundarEssencial.Mvc.Services;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

//builder.Services.AddRouting(options =>
//    options.ConstraintMap["slugify"] = typeof(RouteSlugifyParameterTransformer));

builder.Services.AddSingleton<IHttpContextAccessor, HttpContextAccessor>();

// Registos específicos para as interfaces específicas
builder.Services.AddTransient<IGenerateGuidServiceTransient, GenerateGuidService>();
builder.Services.AddScoped<IGenerateGuidServiceScoped, GenerateGuidService>();
builder.Services.AddSingleton<IGenerateGuidServiceSingleton, GenerateGuidService>();

// Registo do serviço que consome as três dependências
builder.Services.AddTransient<OperacaoService>();

builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddHsts(options =>
{
    options.Preload = true;
    options.IncludeSubDomains = true;
    options.MaxAge = TimeSpan.FromDays(60);
    options.ExcludedHosts.Add("example.com");
    options.ExcludedHosts.Add("www.example.com");
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
}
else
{
    app.UseHsts();
}

app.UseStaticFiles();

app.UseHttpsRedirection();
```

GRUPO JAP

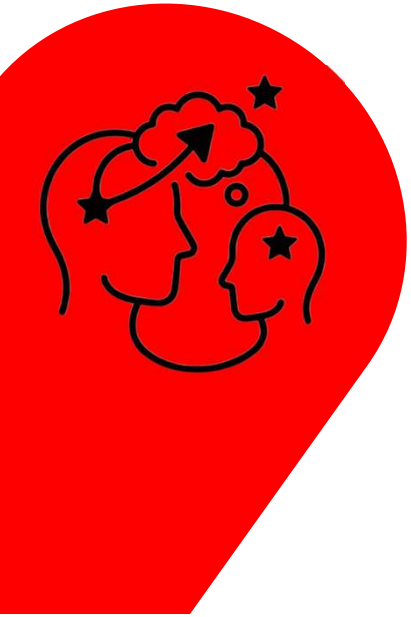
[DEMO] - CSRF + XSS + HSTS = Segurança

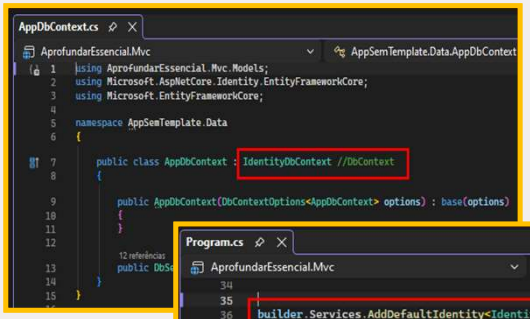


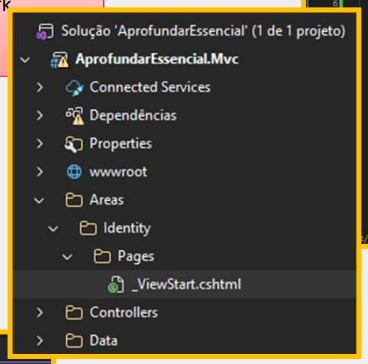
GRUPO JAP

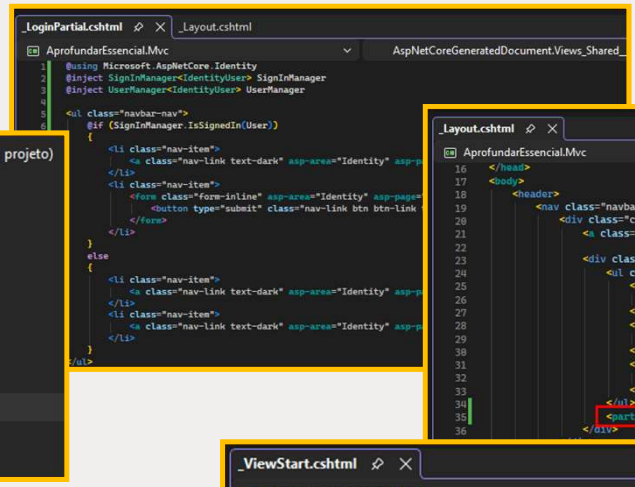
Explorar ASP.NET Identity (Authentication)

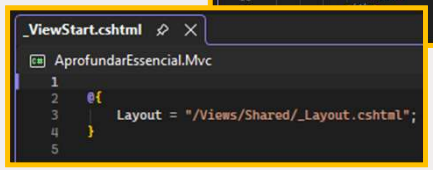
```
PS > dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
PS > dotnet add package Microsoft.AspNetCore.Identiy.UI
PS > dotnet add package Microsoft.AspNetCore.Identiy.UI
ou
PMC > Install-Package Microsoft.AspNetCore.Identity.EntityFramework
PMC > Install-Package Microsoft.AspNetCore.Identiy.UI
PMC > Install-Package Microsoft.AspNetCore.Identiy.UI
```

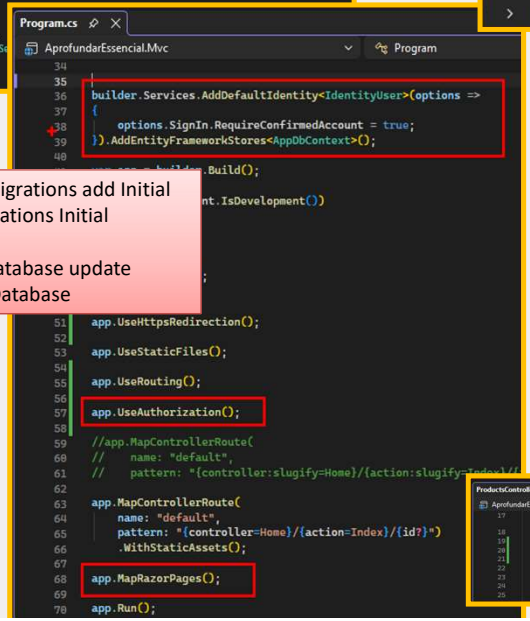




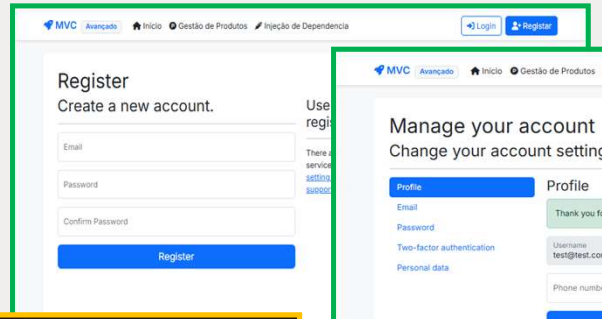


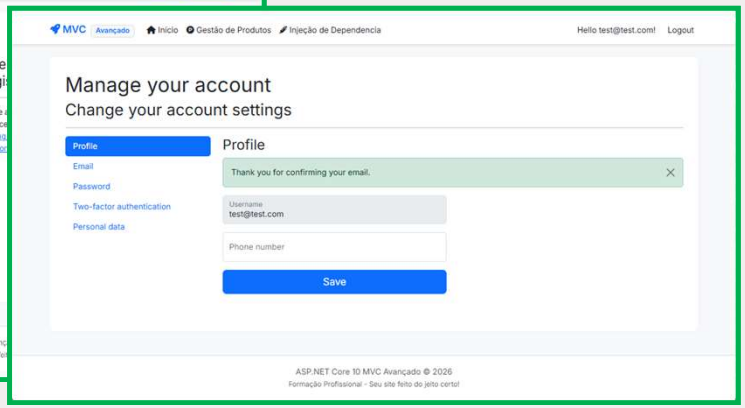


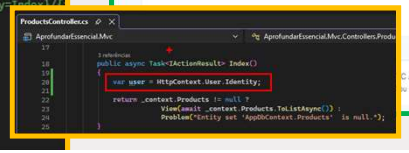





```
PS > dotnet ef migrations add Initial
PMC > Add-Migrations Initial

PS > dotnet ef database update
PMC > Update-Database
```

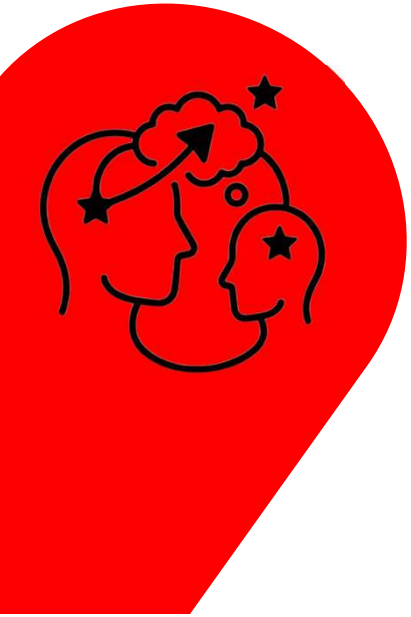


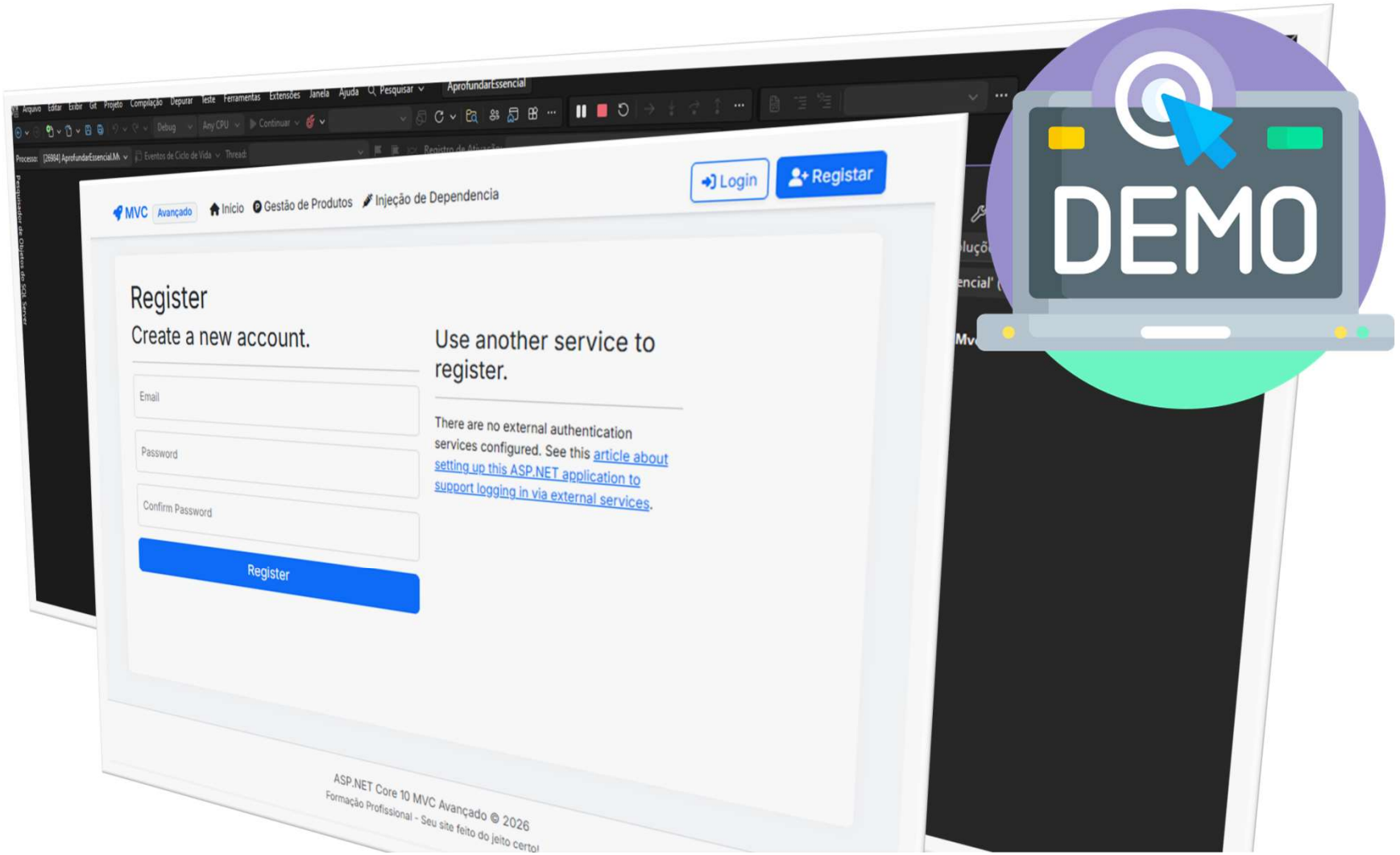




GRUPO JAP

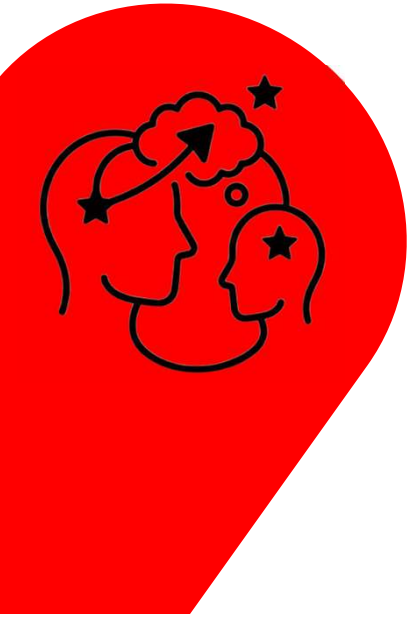
[DEMO] - Identity | Authentication

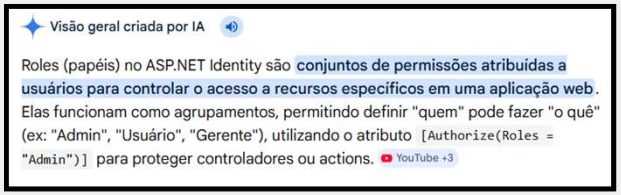


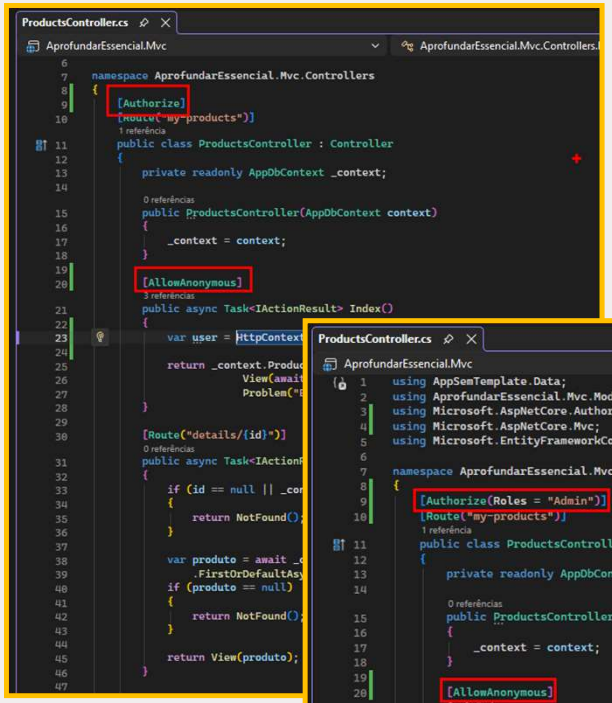


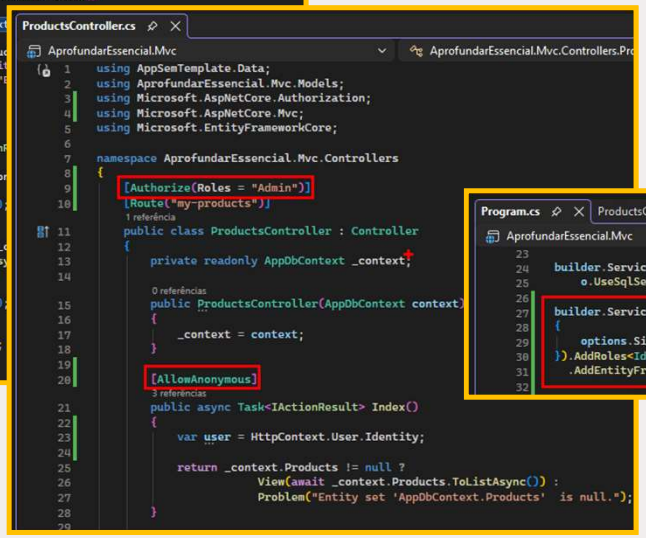
GRUPO JAP

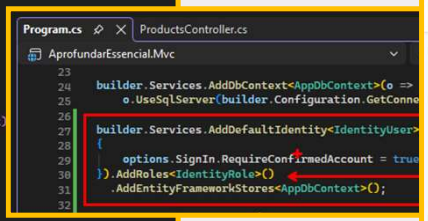
Explorar o Identity | Authorization | Roles

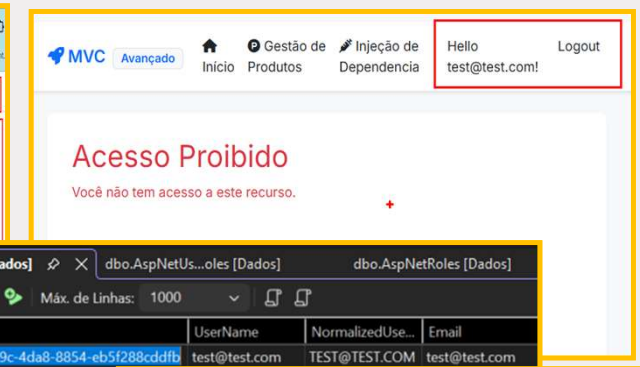


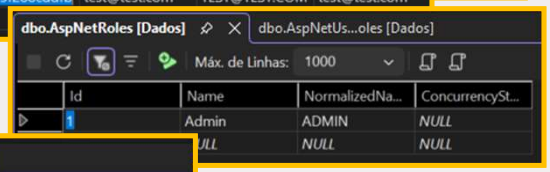


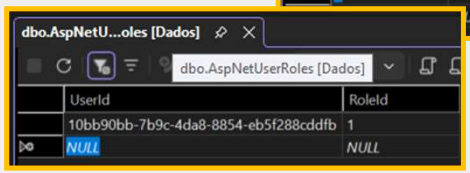


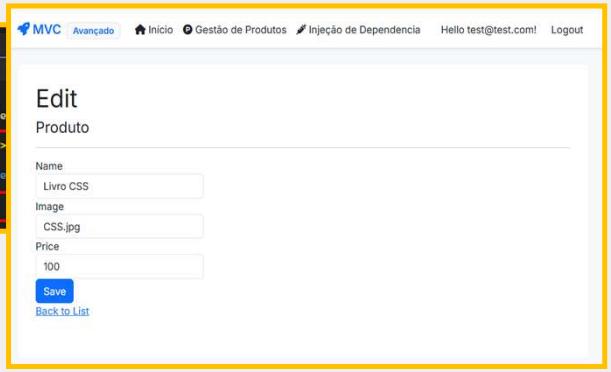










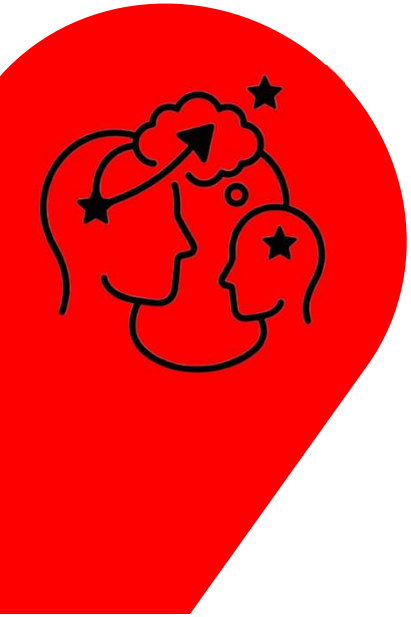


GRUPO JAP

Authorization | Policies Required Roles

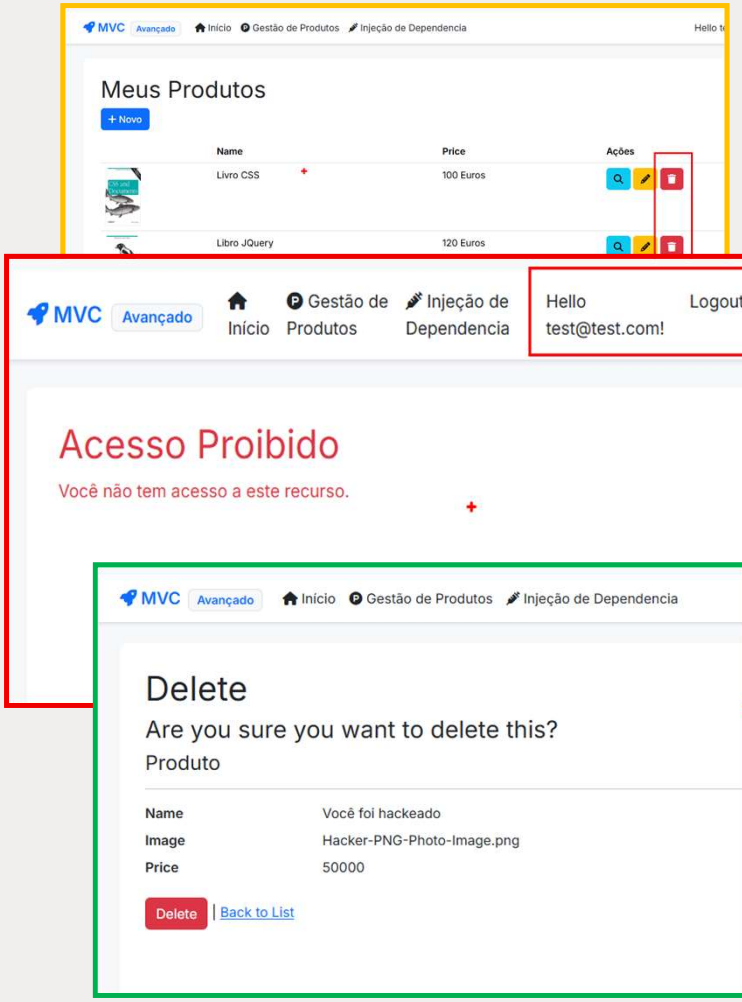
Visão geral criada por IA

No ASP.NET Core Identity, **policies que requerem roles** são regras de autorização baseadas em políticas (Policy-Based Authorization) que exigem que o usuário autenticado pertença a uma ou mais Roles (perfis/papéis) específicas para acessar um recurso. Elas oferecem mais flexibilidade e desacoplamento do que verificar roles diretamente na Controller. YouTube +2



```
ProductsController.cs*
AprofundarEssencial.Mvc
AprofundarEssencial.Mvc.Controllers.Pro
115
116     [Authorize(Policy = "CanDelete")]
117     [HttpPost("delete-item/{id}"), ActionName("Delete")]
118     [ValidateAntiForgeryToken]
        0 referências
119     public async Task<IActionResult> DeleteConfirmed(int id)
120     {
121         if (_context.Products == null)
122         {
123             return Problem("Entity set 'AppDbContext.Products'  is null.");
124         }
125         var product = await _context.Products.FindAsync(id);
126         if (product != null)
127         {
128             _context.Products.Remove(product);
129         }
130
131         await _context.SaveChangesAsync();
132         return RedirectToAction(nameof(Index));
133     }
```

```
Program.cs*
ProductsController.cs
AprofundarEssencial.Mvc
Progr
41
42  builder.Services.AddAuthorization(options =>
43  {
44      options.AddPolicy("CanDelete", policy =>
45              policy.RequireRole("Admin"));
46  });
47
48  var app = builder.Build();
```

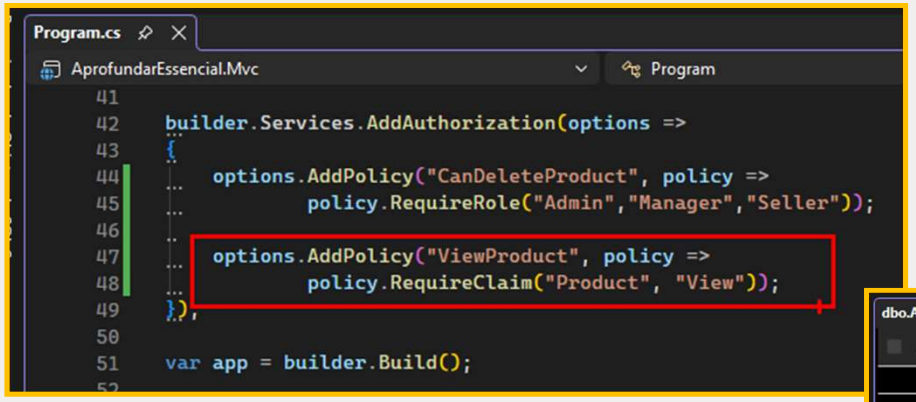


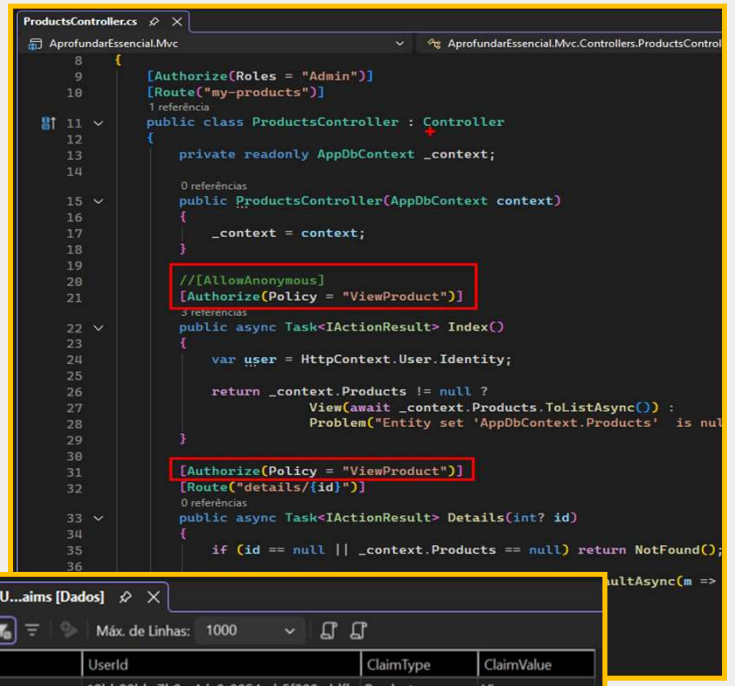
GRUPO JAP

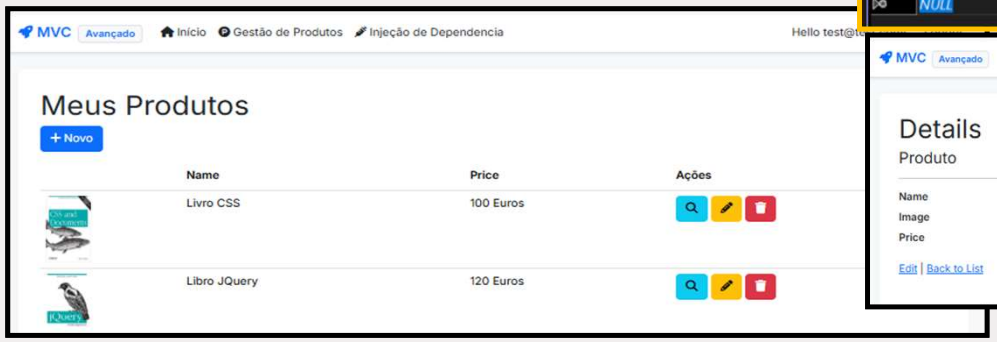
Authorization | Policies Required Claims

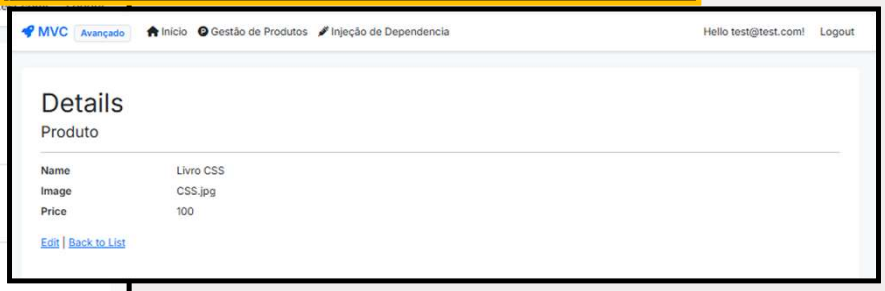
Visão geral criada por IA

No ASP.NET Core Identity, **policies que requerem claims** (declarações) são regras de autorização baseadas em atributos específicos do usuário, indo além de simples funções (roles). Elas verificam se o usuário possui uma declaração (par chave-valor, ex: `Idade: 21` ou `Departamento: TI`) antes de liberar o acesso a um recurso. Microsoft Learn +3



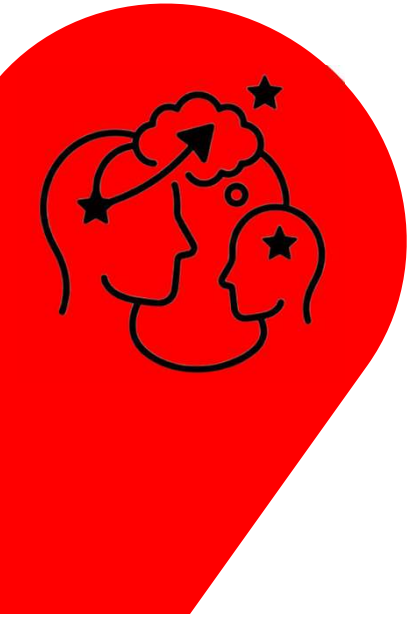


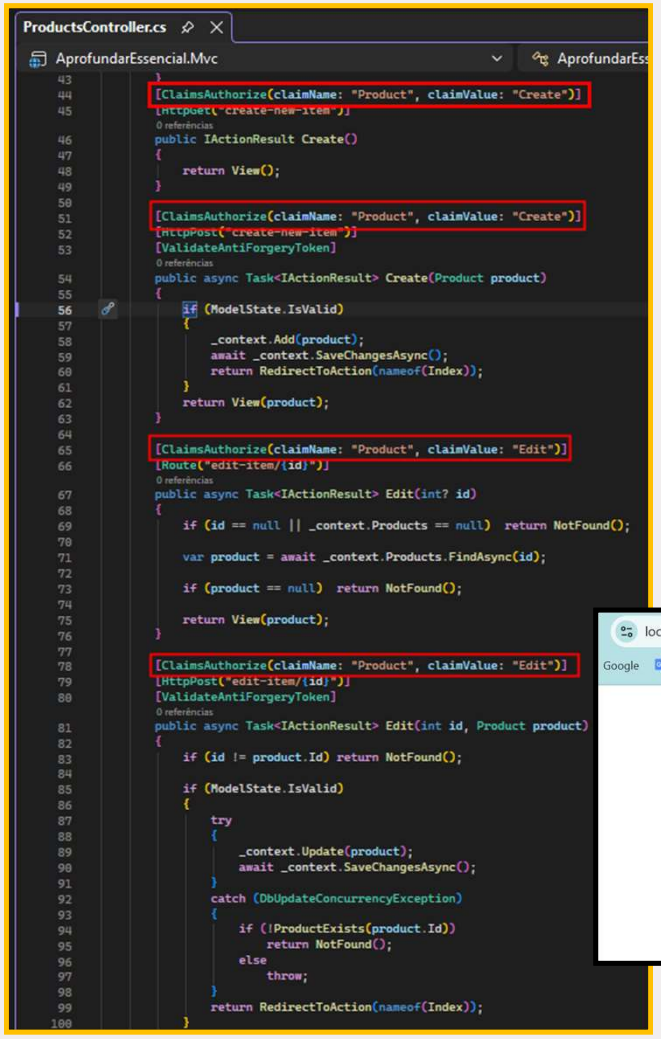


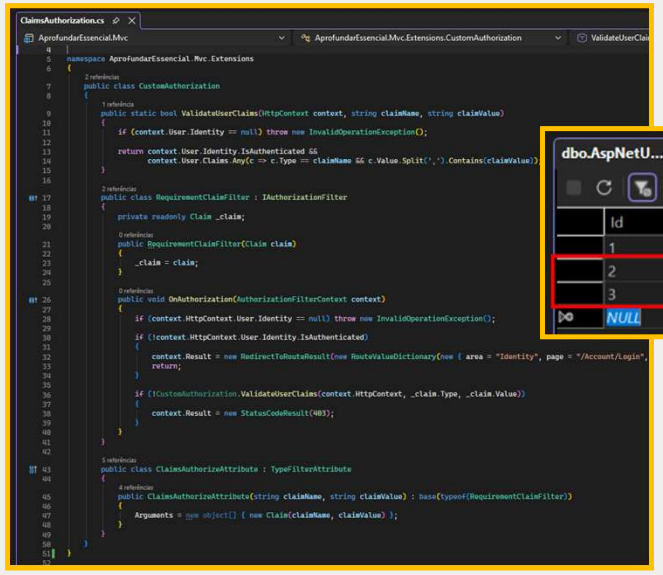


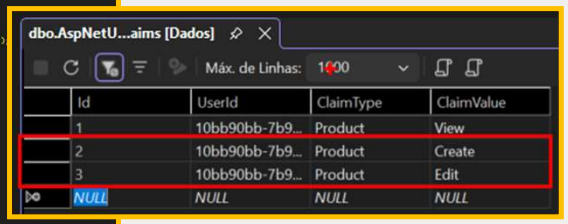
GRUPO JAP

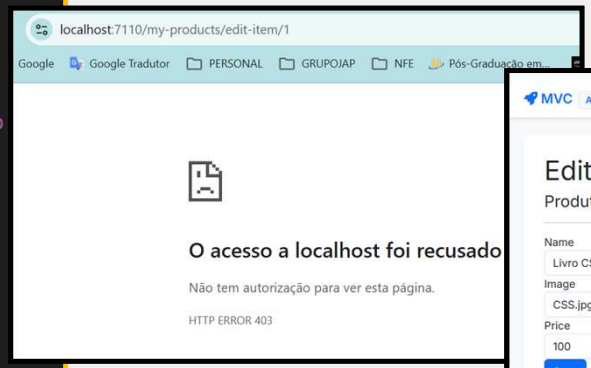
Custom Authorization





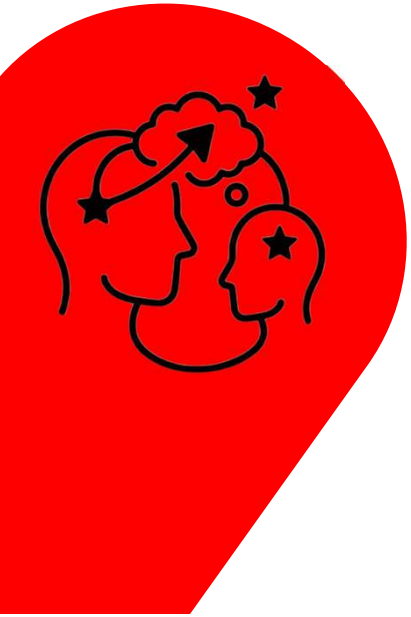


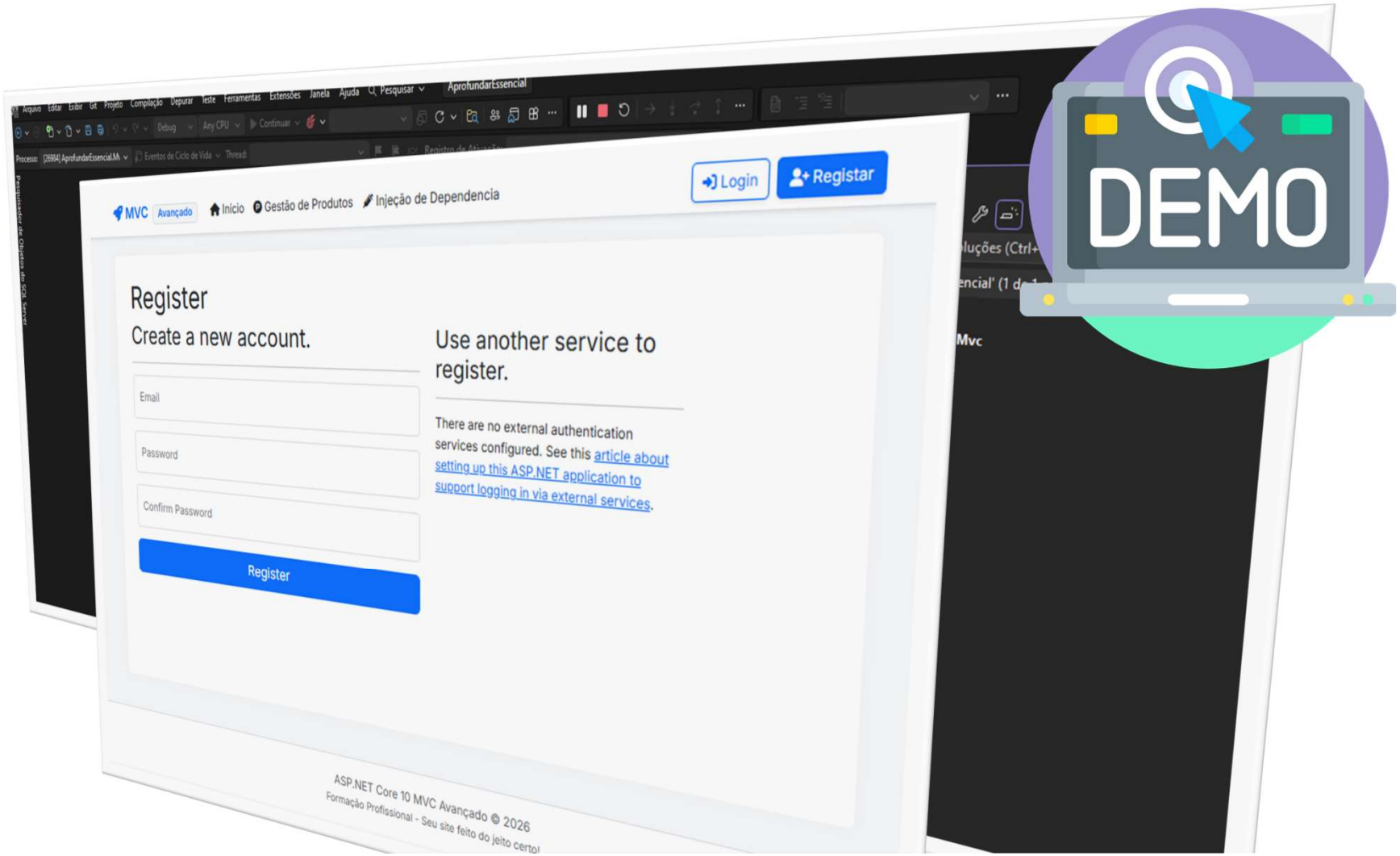




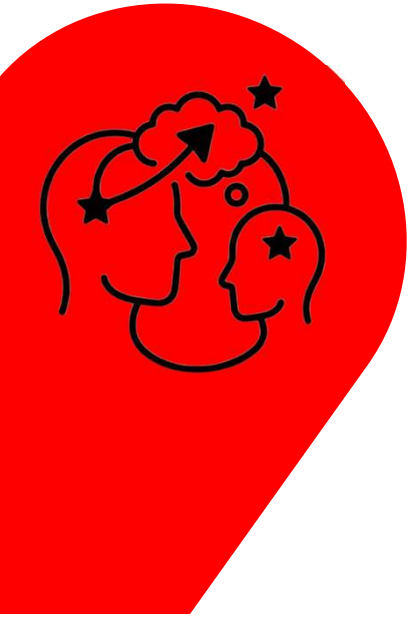
GRUPO JAP

[DEMO] - Identity | Authorization





GRUPO JAP



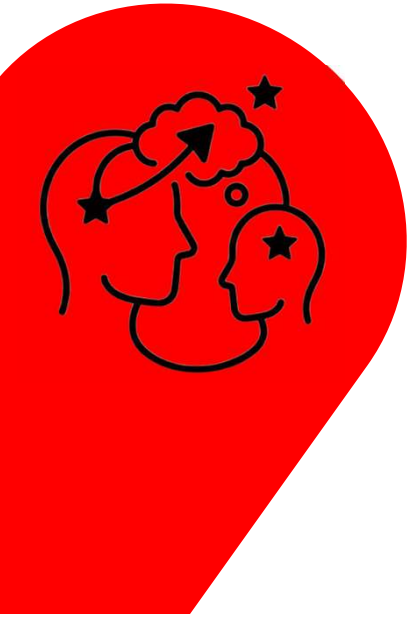

Módulo: Configurações Avançadas
DOMINANDO O PIPELINE
E O AMBIENTE DO ASP.NET CORE
1
CUSTOMIZANDO O PROGRAM.CS (DI & MIDDLEWARE PIPELINE)
Pipeline
2
GERENCIANDO AMBIE-ENTES DE EXECUÇÃO (DEV, STAGING, PROD)
Environment
3
PROTEGENDO DADOS COM USER SECRETS & LENDO CONFIGS
appsettings
ADVANCED CONFIG
4
Error
TRATAMENTO DE ERROS GLOBAL (TRY-CATCH & EXCEPTION HANDLER)
5
LOGGING EFICIENTE: REALIZANDO O LOG DE TUDO
6
TRABALHANDO COM FILTROS DE AÇÃO E VALIDAÇÃO (CROSS-CUTTING)

GRUPO JAP

Organizar a classe program.cs

Visão geral criada por IA

Extension Methods (Métodos de Extensão) em C# permitem adicionar novos métodos a tipos existentes (classes, structs ou interfaces) sem modificar o código-fonte original, herdar ou recompilar o tipo. Eles são definidos como métodos estáticos dentro de uma classe estática, usando a palavra-chave `this` no primeiro parâmetro para indicar qual tipo está sendo estendido. Microsoft Learn +3



Program.cs

```
using AprofundarEssencial.Mvc.Configurations;

var builder = WebApplication.CreateBuilder(args);

builder
    .AddMvcConfiguration()
    .AddIdentityConfiguration()
    .AddDependencyInjectionConfiguration();

var app = builder.Build();

app.UseMvcConfiguration();

app.Run();
```

MvcConfig.cs

```
using Microsoft.AspNetCore.Mvc;

namespace AprofundarEssencial.Mvc.Configurations
{
    public static class MvcConfig
    {
        public static WebApplicationBuilder AddMvcConfiguration(this WebApplicationBuilder builder)
        {
            builder.Services.AddControllersWithViews(options =>
            {
                options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
            });

            //builder.Services.AddRouting(options =>
            //    options.ConstraintMap["slugify"] = typeof(RouteSlugifyParameterTransformer));


            builder.Services.AddHsts(options =>
            {
                options.Preload = true;
                options.IncludeSubDomains = true;
                options.MaxAge = TimeSpan.FromDays(60);
                options.ExcludedHosts.Add("example.com");
                options.ExcludedHosts.Add("www.example.com");
            });

            return builder;
        }

        public static WebApplication UseMvcConfiguration(this WebApplication app)
        {
            if (app.Environment.IsDevelopment())
            {

            }
            else
            {
                app.UseHsts();
            }

            app.UseHttpsRedirection();

            app.UseStaticFiles();

            app.UseRouting();

            app.UseAuthorization();

            //app.MapControllerRoute(
            //    name: "default",
            //    pattern: "{controller:slugify=Home}/{action:slugify=Index}/{id?}");

            app.MapControllerRoute(
                name: "default",
                pattern: "{controller=Home}/{action=Index}/{id?}")
                .WithStaticAssets();

            app.MapRazorPages();

            return app;
        }
    }
}
```

DIConfig.cs

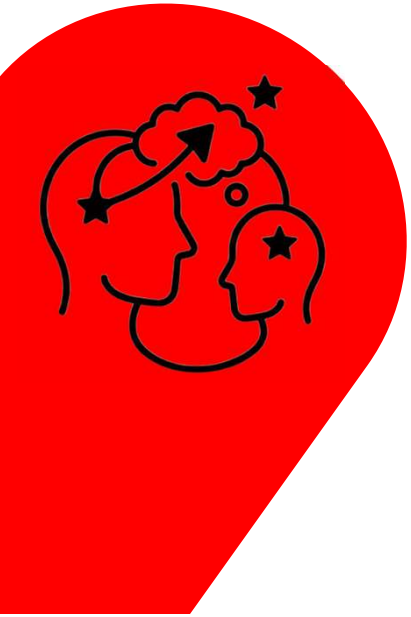
```
using AppSemTemplate.Data;
using AprofundarEssencial.Mvc.Services;
using Microsoft.EntityFrameworkCore;

namespace AprofundarEssencial.Mvc.Configurations
{
    public static class DIConfig
    {
        public static WebApplicationBuilder AddDependencyInjectionConfiguration(this WebApplicationBuilder builder)
        {
            builder.Services.AddSingleton<IHttpContextAccessor, HttpContextAccessor>();

            // Registos específicos para as interfaces especificas
            builder.Services.AddTransient<IGenera
            builder.Services.AddScoped<IGenerateG
            builder.Services.AddSingleton<IGenera

            // Registo do serviço que consome as
            builder.Services.AddTransient<Operaca

            builder.Services.AddDbContext<AppDbCo
                o.UseSqlServer(builder.Configurat

            return builder;
        }
    }
}
```

IdentityConfig.cs

```
using AppSemTemplate.Data;
using Microsoft.AspNetCore.Identity;

namespace AprofundarEssencial.Mvc.Configurations
{
    public static class IdentityConfig
    {
        public static WebApplicationBuilder AddIdentityConfiguration(this WebApplicationBuilder builder)
        {
            builder.Services.AddDefaultIdentity<IdentityUser>(options =>
            {
                options.SignIn.RequireConfirmedAccount = true;
            }).AddRoles<IdentityRole>()
              .AddEntityFrameworkStores<AppDbContext>();

            builder.Services.AddAuthorization(options =>
            {
                options.AddPolicy("CanDeleteProduct", policy =>
                        policy.RequireRole("Admin", "Manager", "Seller"));

                options.AddPolicy("ViewProduct", policy =>
                        policy.RequireClaim("Product", "View"));
            });

            return builder;
        }
    }
}
```

GRUPO JAP

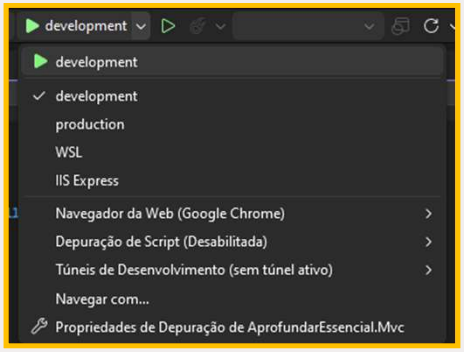
Configurar Ambientes

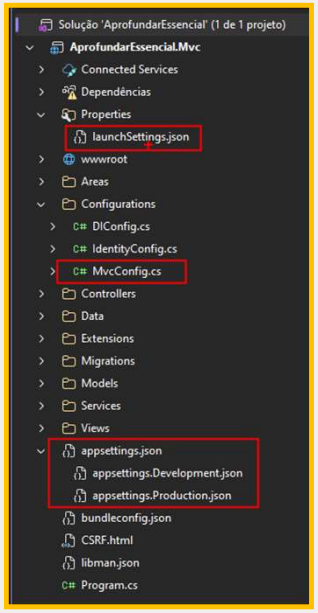


MvcConfig.cs

AprofundarEssencial.Mvc — AprofundarEssencial.Mvc.Configurations.MvcConfig

```
using Microsoft.AspNetCore.Mvc;

namespace AprofundarEssencial.Mvc.Configurations
{
    0 referências
    public static class MvcConfig
    {
        1 referência
        public static WebApplicationBuilder AddMvcConfiguration(this WebApplicationBuilder builder)
        {
            builder.Configuration
                .SetBasePath(builder.Environment.ContentRootPath)
                .AddJsonFile("appsettings.json", true, true)
                .AddJsonFile($"appsettings.{builder.Environment.EnvironmentName}.json", true, true)
                .AddEnvironmentVariables();

            builder.Services.AddControllersWithViews(options =>
            {
                options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
            });
```





appsettings.json

Esquema: https://www.schemastore.org/appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

launchSettings.json

Esquema: https://www.schemastore.org/launchsettings.json

```
{
  "profiles": {
    "development": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "https://localhost:7251;http://localhost:5211",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "production": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "applicationUrl": "https://localhost:7251;http://localhost:5211",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Production"
      }
    }
  }
}
```

appsettings....lopment.json

Esquema: https://www.s C:\Onboarding\Formacao\ASPNET-Core-MVC-Avancado\AprofundarEssencial\src\AprofundarEssencial.Mvc\appsettings.Development.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=OnboardingDotNetDb;Trusted_Connection=True;MultipleActiveResultSets=tr
  }
}
```

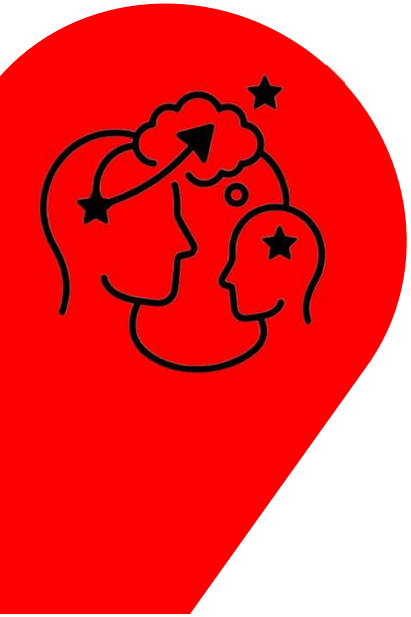
appsettings.Production.json

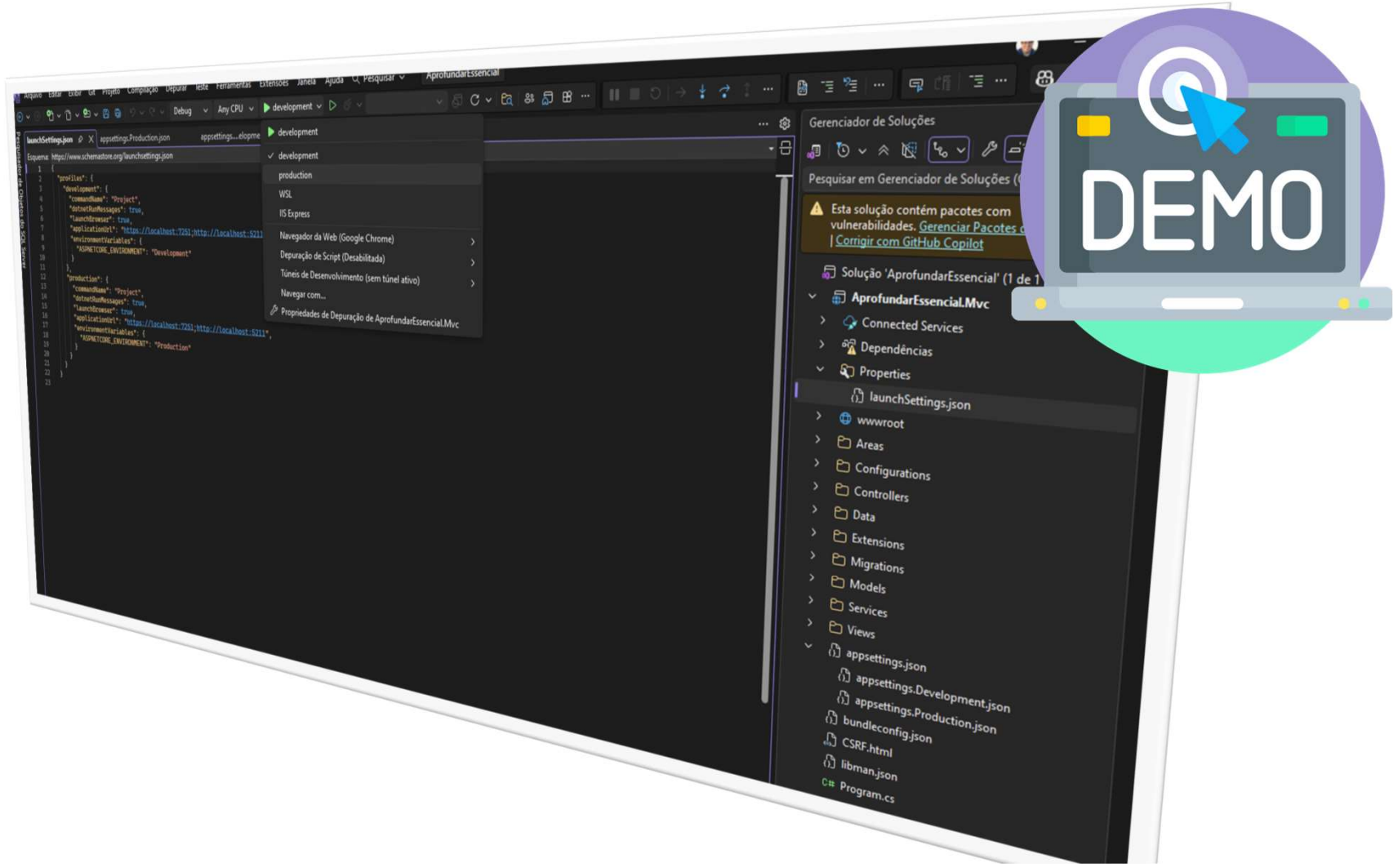
Esquema: https://www. C:\Onboarding\Formacao\ASPNET-Core-MVC-Avancado\AprofundarEssencial\src\AprofundarEssencial.Mvc\appsettings.Production.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=PROD_OnboardingDotNetDb;Trusted_Connection=True;MultipleActiveResultSets=true"
  }
}
```

GRUPO JAP

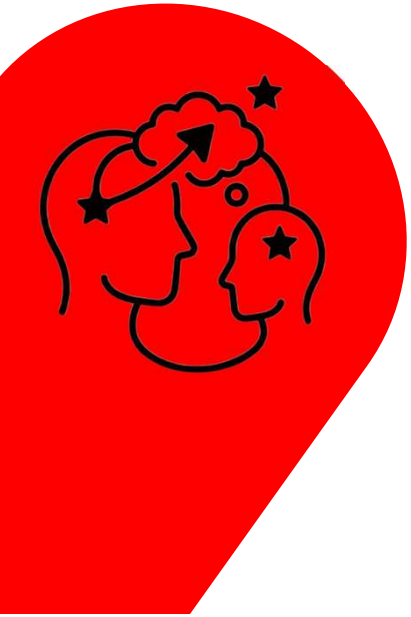
[DEMO] - Organizar e Configurar





GRUPO JAP

Ler ficheiro de configuração



```
HomeController.cs
AprofundarEssencial.Mvc
AprofundarEssencial.Mvc.Controllers.HomeController

using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Options;

namespace AprofundarEssencial.Mvc.Controllers
{
    1 referência
    public class HomeController : Controller
    {
        private readonly IConfiguration Configuration;
        private readonly ApiConfiguration ApiConfig;

        0 referências
        public HomeController(IConfiguration configuration,
                              IOptions<ApiConfiguration> apiConfiguration)
        {
            Configuration = configuration;
            ApiConfig = apiConfiguration.Value;
        }

        0 referências
        public IActionResult Index()
        {
            var env = Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT");

            // Access the configuration values directly from IConfiguration
            var myDomain_directly = Configuration[$"{ApiConfiguration.ConfigName}:Domain"] ?? "DefaultDomain";
            var userKey2_directly = Configuration[$"{ApiConfiguration.ConfigName}:UserKey"] ?? "DefaultUserKey";
            var userSecret2_directly = Configuration[$"{ApiConfiguration.ConfigName}:UserSecret"] ?? "DefaultUserSecret";

            // Access the configuration values
            var apiConfig = new ApiConfiguration();
            Configuration.GetSection(ApiConfiguration.ConfigName).Bind(apiConfig);

            var myDomain = apiConfig.Domain ?? "DefaultDomain";
            var userKey = apiConfig.UserKey ?? "DefaultUserKey";
            var userSecret = apiConfig.UserSecret ?? "DefaultUserSecret";

            return View();
```

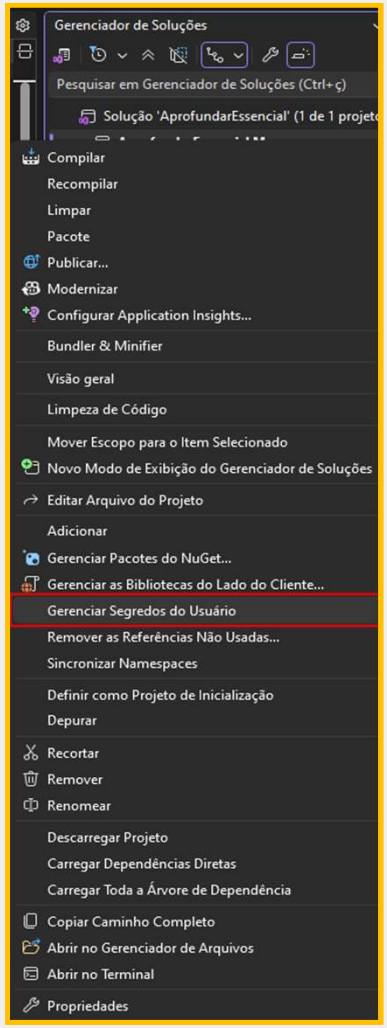
```
appsettings....lopment.json
Esquema: https://www.schemastore.org/appsettings.json

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=(localdb)\\mssqllocal
  },
  "ApiConfiguration": {
    "Domain": "https://dominio.com",
    "UserKey": "MINHACHAVESEGREDO-DEV",
    "UserSecret": "MEUSEGREDODAAPI-DEV"
  }
}
```

```
ApiConfiguration.cs
AprofundarEssencial.Mvc

namespace AprofundarEssencial.Mvc.Configurations
{
    7 referências
    public class ApiConfiguration
    {
        public const string ConfigName = "ApiConfiguration";

        1 referência
        public string? Domain { get; set; }
        1 referência
        public string? UserKey { get; set; }
        1 referência
        public string? UserSecret { get; set; }
    }
}
```

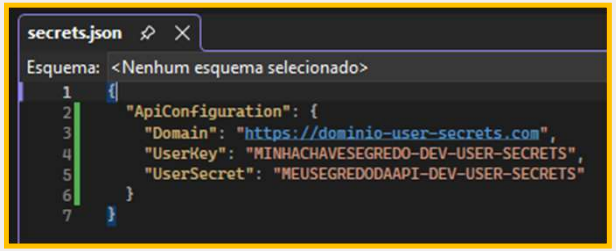
```
0 referências
public IActionResult Index()
{
    var env = Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT");

    // Access the configuration values directly from IConfiguration
    var myDomain_directly = Configuration[$"{ApiConfiguration.ConfigName}:Domain"] ?? "DefaultDomain";
    var userKey2_directly = Configuration[$"{ApiConfiguration.ConfigName}:UserKey"] ?? "DefaultUserKey";
    var userSecret2_directly = Configuration[$"{ApiConfiguration.ConfigName}:UserSecret"] ?? "DefaultUserSecret";

    // Access the configuration values
    var apiConfig = new ApiConfiguration();
    Configurat  apiConfig {AprofundarEssencial.Mvc.Configurations.ApiConfiguration}  );

    var myDomain
    var userKey
    var userSecre
        Domain      Exibir "https://dominio.com"
        UserKey     Exibir "MINHACHAVESEGREDO-DEV"
        UserSecret  Exibir "MEUSEGREDODAAPI-DEV"
        Membros estáticos

    return View();
}
```

GRUPO JAP

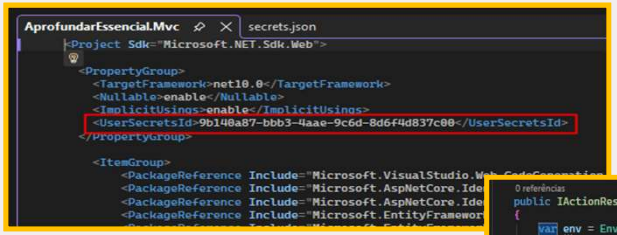
Utilizar User Secrets (secrets.json)

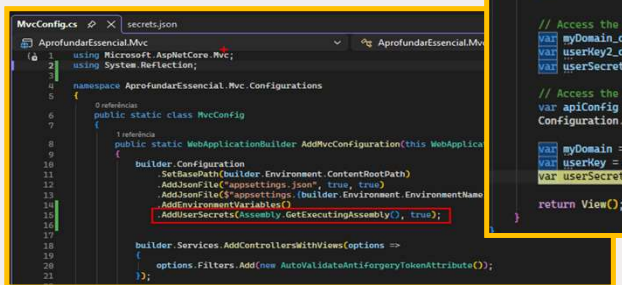


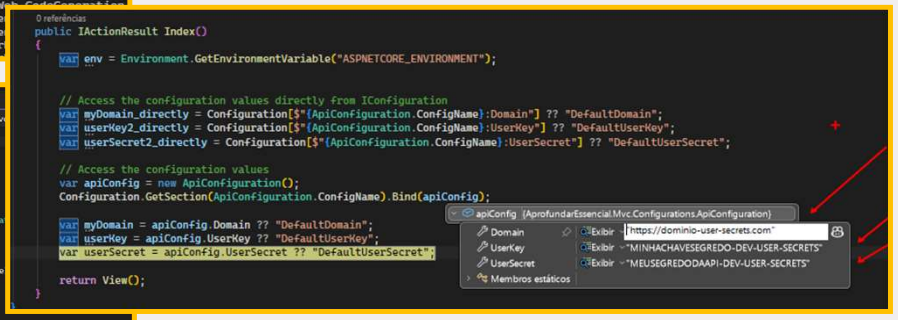
O `secrets.json` no ASP.NET Core serve para **armazenar dados sensíveis de configuração** — como chaves de API, senhas de banco de dados e strings de conexão — durante o **desenvolvimento local**. Ele impede que informações confidenciais sejam acidentalmente enviadas (comitadas) para o controle de código-fonte (como Git/GitHub). Microsoft Learn +1



C:\Users\Carlos\AppData\Roaming\Microsoft\UserSecrets

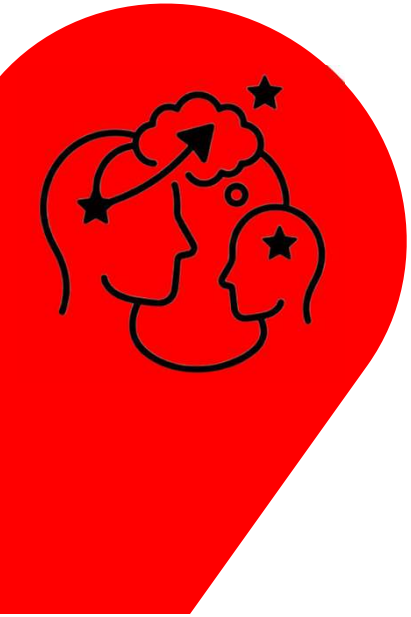


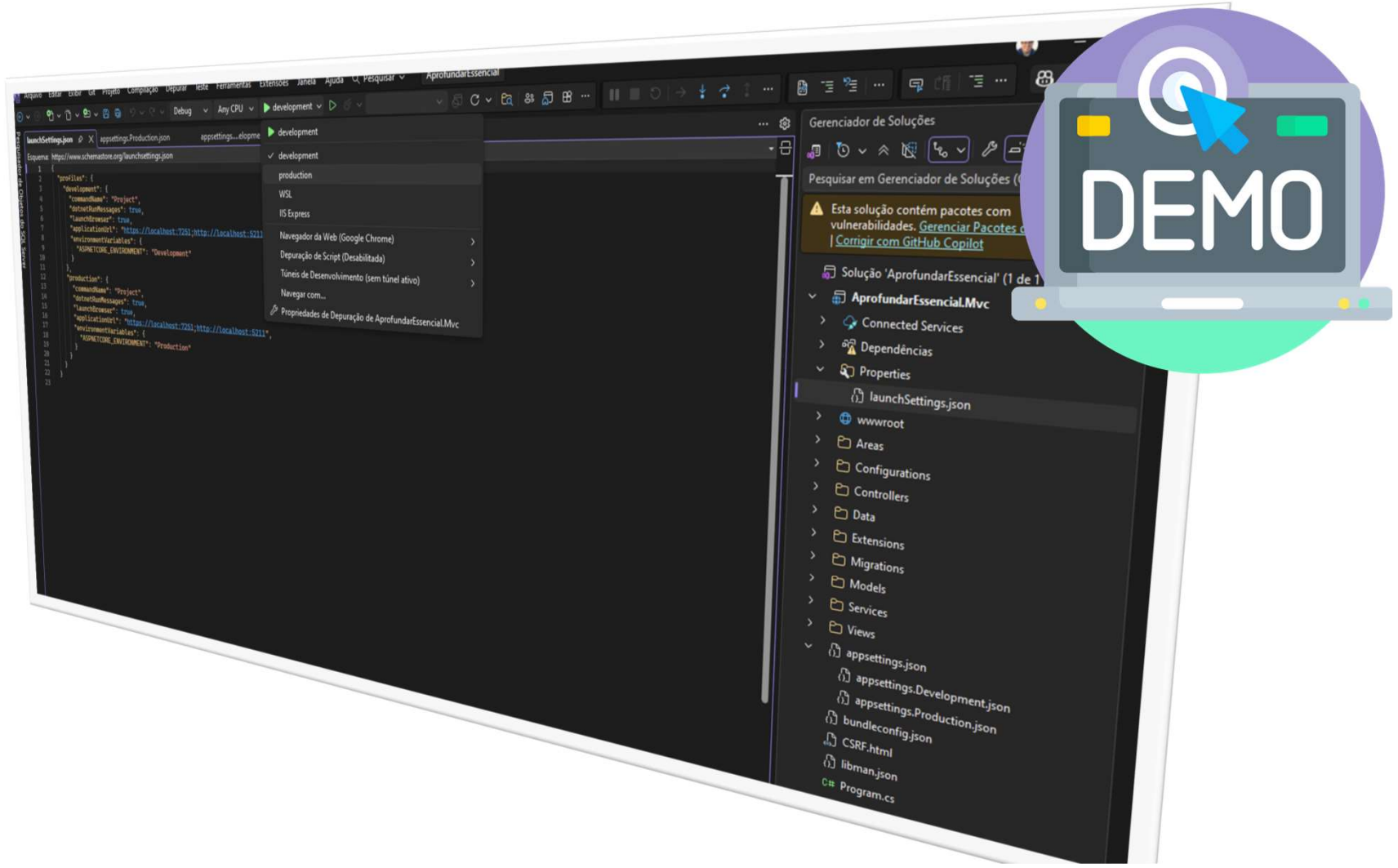




GRUPO JAP

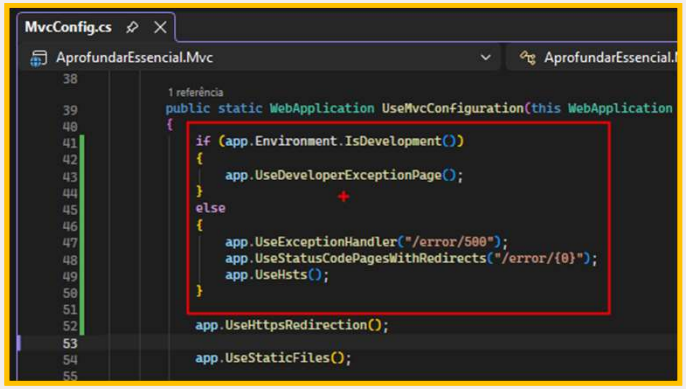
[DEMO] - Ler ficheiro e User Secrets

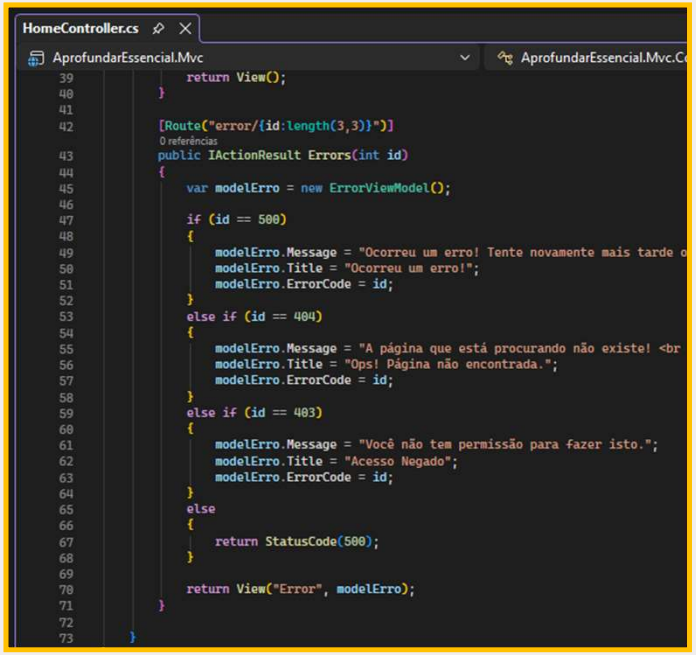


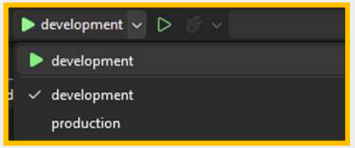


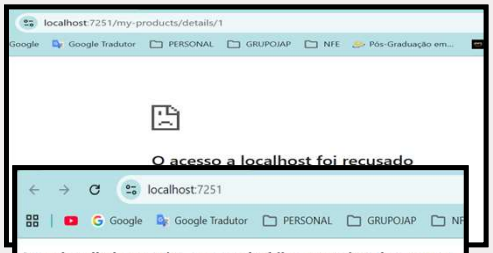
GRUPO JAP

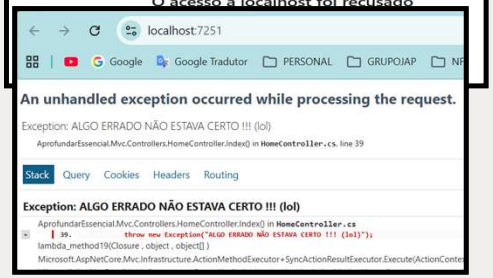
Tratamento de Erros e Middlewares

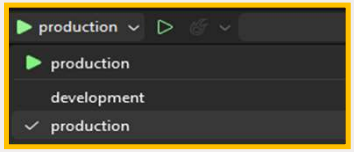


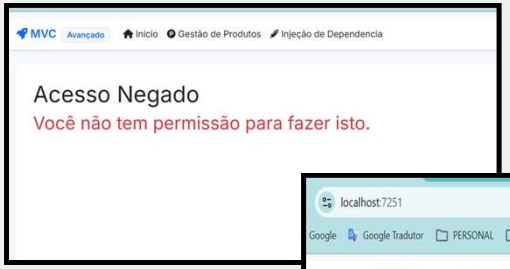


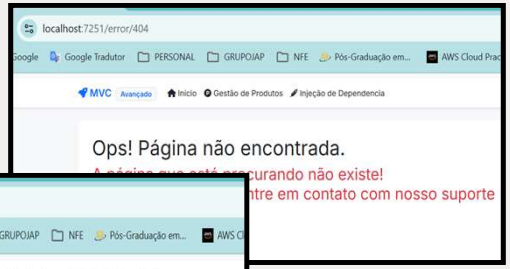


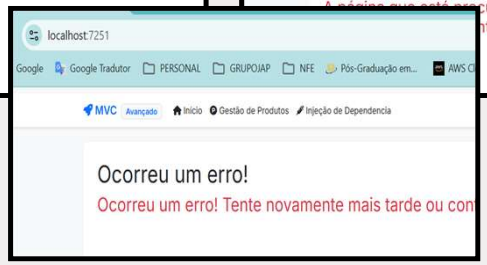






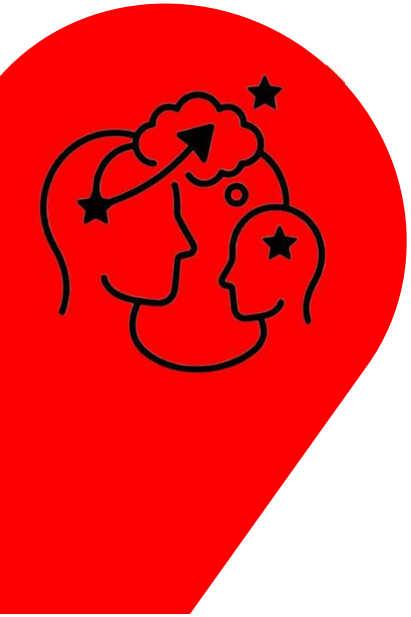






GRUPO JAP

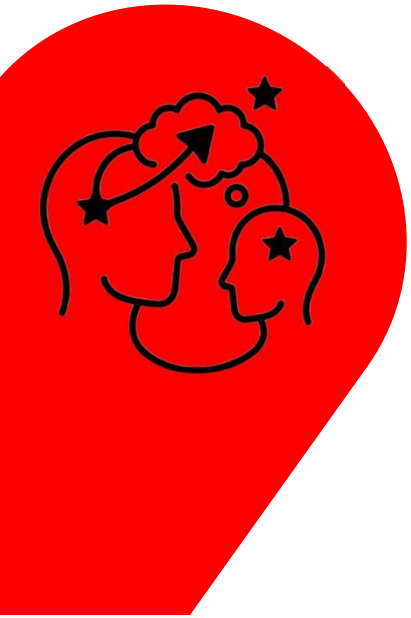
[DEMO] - Tratamento de Erros



GRUPO JAP

Logs para Observabilidade e Monitoria #1

Logs no ASP.NET Core são registros automáticos e estruturados de eventos, erros e atividades que ocorrem durante a execução de uma aplicação, fundamentais para monitoramento, depuração e diagnóstico de problemas. Eles utilizam a API ILogger e podem ser enviados para diversos destinos (console, arquivos, nuvem) usando provedores (*providers*). Microsoft Learn +2



Configure ASP.NET Core SDK

.NET

Error Monitoring | Logs | Metrics | Tracing

Install

Copy instructions

Install the **NuGet** package:

Package Manager | .NET Core CLI

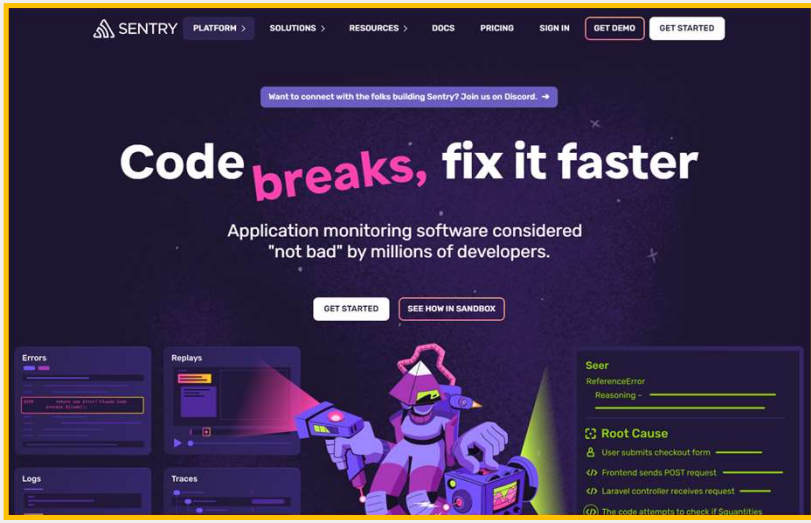
```
Install-Package Sentry.AspNetCore -Version 6.4.1
```

Configure SDK

Copy DSN

Add Sentry to `Program.cs` through the `WebHostBuilder`:

```
public static IHostBuilder CreateHostBuilder(string[] args) =>
  Host.CreateDefaultBuilder(args)
      .ConfigureWebHostDefaults(webBuilder =>
      {
          // Add the following line:
          webBuilder.UseSentry(o =>
          {
              o.Dsn = "https://747bc5778051dd5294aad3eaee781b79@o4511277275873280.ingest.us.sentry.io/45112772816404
              // When configuring for the first time, to see what the SDK is doing:
              o.Debug = true;
          });
      });
```



https://sentry.io

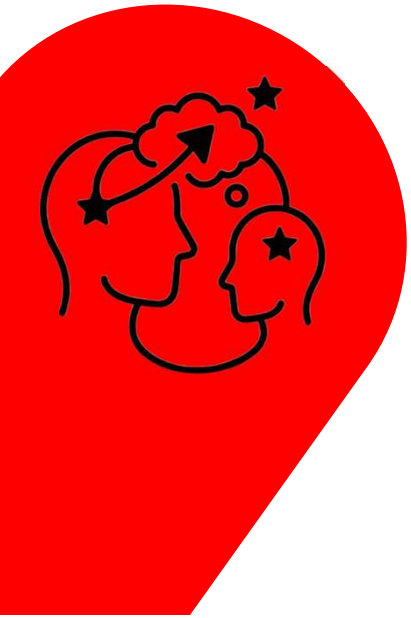
secrets.json

Esquema: <Nenhum esquema selecionado>

```
{
  "ApiConfiguration": {
    "Domain": "https://dominio-user-secrets.com",
    "UserKey": "MINHACHAVESEGREDO-DEV-USER-SECRETS",
    "UserSecret": "MEUSEGREDODAAPI-DEV-USER-SECRETS"
  },
  "SentryConfiguration": {
    "Dsn": "https://747bc5778051dd5294aad3eaee781b79@o4511277275873280.ingest.us.sentry.io/4511277281640448",
    "Debug": true,
    "TracesSampleRate": 1.0
  }
}
```

GRUPO JAP

Logs para Observabilidade e Monitoria #2



LoggingConfig.cs

```
AprofundarEssencial.Mvc                AprofundarEssencial.Mvc.Configurations.LoggingConfig
1  using Sentry.Extensibility;
2
3  namespace AprofundarEssencial.Mvc.Configurations
4  {
       0 referências
5      public static class LoggingConfig
6      {
           1 referência
7          public static WebApplicationBuilder AddLoggingConfiguration(this WebApplicationBuilder builder)
8          {
9              // Aqui integramos o Sentry diretamente no WebHost
10             builder.WebHost.UseSentry(options =>
11             {
12                 // Puxamos a DSN do appsettings.json para segurança
13                 builder.Configuration.GetSection("SentryConfiguration").Bind(options);
14
15                 // Configurações para demonstração em aula
16                 options.TracesSampleRate = 1.0;
17                 options.IncludeActivityData = true;
18                 options.MaxRequestBodySize = RequestSize.Always;
19                 options.SendDefaultPii = true; // Util para ver o usuário do Identity no log
20             });
21
22             return builder;
23         }
24     }
25 }
```

Program.cs | LoggingConfig.cs

```
AprofundarEssencial.Mvc
1  using AprofundarEssencial.Mvc.Configurations;
2
3  var builder = WebApplication.CreateBuilder(args);
4
5  builder
6      .AddMvcConfiguration()
7      .AddIdentityConfiguration()
8      .AddDependencyInjectionConfiguration()
9      .AddLoggingConfiguration();
10
11 var app = builder.Build();
12
13 app.UseMvcConfiguration();
14
15 app.Run();
16
```

HomeController.cs

```
AprofundarEssencial.Mvc                AprofundarEssencial.Mvc.Controllers.HomeController
74
75         [Route("home-error")]
           0 referências
76         public IActionResult TestError()
77         {
78             throw new Exception("Na Controller Home gerar erro de demonstração para o Módulo Avançado!");
79         }
80
81     }
82 }
```

HomeController.cs

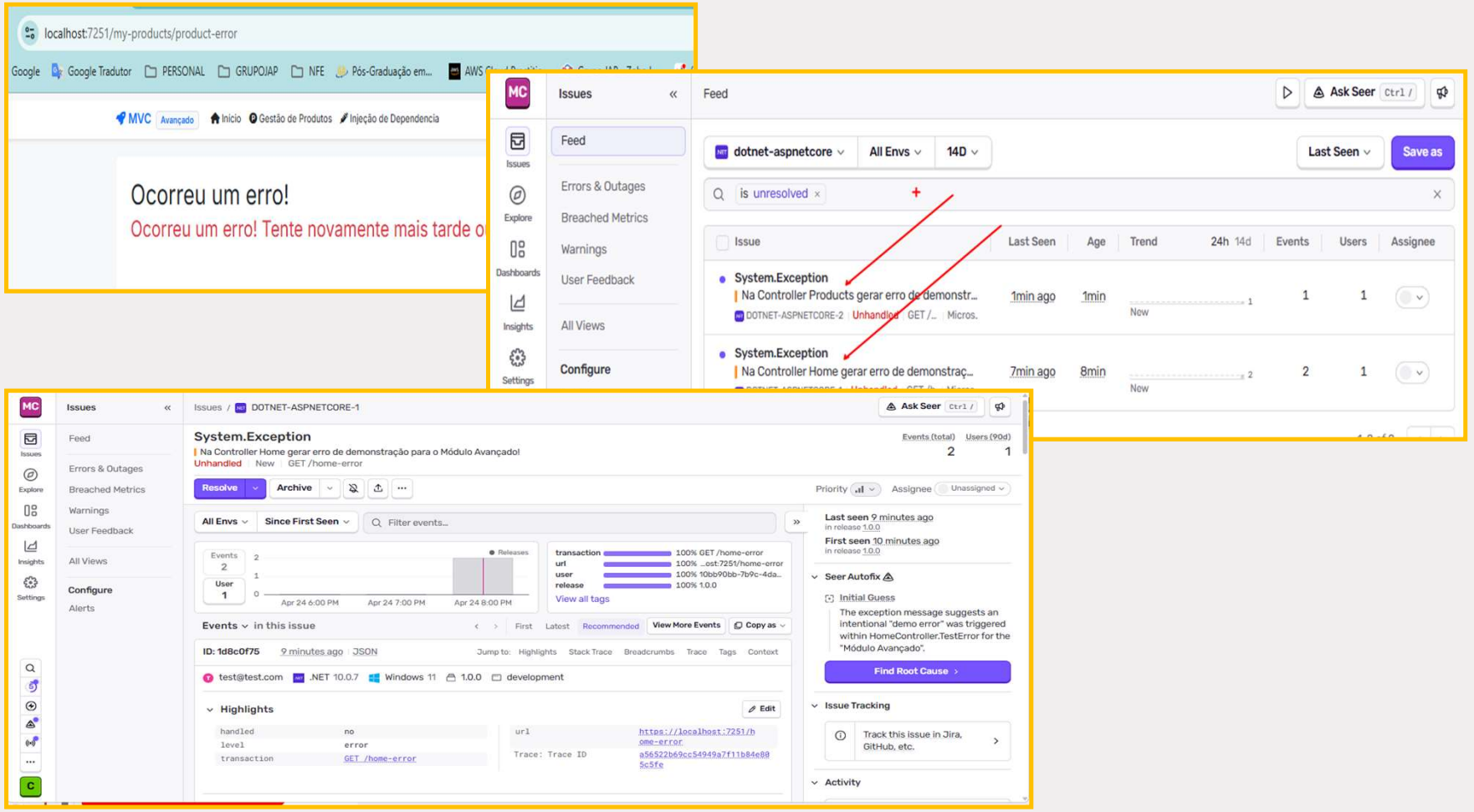
```
AprofundarEssencial.Mvc                AprofundarEssencial.Mvc.Controllers.HomeController
43
44         [Route("error/{id:length(3,3)}")]
           0 referências
45         public IActionResult Errors(int id)
46         {
47             var modelErro = new ErrorViewModel();
48
49             if (id == 500)
50             {
51                 modelErro.Message = "Ocorreu um erro! Tente novamente mais tarde ou contate nosso suporte.";
52                 modelErro.Title = "Ocorreu um erro!";
53                 modelErro.ErrorCode = id;
54
55                 SentrySdk.CaptureMessage($"Erro HTTP detectado: {id}", SentryLevel.Fatal);
56             }
57             else if (id == 404)
58             {
59                 modelErro.Message = "A página que está procurando não existe! <br />Em caso de dúvidas entre
60                 modelErro.Title = "Ops! Página não encontrada.";
61                 modelErro.ErrorCode = id;
62
63                 SentrySdk.CaptureMessage($"Erro HTTP detectado: {id}", SentryLevel.Warning);
64
65             }
66             else if (id == 403)
67             {
68                 modelErro.Message = "Você não tem permissão para fazer isto.";
69                 modelErro.Title = "Acesso Negado";
70                 modelErro.ErrorCode = id;
71
72                 SentrySdk.CaptureMessage($"Erro HTTP detectado: {id}", SentryLevel.Warning);
73
74             }
75             else
76             {
77                 SentrySdk.CaptureMessage($"Erro HTTP detectado: {id}", SentryLevel.Fatal);
78
79                 return StatusCode(500);
80             }
81
82             return View("Error", modelErro);
83         }
```

ProductsController.cs

```
AprofundarEssencial.Mvc                AprofundarEssencial.Mvc.Controllers.ProductsController
132
133        [AllowAnonymous]
134        [Route("product-error")]
           0 referências
135        public IActionResult TestError()
136        {
137            throw new Exception("Na Controller Products gerar erro de demonstração para o Módulo Avançado!");
138        }
```

GRUPO JAP

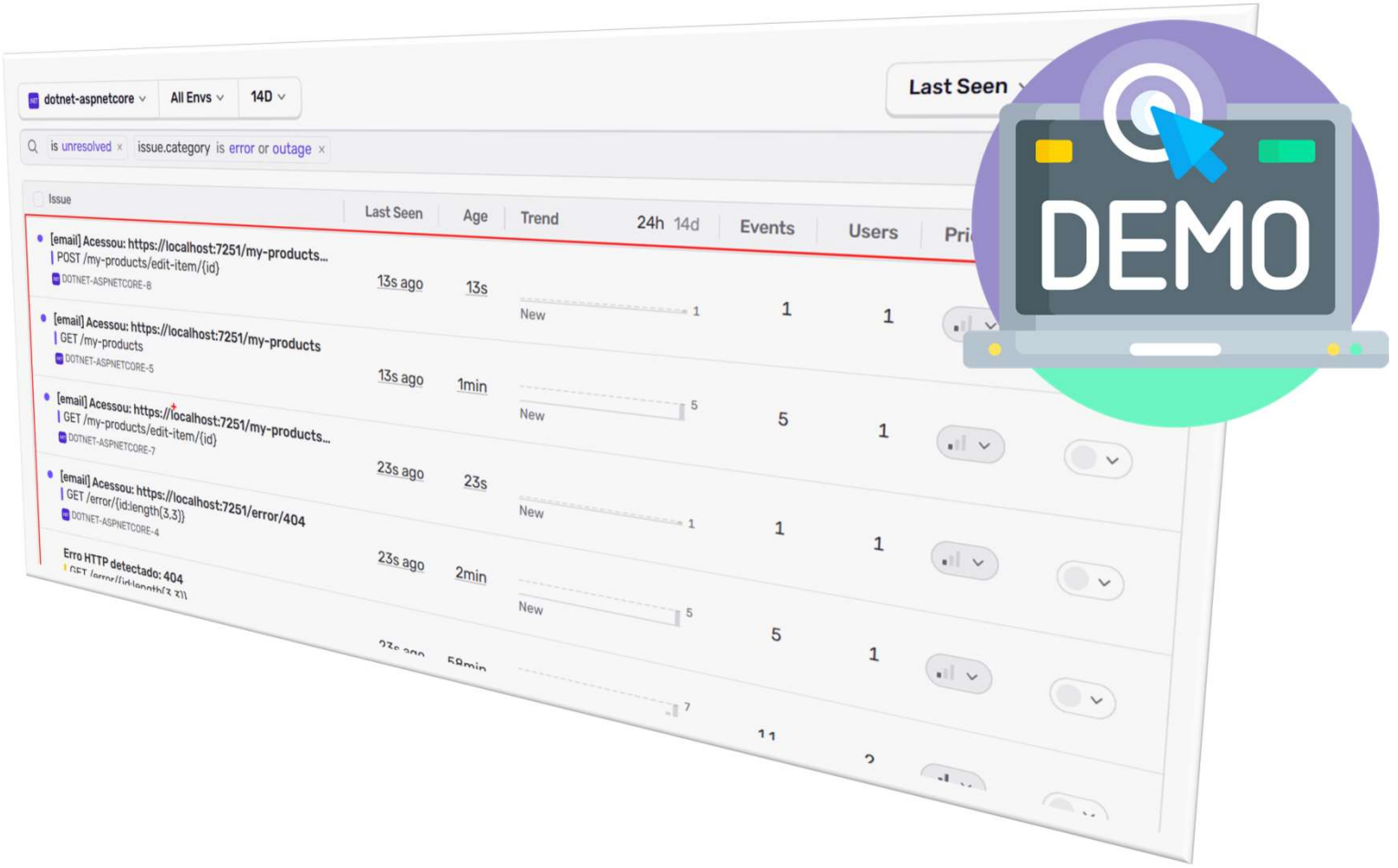
Logs para Observabilidade e Monitoria #3

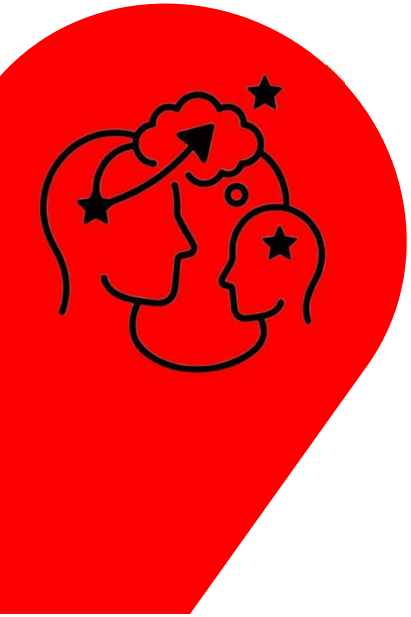


https://my-own-company-em.sentry.io/issues/errors-outages/?project=4511277281640448

GRUPO JAP

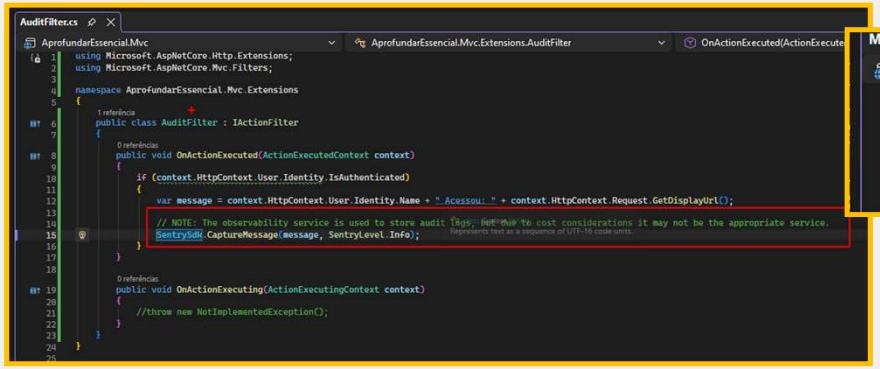
[DEMO] - Observabilidade e Monitoria

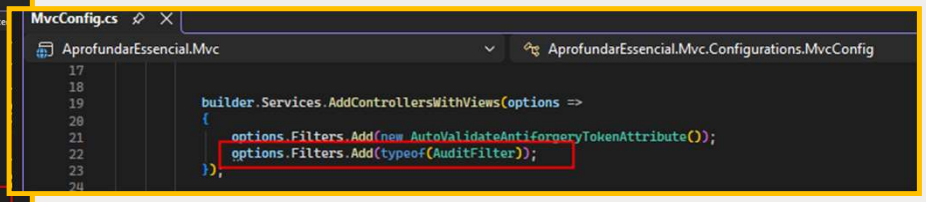


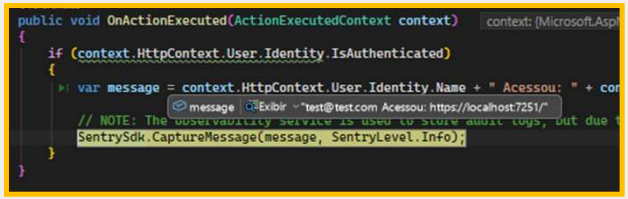


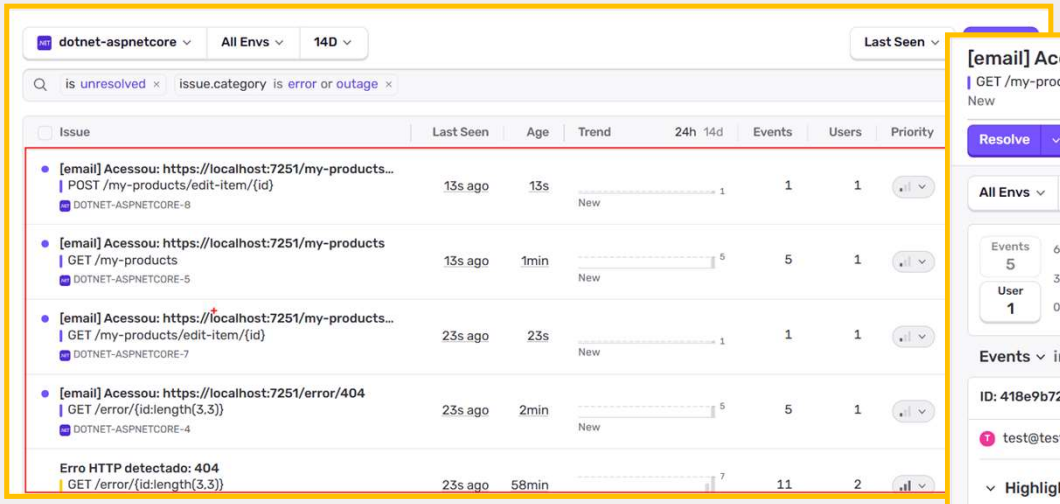
GRUPO JAP

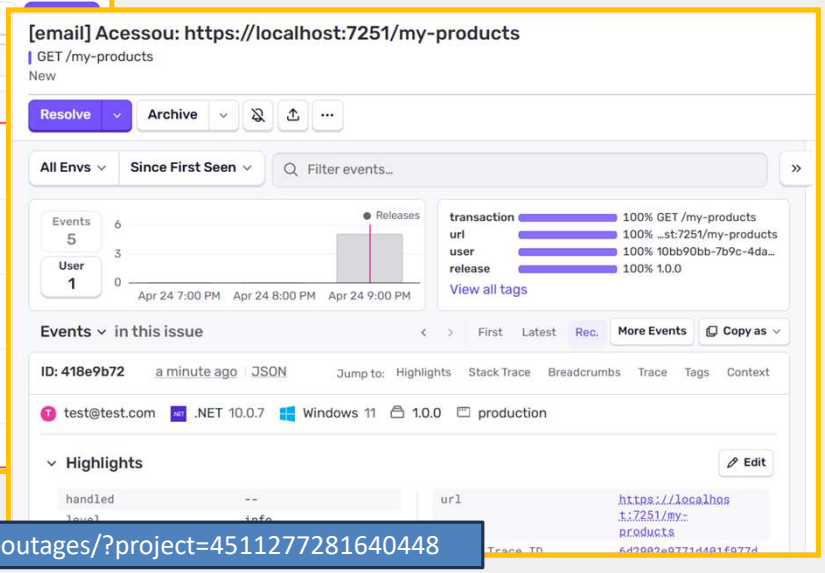
Auditoria/Log através de Filtro







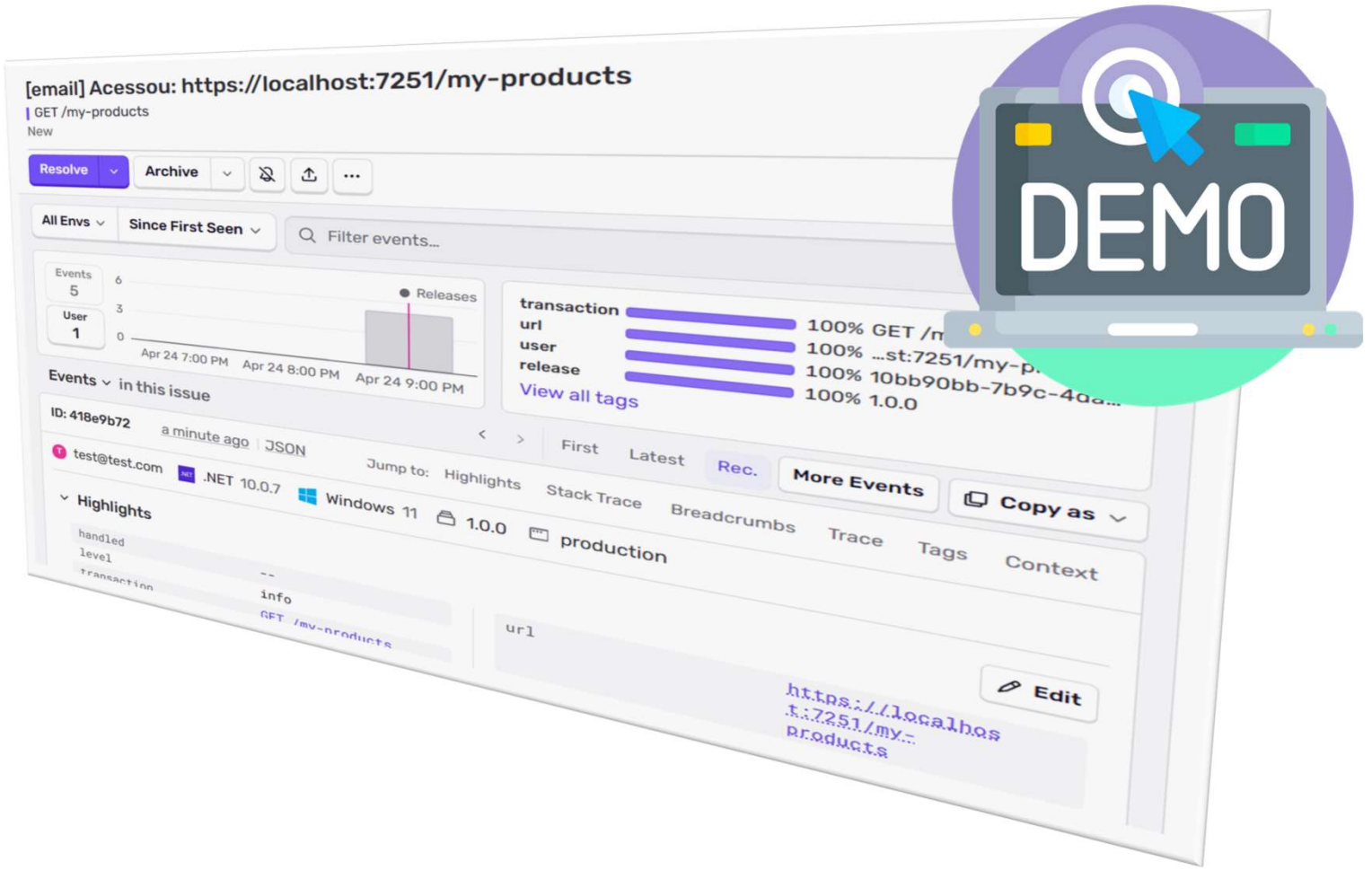




https://my-own-company-em.sentry.io/issues/errors-outages/?project=4511277281640448

GRUPO JAP

[DEMO] - Monitoria com Filtro



GRUPO JAP

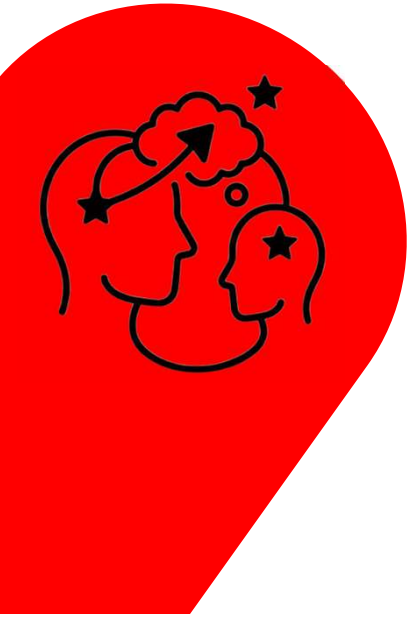

Módulo: Globalização da Aplicação
PREPARANDO SUA APP ASP.NET CORE
PARA O MERCADO MUNDIAL
1
TRABALHANDO
COM CULTURAS
(I18N & L10N)
pt-BR
2
AJUSTANDO
VALIDAÇÕES
FRONT-END
Validation
GLOBALIZAÇÃO
DA APLICAÇÃO
3
CRIANDO UM
DATA
ANNOTATION
CUSTOMIZADO
[MinAge(18)]
4
ADICIONANDO
SUPORTE A
OUTROS
IDIOMAS
.resx

GRUPO JAP

Trabalhar com Culturas

https://learn.microsoft.com/en-us/aspnet/core/fundamentals/localization?view=aspnetcore-10.0

Globalização (G11N) e localização (L10N) no ASP.NET Core **permitem que aplicativos suportem múltiplos idiomas, formatos de data/moeda e regiões sem refazer o código**. A globalização prepara o app para várias culturas, enquanto a localização traduz e ajusta o conteúdo (textos, datas) para locais específicos. Microsoft Learn +2



GlobalizationConfig.cs

```cs
using Microsoft.AspNetCore.Localization;
using System.Globalization;

namespace AprofundarEssencial.Mvc.Configurations
{
    0 referências
    public static class GlobalizationConfig
    {
        1 referência
        public static WebApplication UseGlobalizationConfig(this WebApplication app)
        {
            var defaultCulture = new CultureInfo("pt-PT");

            var localizationOptions = new RequestLocalizationOptions
            {
                DefaultRequestCulture = new RequestCulture(defaultCulture),
                SupportedCultures = new List<CultureInfo> { defaultCulture },
                SupportedUICultures = new List<CultureInfo> { defaultCulture }
            };

            app.UseRequestLocalization(localizationOptions);

            return app;
        }
    }
}
```

MvcConfig.cs

```cs
1 referência
public static WebApplication UseMvcConfiguration(this WebApplication app)
{
    if (app.Environment.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();
    }
    else
    {
        app.UseExceptionHandler("/error/500");
        app.UseStatusCodePagesWithRedirects("/error/{0}");
        app.UseHsts();
    }

    app.UseGlobalizationConfig();

    app.UseHttpsRedirection();
```

_LayoutAdmin.cshtml

```cshtml
<footer class="border-top footer text-muted mt-5">
    <div class="container text-center">
        <p class="mb-0">ASP.NET Core 10 MVC Avançado &copy; @DateTime.Now.Year</p>
        <small>Formação Profissional - Seu site feito do jeito certo!</small>
        <p class="mb-0">Idioma selecionado: @System.Globalization.CultureInfo.CurrentCulture.Name</p>
    </div>
</footer>
```

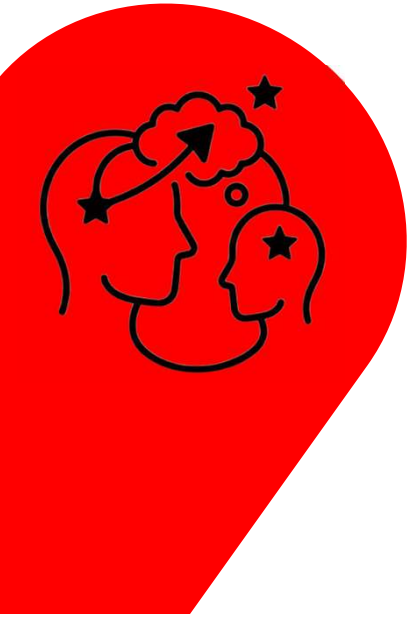
ASP.NET Core 10 MVC Avançado © 2026

Formação Profissional - Seu site feito do jeito certo!

Idioma selecionado: pt-PT

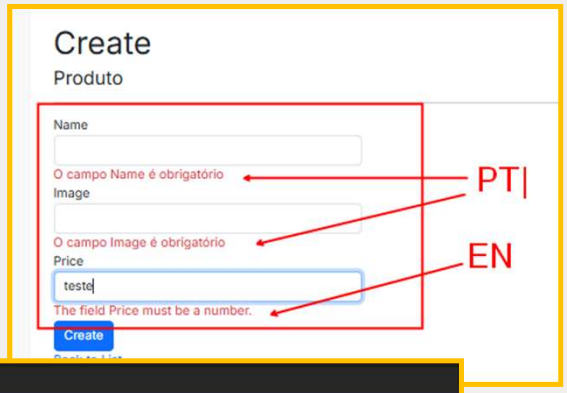
GRUPO JAP

Ajustar validações do front-end



```
Product.cs
AprofundarEssencial.Mvc
using System.ComponentModel.DataAnnotations;

namespace AprofundarEssencial.Mvc.Models
{
    13 referências
    public class Product
    {
        [Key]
        11 referências
        public int Id { get; set; }

        [Required(ErrorMessage = "O campo {0} é obrigatório")]
        12 referências
        public string? Name { get; set; }

        [Required(ErrorMessage = "O campo {0} é obrigatório")]
        11 referências
        public string? Image { get; set; }

        [Required(ErrorMessage = "O campo {0} é obrigatório")]
        12 referências
        public decimal Price { get; set; }
    }
}
```

PS > dotnet ef migrations add AlterProducts
PMC > Add-Migrations AlterProducts
e
PS > dotnet ef database update
PMC > Update-Database



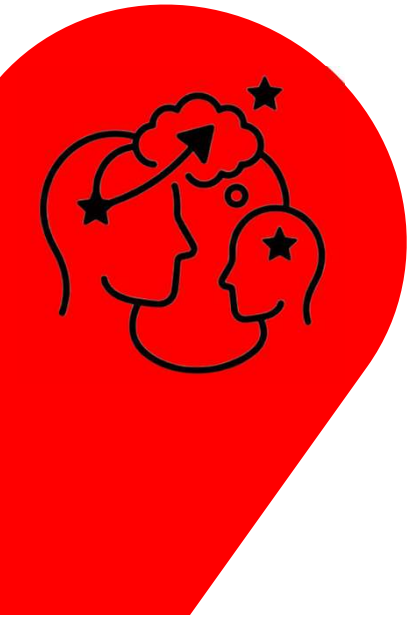

```
MvcConfig.cs
AprofundarEssencial.Mvc
AprofundarEssencial.Mvc.Configurations.MvcConfig

builder.Services.AddControllersWithViews(options =>
{
    options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
    options.Filters.Add(typeof(AuditFilter));

    MvcOptionsConfig.ConfigureModelBindingMessages(options.ModelBindingMessag
});
```

```
_ValidationSc...artial.cshtml
AprofundarEssencial.Mvc
AspNetCoreGeneratedDocument.Views_Shared__ValidationSc

<environment names="Development">
    <script src="~/lib/jquery-validation/jquery.validate.js"></script>
    <script src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.js"></script>
</environment>
<environment names="Staging,Production">
    <script src="https://ajax.aspnetcdn.com/ajax/jquery.validate/1.17.0/jquery.validate.min.js"
            asp-fallback-src="~/lib/jquery-validation/jquery.validate.min.js"
            asp-fallback-test="window.jQuery && window.jQuery.validator"
            crossorigin="anonymous"
            integrity="sha384-rZfj/ogBloos6wzLGpPkkOr/gpkBNLZ6b6yLy4o+ok+t/SAKLL5mvXLr0OXNi1Hp">
    </script>
    <script src="https://ajax.aspnetcdn.com/ajax/jquery.validation.unobtrusive/3.2.9/jquery.validate.unobtrusive.min.js"
            asp-fallback-src="~/lib/jquery-validation-unobtrusive/jquery.validate.unobtrusive.min.js"
            asp-fallback-test="window.jQuery && window.jQuery.validator && window.jQuery.validator.unobtrusive"
            crossorigin="anonymous"
            integrity="sha384-ifv0TYDWxBHzvAk2Z0n8R434FL1Rlv/Av18DXE43N/1rvHyOG4izKst0f2iSLdds">
    </script>
</environment>

<script>
    $.validator.methods.range = function (value, element, param) {
    var globalizedValue = value.replace(",", ".");
    return this.optional(element) || (globalizedValue >= param[0] && globalizedValue <= param[1]);
    };

    $.validator.methods.number = function (value, element) {
    return this.optional(element) || /-?(?:\d+|\d{1,3}(?:[\s\.,]\d{3})+)(?:[\.,]\d+)?$/.test(value);
    };

    $.validator.methods.date = function (value, element) {
    var date = value.split("/");
    return this.optional(element) || !/Invalid|NaN/.test(new Date(date[2], date[1], date[0]).toString());
    };
</script>
```

```
MvcOptions.cs
MvcConfig.cs
AprofundarEssencial.Mvc
AprofundarEssencial.Mvc.Configurations.MvcOptionsConfig

using Microsoft.AspNetCore.Mvc.ModelBinding.Metadata;

namespace AprofundarEssencial.Mvc.Configurations
{
    1 referência
    public class MvcOptionsConfig
    {
        1 referência
        public static void ConfigureModelBindingMessages(DefaultModelBindingMessageProvider messageProvider)
        {
            messageProvider.SetAttemptedValueIsInvalidAccessor((x, y) => "O valor preenchido é inválido para es
            messageProvider.SetMissingBindRequiredValueAccessor(x => "Este campo precisa ser preenchido.");
            messageProvider.SetMissingKeyOrValueAccessor(() => "Este campo precisa ser preenchido.");
            messageProvider.SetMissingRequestBodyRequiredValueAccessor(() => "É necessário que o body na requi
            messageProvider.SetNonPropertyAttemptedValueIsInvalidAccessor(x => "O valor preenchido é inválido p
            messageProvider.SetNonPropertyUnknownValueIsInvalidAccessor(() => "O valor preenchido é inválido p
            messageProvider.SetNonPropertyValueMustBeANumberAccessor(() => "O campo deve ser numérico");
            messageProvider.SetUnknownValueIsInvalidAccessor(x => "O valor preenchido é inválido para este camp
            messageProvider.SetValueIsInvalidAccessor(x => "O valor preenchido é inválido para este campo.");
            messageProvider.SetValueMustBeANumberAccessor(x => "O campo deve ser numérico.");
            messageProvider.SetValueMustNotBeNullAccessor(x => "Este campo precisa ser preenchido.");
        }
    }
}
```

GRUPO JAP

Criar Data Annotation Customizado



CoinAttribute.cs

AprofundarEssencial.Mvc — AprofundarEssencial.Mvc.Extensions.CoinAttrib

```
using Microsoft.AspNetCore.Mvc.DataAnnotations;
using Microsoft.AspNetCore.Mvc.ModelBinding.Validation;
using Microsoft.Extensions.Localization;
using System.ComponentModel.DataAnnotations;
using System.Globalization;

namespace AprofundarEssencial.Mvc.Extensions
{
    4 referências
    public class CoinAttribute : ValidationAttribute
    {
        0 referências
        protected override ValidationResult IsValid(object value, ValidationContext validationContext)
        {
            try
            {
                var coin = Convert.ToDecimal(value, new CultureInfo("pt-PT"));
            }
            catch (Exception)
            {
                return new ValidationResult("Moeda em formato inválido");
            }

            return ValidationResult.Success;
        }
    }

    2 referências
    public class CoinAttributeAdapter : AttributeAdapterBase<CoinAttribute>
    {
        1 referência
        public CoinAttributeAdapter(CoinAttribute attribute, IStringLocalizer stringLocalizer) : base(attribute, stringLocalizer)
        {

        }

        0 referências
        public override void AddValidation(ClientModelValidationContext context)
        {
            if (context == null)
            {
                throw new ArgumentNullException(nameof(context));
            }

            MergeAttribute(context.Attributes, "data-val", "true");
            MergeAttribute(context.Attributes, "data-val-coin", GetErrorMessage(context));
            MergeAttribute(context.Attributes, "data-val-number", GetErrorMessage(context));
        }
        2 referências
        public override string GetErrorMessage(ModelValidationContextBase validationContext)
        {
            return "Moeda em formato inválido";
        }
    }

    1 referência
    public class CoinValidationAttributeAdapterProvider : IValidationAttributeAdapterProvider
    {
        private readonly IValidationAttributeAdapterProvider _baseProvider = new ValidationAttributeAdapterProvider();
        0 referências
        public IAttributeAdapter GetAttributeAdapter(ValidationAttribute attribute, IStringLocalizer stringLocalizer)
        {
            if (attribute is CoinAttribute coinAttribute)
            {
                return new CoinAttributeAdapter(coinAttribute, stringLocalizer);
            }

            return _baseProvider.GetAttributeAdapter(attribute, stringLocalizer);
        }
    }
}
```

DIConfig.cs

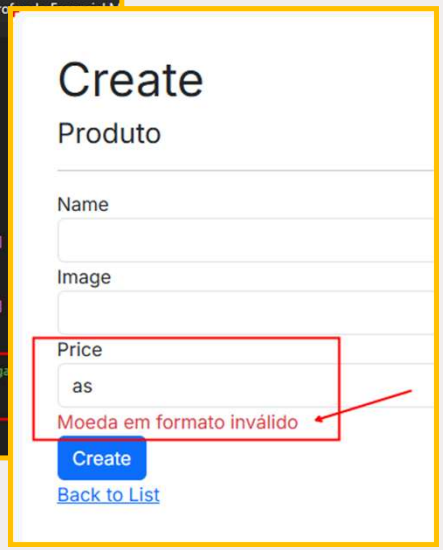
AprofundarEssencial.Mvc — AprofundarEssencial.Mvc.Configurations.DIConfig

```
namespace AprofundarEssencial.Mvc.Configurations
{
    0 referências
    public static class DIConfig
    {
        1 referência
        public static WebApplicationBuilder AddDependencyInjectionConfiguration(this WebApplicationBuilder builder)
        {
            builder.Services.AddSingleton<IHttpContextAccessor, HttpContextAccessor>();

            // Registos específicos para as interfaces específicas
            builder.Services.AddTransient<IGenerateGuidServiceTransient, GenerateGuidService>();
            builder.Services.AddScoped<IGenerateGuidServiceScoped, GenerateGuidService>();
            builder.Services.AddSingleton<IGenerateGuidServiceSingleton, GenerateGuidService>();

            // Registo do serviço que consome as três dependências
            builder.Services.AddTransient<OperacaoService>();

            builder.Services.AddSingleton<IValidationAttributeAdapterProvider, CoinValidationAttributeAdapterProvider>();

            builder.Services.AddDbContext<AppDbContext>(o =>
                o.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

            return builder;
        }
    }
}
```

Product.cs

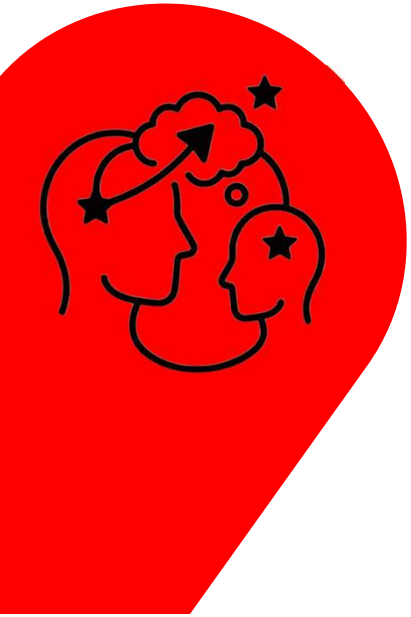
AprofundarEssencial.Mvc

```
using AprofundarEssencial.Mvc.Extensions;
ng System.ComponentModel.DataAnnotations;

namespace AprofundarEssencial.Mvc.Models
{
    13 referências
    public class Product
    {
        [Key]
        11 referências
        public int Id { get; set; }

        [Required(ErrorMessage = "O campo {0} é obrigatório")]
        12 referências
        public string? Name { get; set; }

        [Required(ErrorMessage = "O campo {0} é obrigatório")]
        11 referências
        public string? Image { get; set; }

        [Coin] //[Required(ErrorMessage = "O campo {0} é obriga
        12 referências
        public decimal Price { get; set; }
    }
}
```



GRUPO JAP

Adicionar Suporte para mais Culturas #1



_SelectLangu...artial.cshtml | Product.cs

C:\Onboarding\Formacao\ASPNET-Core-MVC-Avancado\AprofundarEssencial\src\AprofundarEssencial.Mvc\Views\Shared_SelectLanguagePartial.cshtml

```
@using Microsoft.AspNetCore.Builder
@using Microsoft.AspNetCore.Localization
@using Microsoft.AspNetCore.Mvc.Localization
@using Microsoft.Extensions.Options

@inject IViewLocalizer Localizer
@inject IOptions<RequestLocalizationOptions> LocOptions

@{
    var requestCulture = Context.Features.Get<IRequestCultureFeature>();
    var cultureItems = LocOptions.Value.SupportedUICultures
        .Select(c => new SelectListItem { Value = c.Name, Text = c.DisplayName })
        .ToList();
    var returnUrl = string.IsNullOrEmpty(Context.Request.Path) ? "~/" : $"~{Context.Request.Path.Value}";
}

<div title="@Localizer["Provedor da cultura do request:"] @requestCulture?.Provider?.GetType().Name">
    <form id="selectLanguage" asp-controller="Home" asp-action="SetLanguage" asp-route-returnUrl="@returnUrl"
          method="post" class="form-horizontal" role="form">
        <label asp-for="@requestCulture.RequestCulture.UICulture.Name">@Localizer["Idioma:"]</label>
        <select name="culture" onchange="this.form.submit();"
                asp-for="@requestCulture.RequestCulture.UICulture.Name" asp-items="cultureItems">
        </select>
    </form>
</div>
```

_LayoutAdmin.cshtml | HomeController.cs

AprofundarEssencial.Mvc | AspNetCoreGeneratedDocument.Views_Shared__LayoutAdmin

```
    <footer class="border-top footer text-muted mt-5">
        <div class="container text-center">
            <p class="mb-0">ASP.NET Core 10 MVC Avançado &copy; @DateTime.Now.Year</p>
            <small>Formação Profissional - Seu site feito do jeito certo!</small>
            @* <p class="mb-0">Idioma selecionado: @System.Globalization.CultureInfo.CurrentCulture.Name</p> *@
            <partial name="_SelectLanguagePartial" />
        </div>
    </footer>
```

HomeController.cs

AprofundarEssencial.Mvc | AprofundarEssencial.Mvc.Controllers.HomeC

```
3 referências
public class HomeController : Controller
{
    private readonly IConfiguration Configuration;
    private readonly ApiSetting ApiConfig;
    private readonly IStringLocalizer<HomeController> _localizer;

    0 referências
    public HomeController(IConfiguration configuration,
                          IOptions<ApiSetting> apiConfiguration,
                          IStringLocalizer<HomeController> localizer)
    {
        Configuration = configuration;
        ApiConfig = apiConfiguration.Value;
        _localizer = localizer;
    }

    0 referências
    public IActionResult Index()
    {
        var env = Environment.GetEnvironmentVariable("ASPNETCORE_ENVIRONMENT");


        // Access the configuration values directly from IConfiguration
        var myDomain_directly = Configuration[$"{ApiSetting.ConfigName}:Domain"] ?? "Default
        var userKey2_directly = Configuration[$"{ApiSetting.ConfigName}:UserKey"] ?? "Defaul
        var userSecret2_directly = Configuration[$"{ApiSetting.ConfigName}:UserSecret"] ?? "

        // Access the configuration values
        var apiConfig = new ApiSetting();
        Configuration.GetSection(ApiSetting.ConfigName).Bind(apiConfig);

        var myDomain = apiConfig.Domain ?? "DefaultDomain";
        var userKey = apiConfig.UserKey ?? "DefaultUserKey";
        var userSecret = apiConfig.UserSecret ?? "DefaultUserSecret";

        //throw new Exception("ALGO ERRADO NÃO ESTAVA CERTO !!! (lol)");

        return View();
    }

    [HttpPost]
    0 referências
    public IActionResult SetLanguage(string culture, string returnUrl)
    {
        Response.Cookies.Append(
            CookieRequestCultureProvider.DefaultCookieName,
            CookieRequestCultureProvider.MakeCookieValue(new RequestCulture(culture)),
            new CookieOptions { Expires = DateTimeOffset.UtcNow.AddYears(1) }
        );

        return LocalRedirect(returnUrl);
    }
```

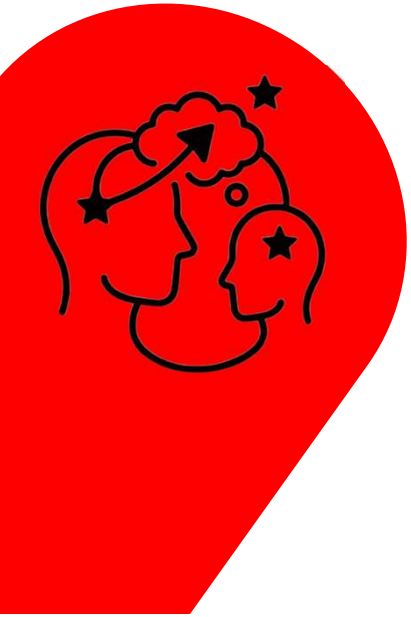
GlobalizationConfig.cs

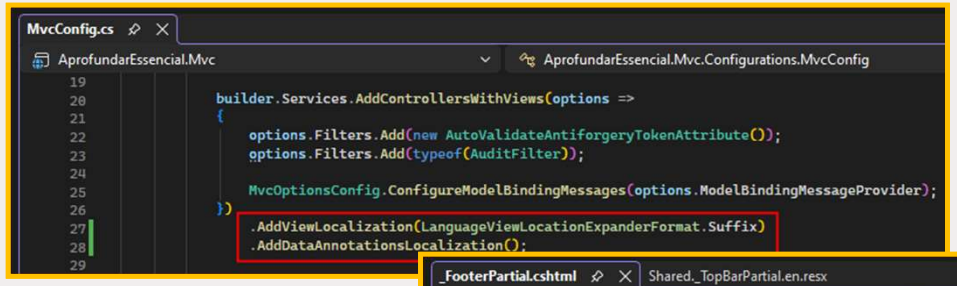
AprofundarEssencial.Mvc | AprofundarEssencial.Mvc.Configurations.GlobalizationConfig

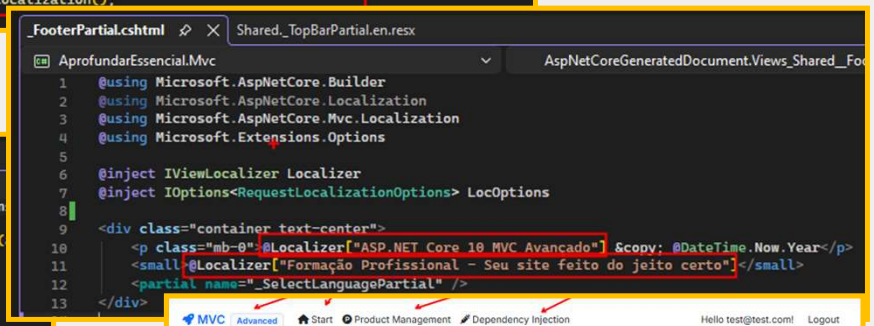
```
using Microsoft.Extensions.Options;

namespace AprofundarEssencial.Mvc.Configurations
{
    0 referências
    public static class GlobalizationConfig
    {
        //public static WebApplication UseGlobalizationConfig(this WebApplication app)
        //{
        //    var defaultCulture = new CultureInfo("pt-PT");

        //    var localizationOptions = new RequestLocalizationOptions
        //    {
        //        DefaultRequestCulture = new RequestCulture(defaultCulture),
        //        SupportedCultures = new List<CultureInfo> { defaultCulture },
        //        SupportedUICultures = new List<CultureInfo> { defaultCulture }
        //    };

        //    app.UseRequestLocalization(localizationOptions);

        //    return app;
        //}

        1 referência
        public static WebApplication UseGlobalizationConfig(this WebApplication app)
        {
            var localizationOptions = app.Services.GetService<IOptions<RequestLocalizationOptions>>();
            app.UseRequestLocalization(localizationOptions.Value);

            return app;
        }

        1 referência
        public static WebApplicationBuilder AddGlobalizationConfig(this WebApplicationBuilder builder)
        {
            builder.Services.AddLocalization(options => options.ResourcesPath = "Resources");

            builder.Services.Configure<RequestLocalizationOptions>(options =>
            {
                var supportedCultures = new[] { "en-EN", "pt-PT" };
                options.SetDefaultCulture(supportedCultures[0])
                    .AddSupportedCultures(supportedCultures)
                    .AddSupportedUICultures(supportedCultures);
            });

            return builder;
        }
    }
}
```

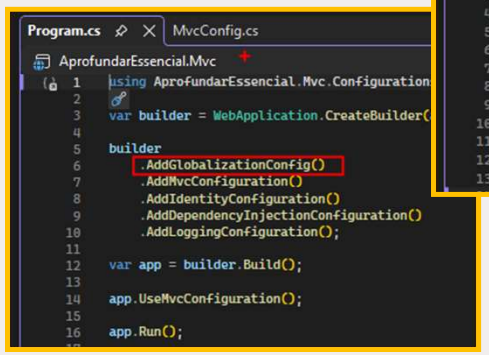
GRUPO JAP

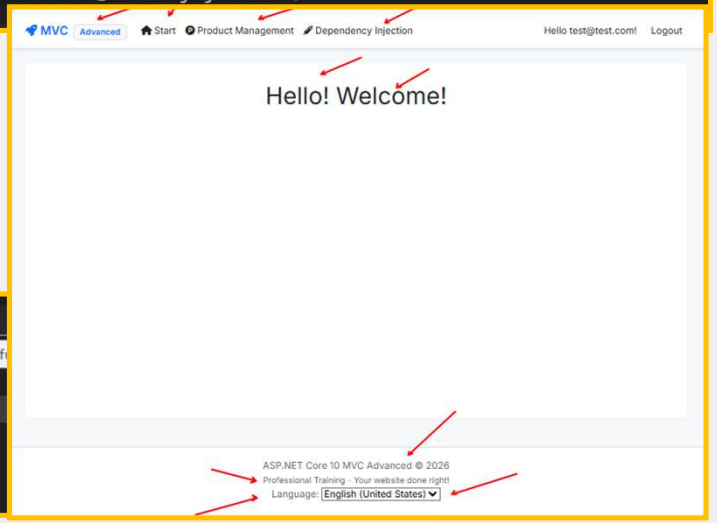
Adicionar Suporte para mais Culturas #2

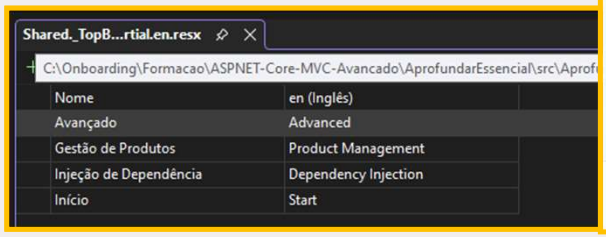


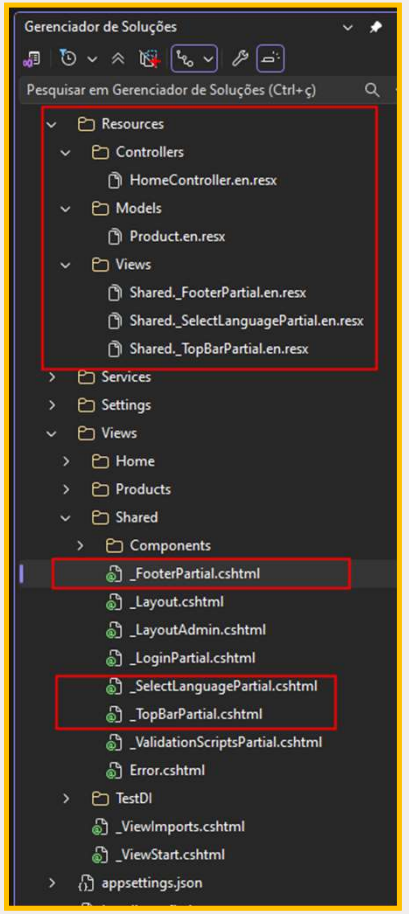






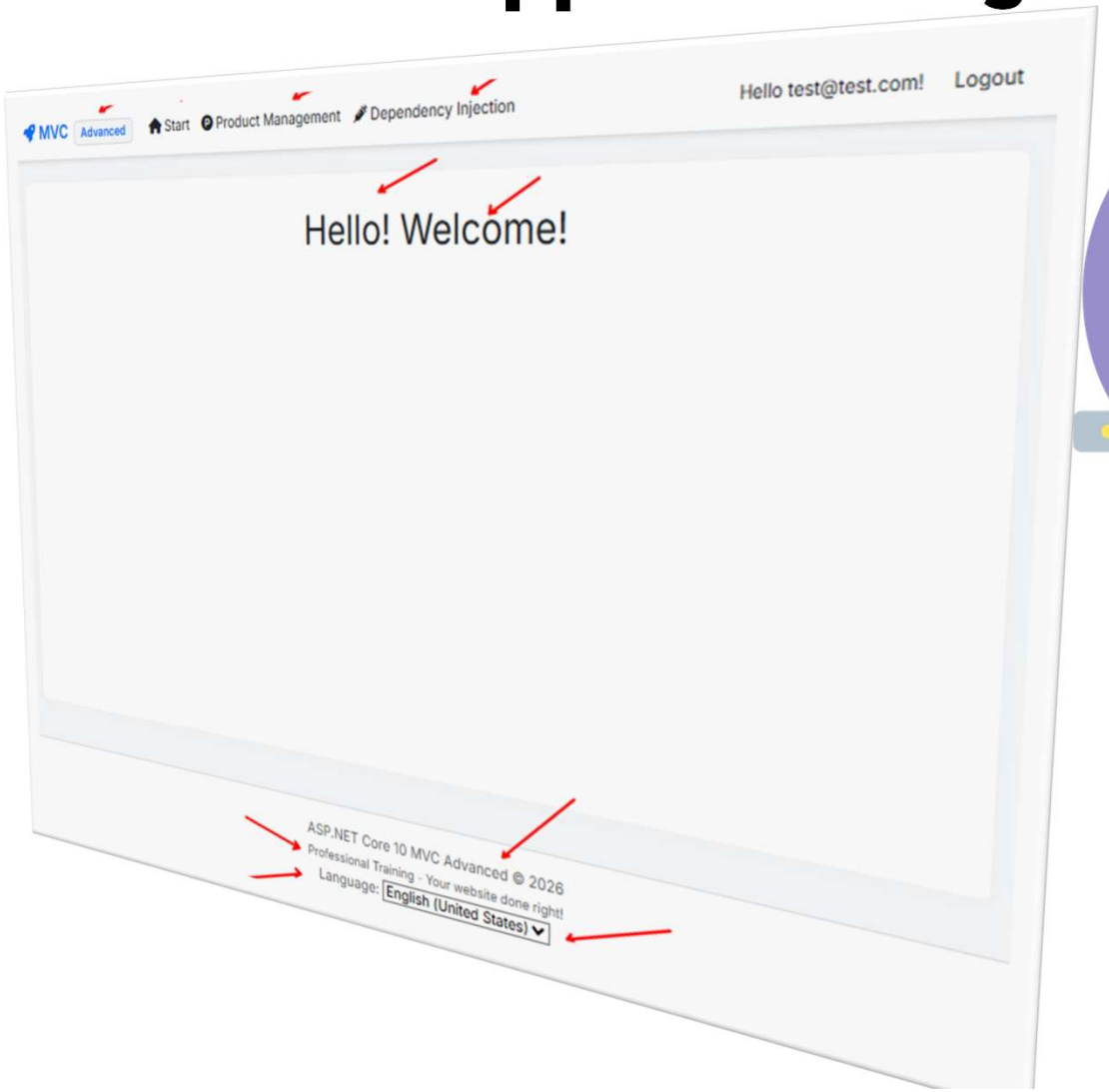


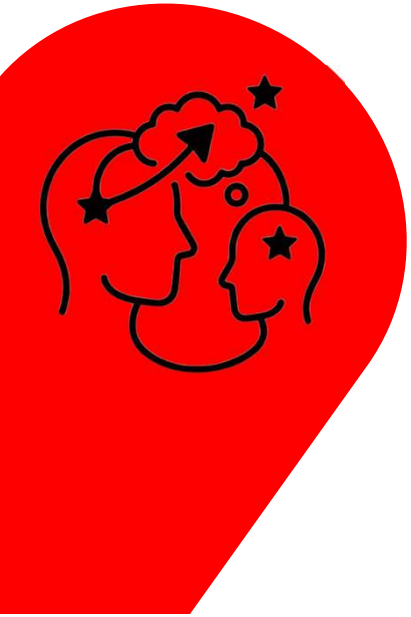




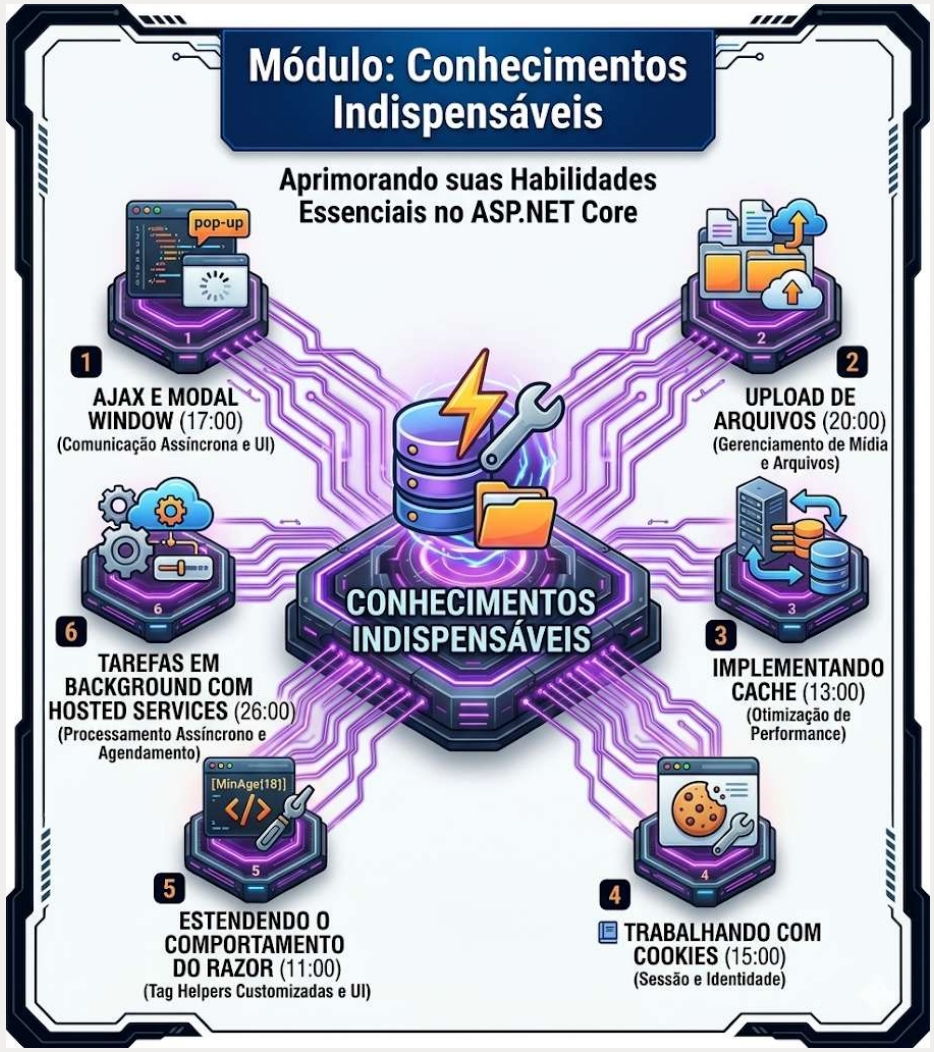
GRUPO JAP

[DEMO] - WebApp Multi-Linguagem





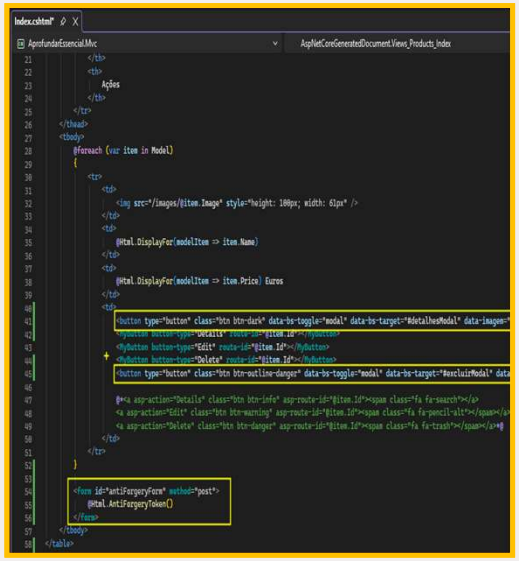
GRUPO JAP

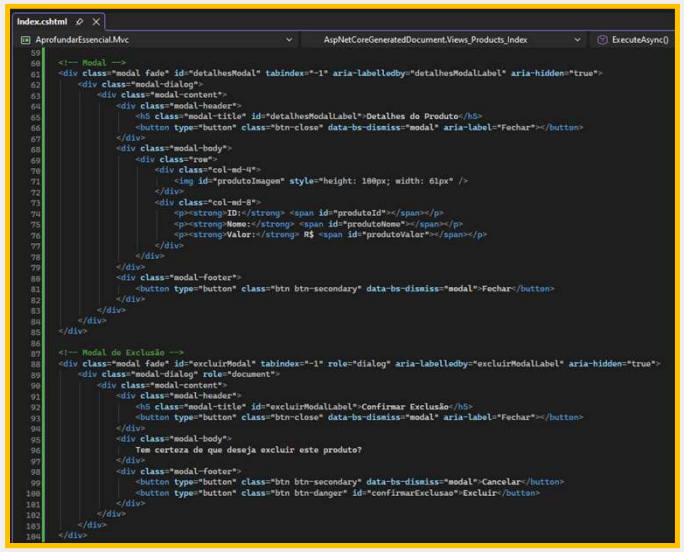

Módulo: Conhecimentos Indispensáveis
Aprimorando suas Habilidades Essenciais no ASP.NET Core
pop-up
1
AJAX E MODAL WINDOW (17:00)
(Comunicação Assíncrona e UI)
2
UPLOAD DE ARQUIVOS (20:00)
(Gerenciamento de Mídia e Arquivos)
CONHECIMENTOS INDISPENSÁVEIS
6
TAREFAS EM BACKGROUND COM HOSTED SERVICES (26:00)
(Processamento Assíncrono e Agendamento)
3
IMPLEMENTANDO CACHE (13:00)
(Otimização de Performance)
[MinAge(18)]
5
ESTENDENDO O COMPORTAMENTO DO RAZOR (11:00)
(Tag Helpers Customizadas e UI)
4
TRABALHANDO COM COOKIES (15:00)
(Sessão e Identidade)

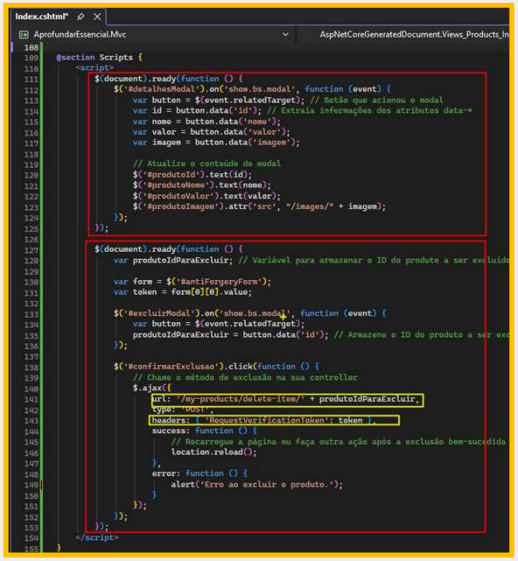
GRUPO JAP

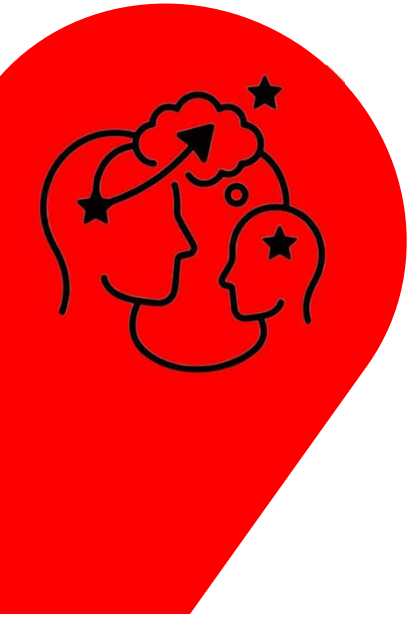
Trabalhar com Ajax e Modal Window

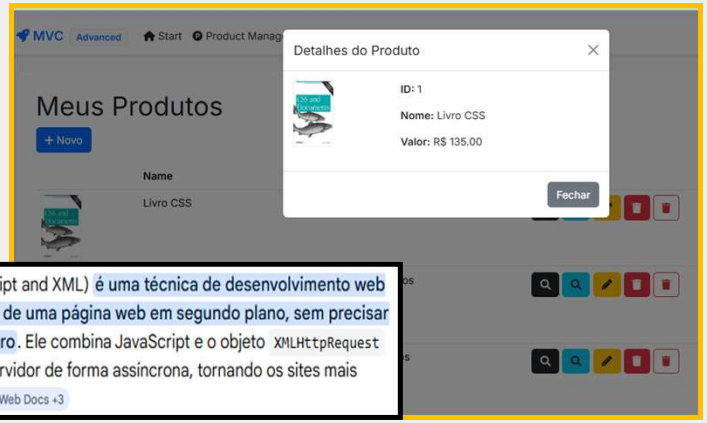
https://getbootstrap.com/docs/5.3/components/modal/#how-it-works



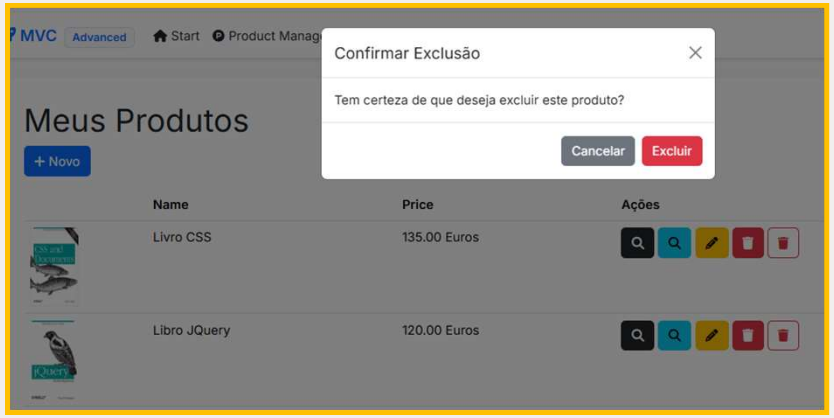






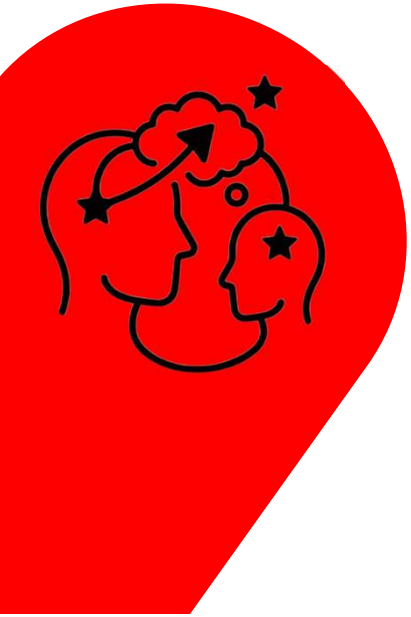


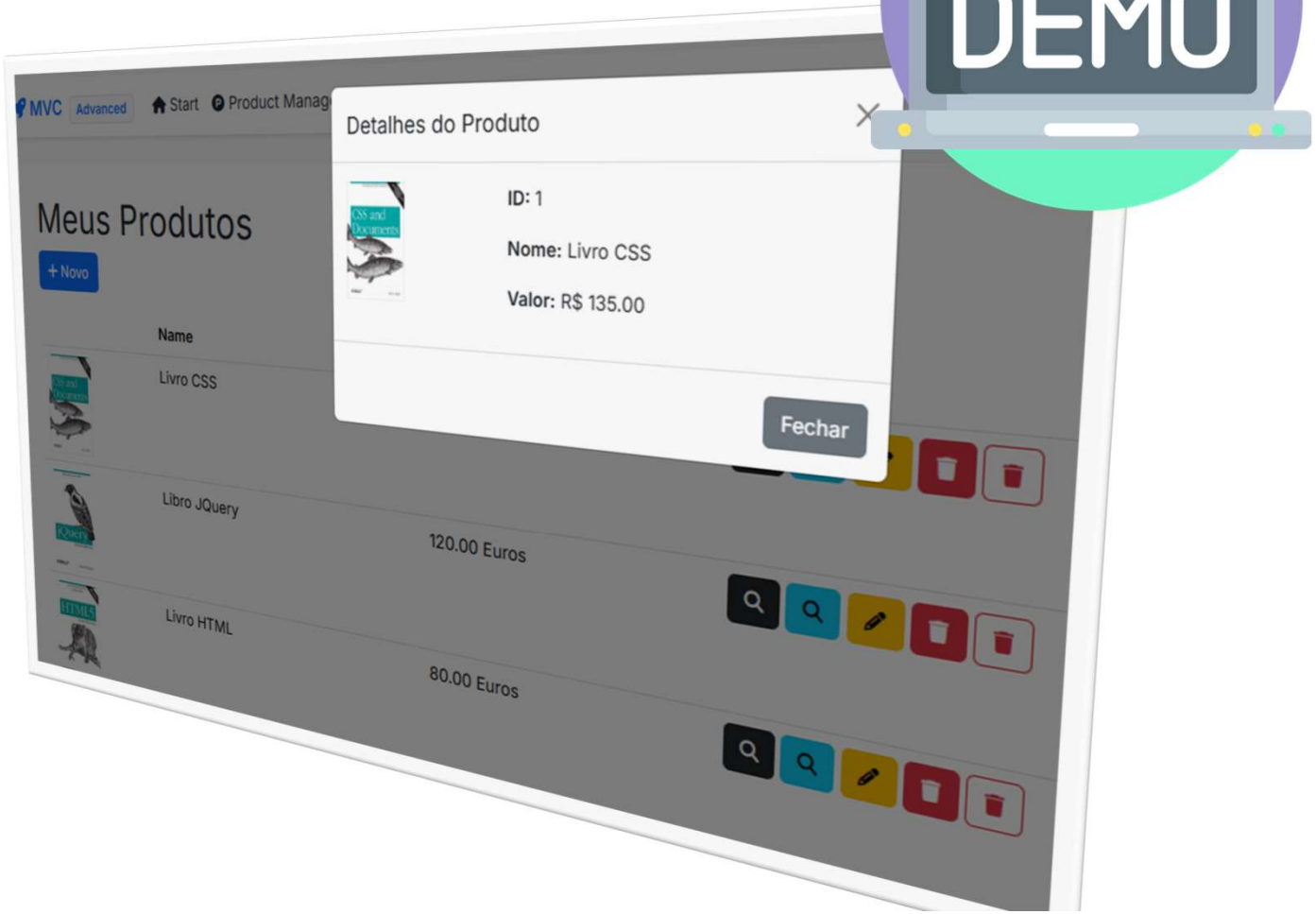
AJAX (Asynchronous JavaScript and XML) é uma técnica de desenvolvimento web que permite atualizar partes de uma página web em segundo plano, sem precisar recarregar o navegador inteiro. Ele combina JavaScript e o objeto `XMLHttpRequest` para trocar dados com um servidor de forma assíncrona, tornando os sites mais rápidos e responsivos. MDN Web Docs +3



GRUPO JAP

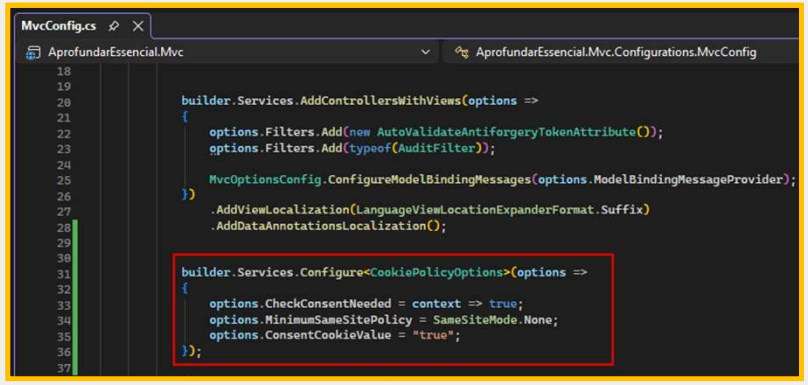
[DEMO] - Modal Window

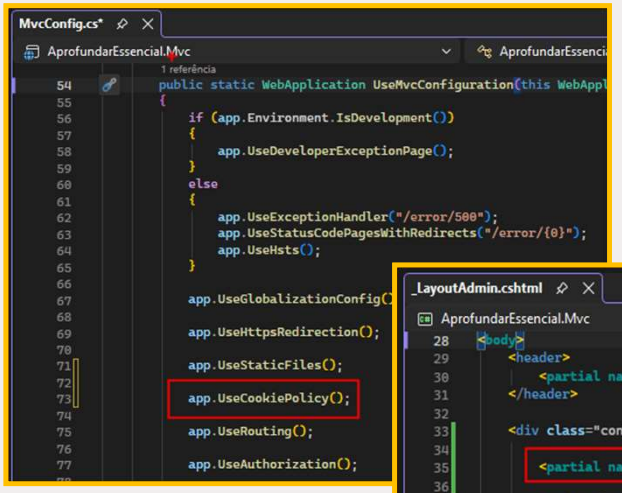


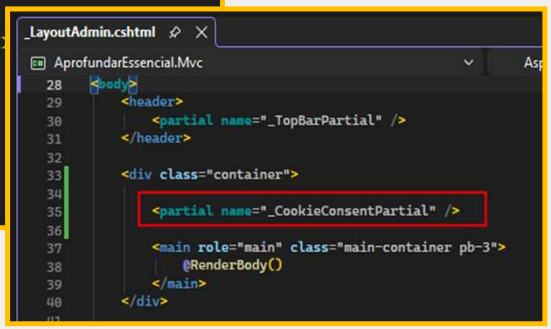


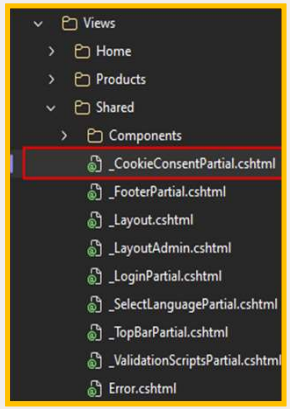
GRUPO JAP

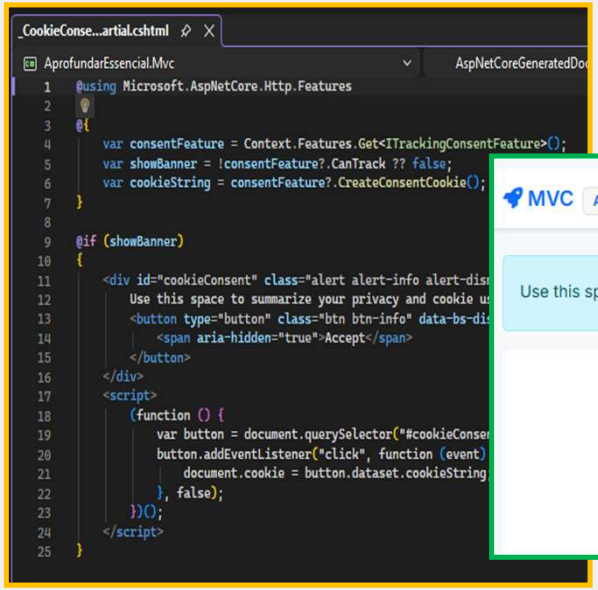
Políticas de Cookies

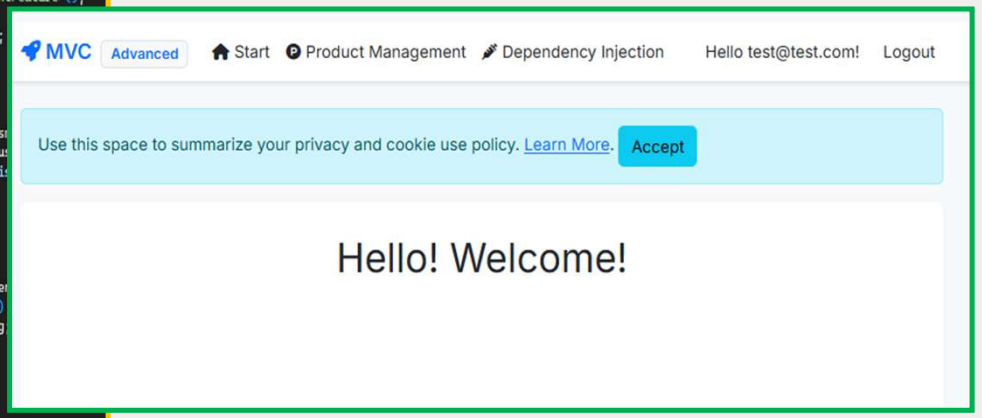






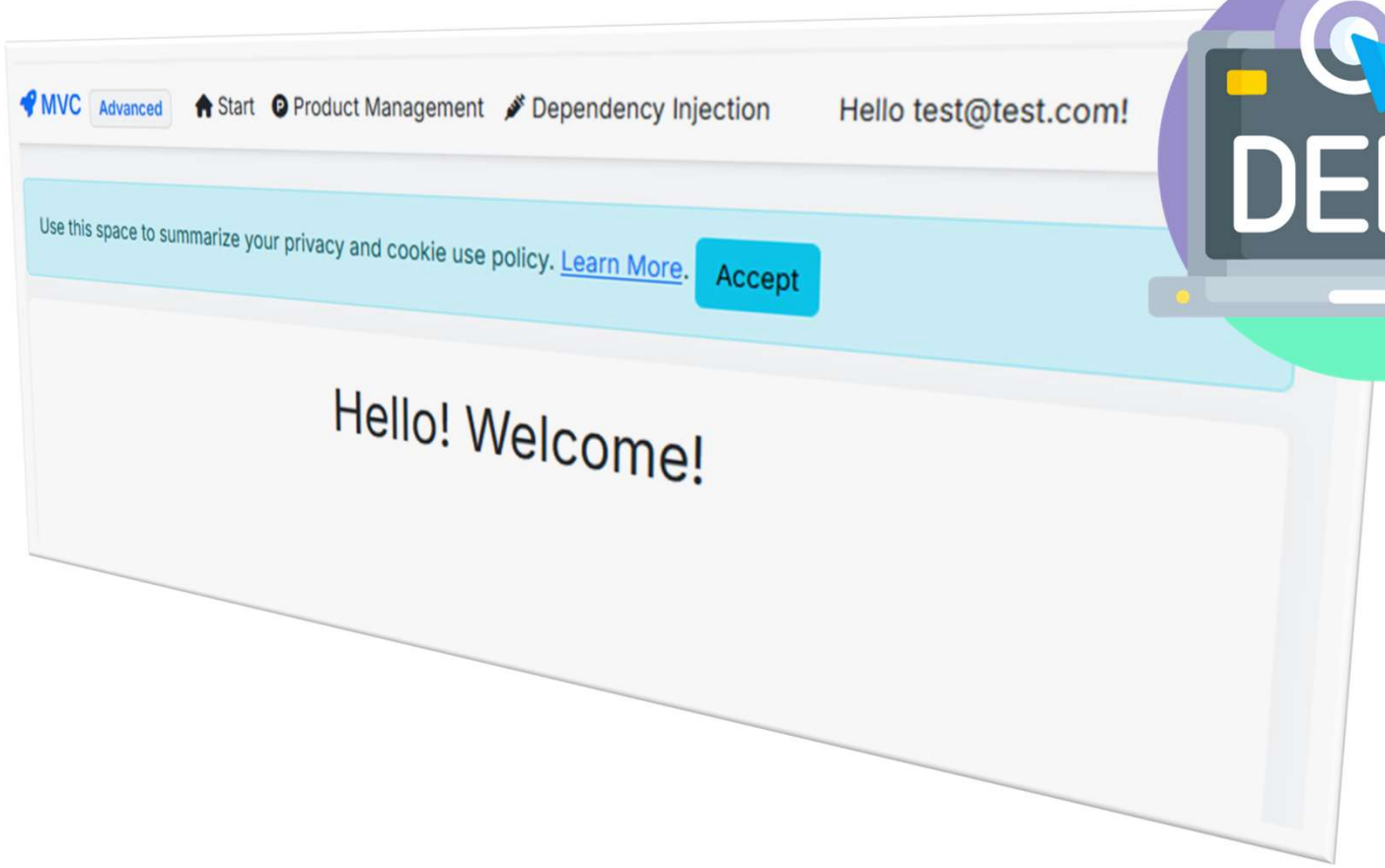


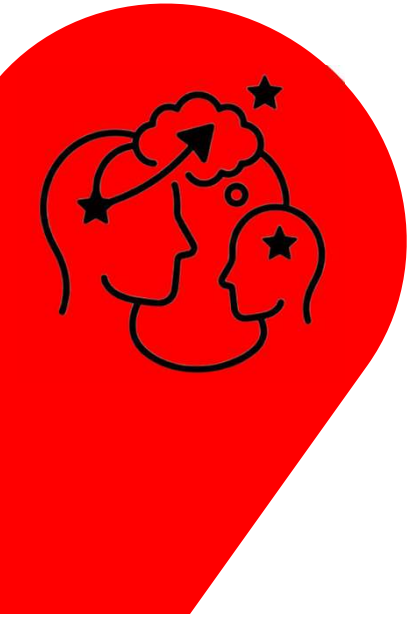




GRUPO JAP

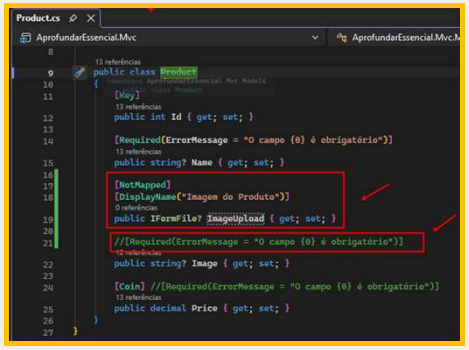
[DEMO] - Políticas de Cookie

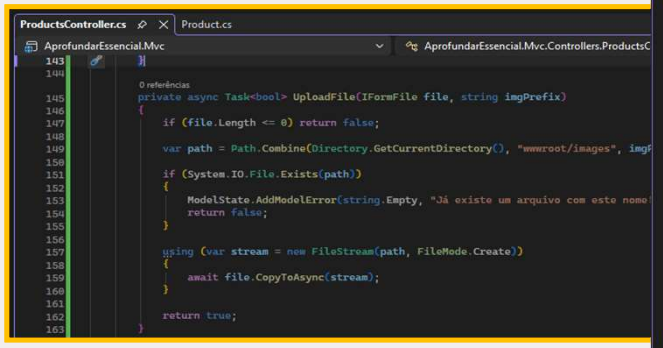


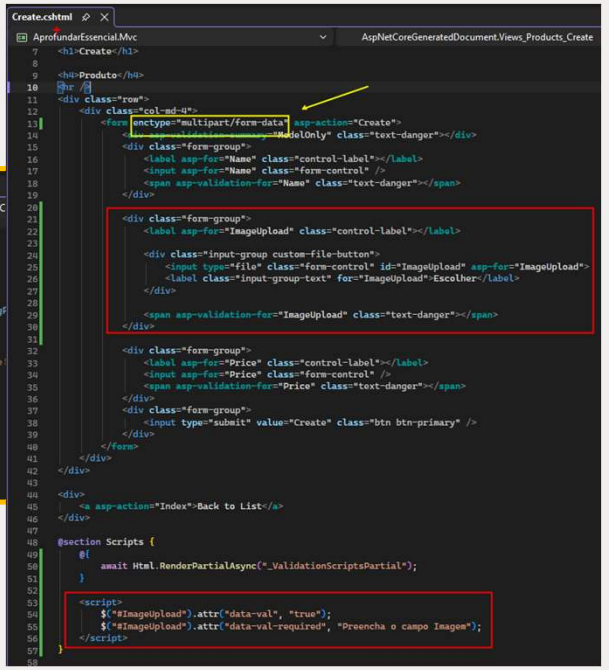


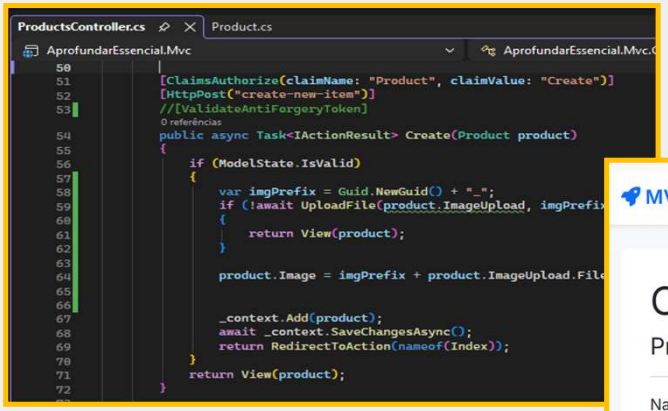
GRUPO JAP

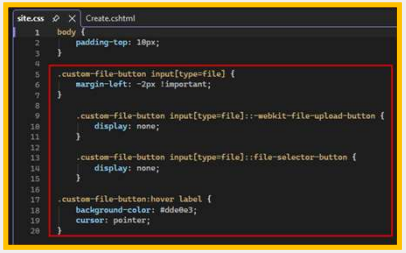
Upload de ficheiro (Create)

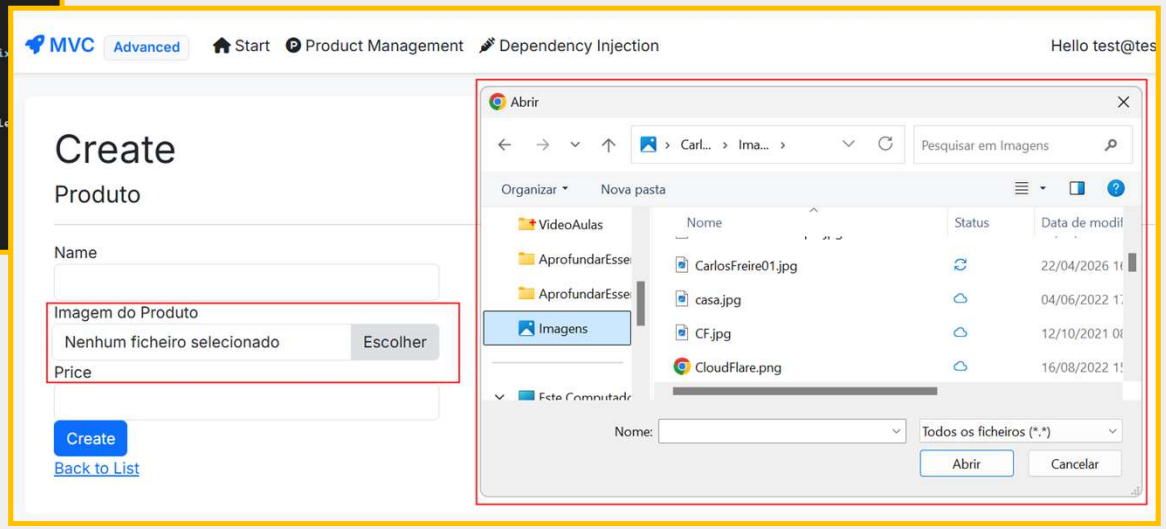






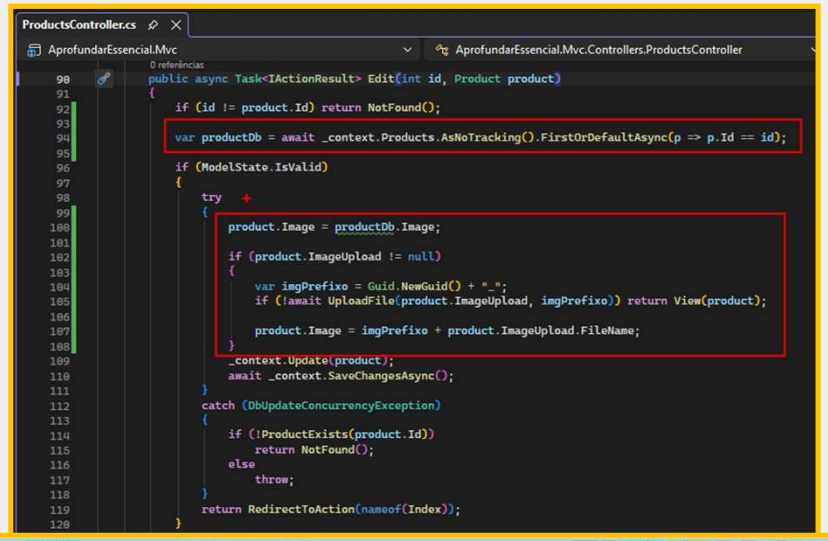


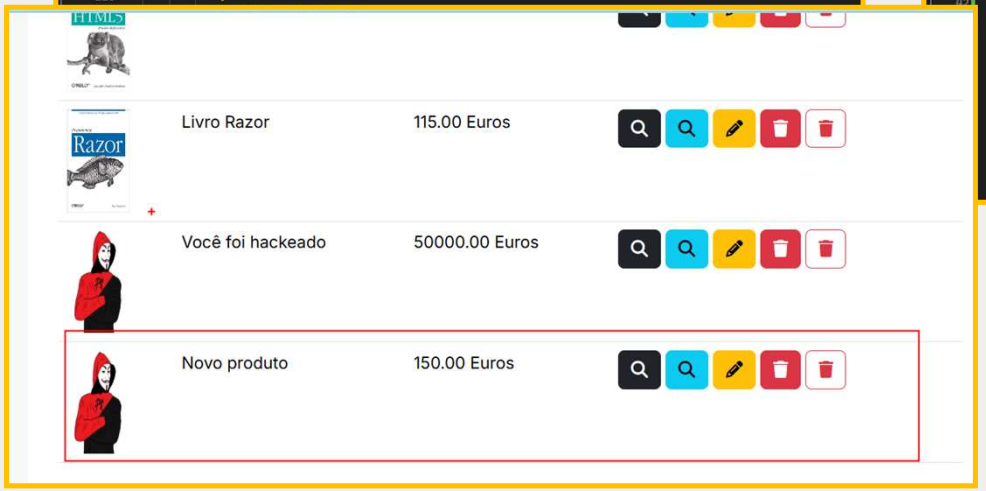


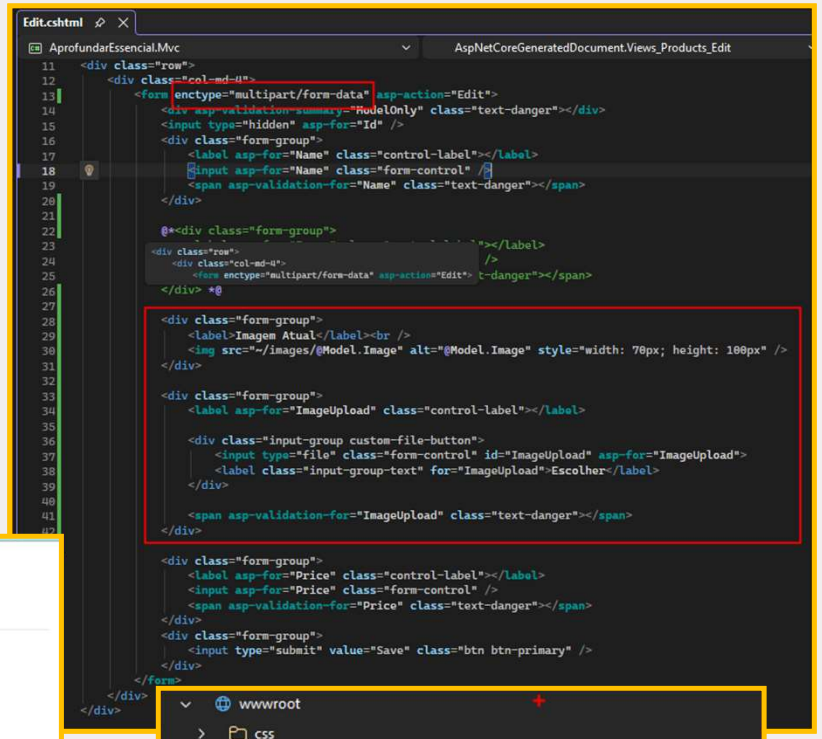


GRUPO JAP

Upload de ficheiro (Edit)

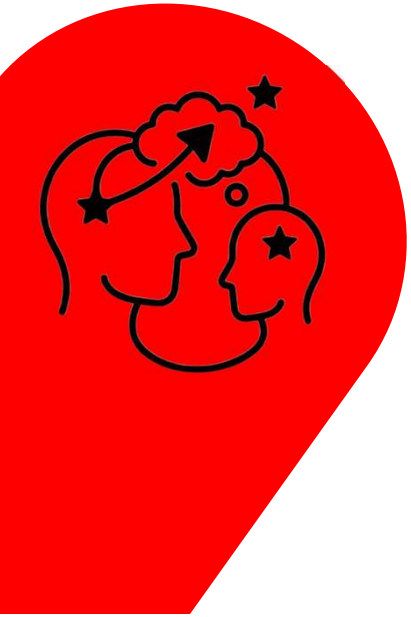






GRUPO JAP

[DEMO] - Upload de ficheiro



GRUPO JAP

Tarefas em Background Services

Background Services no .NET são tarefas executadas de forma assíncrona em segundo plano, essenciais para realizar operações contínuas, periódicas ou de longa duração (como envio de e-mails, processamento de filas ou relatórios) sem bloquear a interface do usuário ou o fluxo principal da aplicação web/API. Balta.io +1

```
PS > dotnet add package System.Drawing.Common --version 10.0.7
Ou
PMC  >  Install-Package System.Drawing.Common -Version 10.0.7
```

MvcConfig.cs | ImageWatermarkService.cs* | Edit.cshtml

AprofundarEssencial.Mvc

```
41
42
43          builder.Services.AddHsts(options =>
44          {
45              options.Preload = true;
46              options.IncludeSubDomains = true;
47              options.MaxAge = TimeSpan.FromDays(60);
48              options.ExcludedHosts.Add("example.com");
49              options.ExcludedHosts.Add("www.example.com");
50          });
51
52          builder.Services.AddHostedService<ImageWatermarkService>();
53
54          return builder;
55      }
```

ImageWatermarkService.cs

AprofundarEssencial.Mvc

AprofundarEssencial.Mvc.Services.Image

```
using System.Drawing;

namespace AprofundarEssencial.Mvc.Services
{
    2 referências
    public class ImageWatermarkService : BackgroundService
    {
        private readonly IServiceProvider _serviceProvider;
        private readonly IWebHostEnvironment _webHostEnvironment;
        private readonly string _watermarkPath;

        0 referências
        public ImageWatermarkService(IServiceProvider serviceProvider, IWebHostEnvironment webHostEnvironment)
        {
            _serviceProvider = serviceProvider;
            _webHostEnvironment = webHostEnvironment;
            _watermarkPath = Path.Combine(_webHostEnvironment.WebRootPath, "images/logo_watermark.png");
        }

        0 referências
        protected override async Task ExecuteAsync(CancellationToken stoppingToken)
        {
            while (!stoppingToken.IsCancellationRequested)
            {
                using (var scope = _serviceProvider.CreateScope())
                {
                    var context = scope.ServiceProvider.GetRequiredService<AppDbContext>();

                    var unprocessedProducts = await context.Products.Where(p => !p.Processed).ToListAsync();

                    foreach (var product in unprocessedProducts)
                    {
                        try
                        {
                            var imagePath = Path.Combine(_webHostEnvironment.WebRootPath, product.Image.TrimStart('/'));
                            var newImagePath = Path.Combine(_webHostEnvironment.WebRootPath, "images", $"{product.Id}_watermarked.jpg");

                            using (Bitmap watermarkImage = new Bitmap(_watermarkPath))
                            using (Bitmap imagemOriginal = new Bitmap(imagePath))
                            {
                                int newWidth = (int)(imagemOriginal.Width * 0.5);
                                int newHeight = (int)(imagemOriginal.Height * 0.5);

                                using (Bitmap resizedImage = new Bitmap(newWidth, newHeight))
                                using (var graphic = Graphics.FromImage(resizedImage))
                                {
                                    graphic.DrawImage(imagemOriginal, 0, 0, newWidth, newHeight);

                                    var point = new Point(resizedImage.Width - watermarkImage.Width - 10,
                                                          resizedImage.Height - watermarkImage.Height - 10);

                                    graphic.DrawImage(watermarkImage, point);

                                    resizedImage.Save(newImagePath, System.Drawing.Imaging.ImageFormat.Jpeg);
                                }
                            }

                            product.Processed = true;
                        }
                        catch (Exception ex)
                        {
                            SentrySdk.CaptureException(ex);
                        }
                    }

                    await context.SaveChangesAsync();
                }

                await Task.Delay(TimeSpan.FromMinutes(1), stoppingToken);
            }
        }
    }
}
```

Product.cs* | ImageWatermarkService.cs | Edit.cshtml

AprofundarEssencial.Mvc

```
using AprofundarEssencial.Mvc.Extensions;
using System.ComponentModel;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace AprofundarEssencial.Mvc.Models
{
    13 referências
    public class Product
    {
        [Key]
        14 referências
        public int Id { get; set; }

        [Required(ErrorMessage = "O ca
        13 referências
        public string? Name { get; set;

        [NotMapped]
        [DisplayName("Imagem do Produt
        11 referências
        public IFormFile? ImageUpload { get; set; }

        //[Required(ErrorMessage = "O campo {0} é obrigatório")]
        12 referências
        public string? Image { get; set; }

        [Coin] //[Required(ErrorMessage = "O campo {0} é obrigatório")]
        13 referências
        public decimal Price { get; set; }

        0 referências
        public bool Processed { get; set; }
    }
}
```

```
PS > dotnet ef migrations add ProcessedInProducts
ou
PMC > Add-Migrations ProcessedInProducts
e
PS > dotnet ef database update
ou
PMC > Update-Database
```

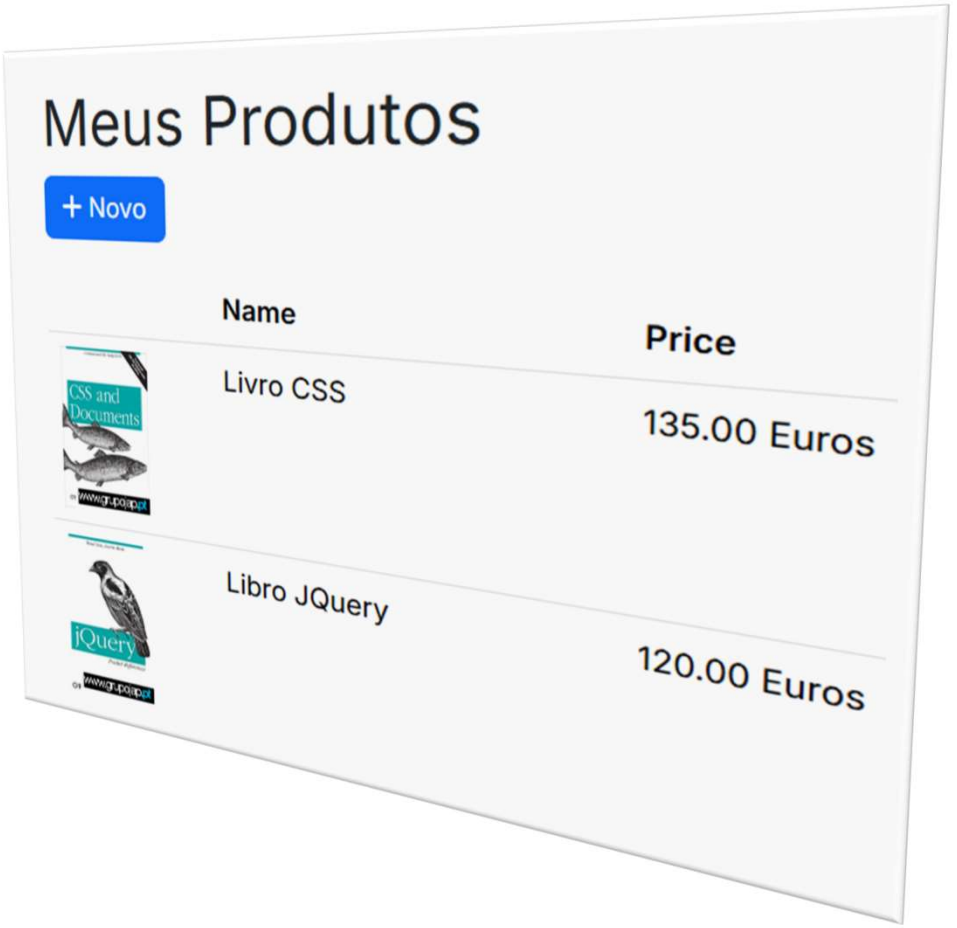
Meus Produtos

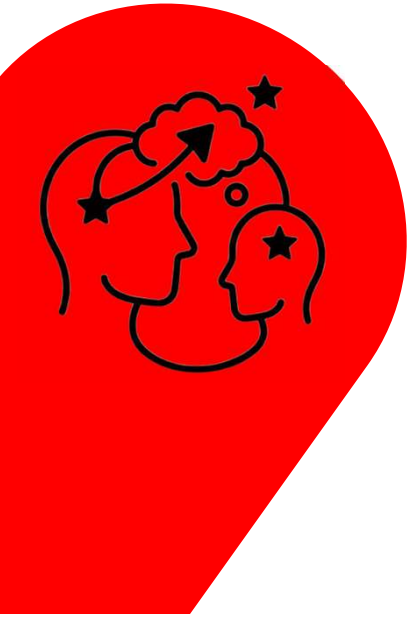
+ Novo

	Name	Price
	Livro CSS	135.00 Euros
	Libro JQuery	120.00 Euros

GRUPO JAP

[DEMO] - Adicionar Watermark





GRUPO JAP

Encerramento:

Com isso, encerramos a formação **ASP.NET Core MVC Avançado**. Passamos a perceber como funciona de modo avançado o MVC através de conceitos essências de front-end, aprofundamos em Injeção de Dependencia, Segurança com proteção a CSRF, XSS, HSTS e mais, aprendemos a gerir melhor os ambientes, a tratar melhor os erros e logar de forma inteligente, aprendemos a globalização da aplicação e trabalhar com mais de uma cultura e obtivemos conhecimentos indispensáveis para evoluir mais nosso nível de MVC. Mas entender o MVC é apenas parte necessária para domínio de desenvolvimento de software. Para construir aplicações mais complexas, convido vocês para o próximo nível:

Nossa quinta formação será **POO + SOLID + CLEAN CODE**.

Vamos afinar mais ainda nossos domínios em boas práticas de mercado.

Vejo vocês na próxima formação!